

Introduction to Numerical Methods using Python

Raghunathan Ramakrishnan
Tata Institute of Fundamental Research Hyderabad
Hyderabad, INDIA
ramakrishnan@tifrh.res.in

*“The purpose of computation is insight, not numbers”,
Richard Hamming, author of Numerical Methods for Scientists and Engineers, 1962 & 1972*

Content (tentative)

Topic		Page
1. Python	...	6
2. Errors and Big-O notation	...	36
3. Linear equations (direct methods)	...	51
4. Linear equations (iterative methods)	...	83
5. Eigenvalue problems	...	101
6. Rectangular problems	...	
7. Polynomial interpolation	...	
8. Function approximation	...	
9. Numerical derivatives	...	
10. Numerical integration	...	
11. Non-linear equations	...	
12. Optimization problems	...	
13. Initial value problems	...	

References

■ Main reference

- J. V. Lambers, A. S. Mooney, V. A. Montiforte, *Explorations in Numerical Analysis Python Edition*, World Scientific, First Indian Edition 2021

■ Further references for Numerical Methods

- W. Cheney, D. Kincaid, *Numerical Mathematics and Computing*, Cengage Learning, 2013
- S. D. Conte, C. De Boor, *Elementary Numerical Analysis An Algorithmic Approach*, McGraw-Hill International Editions, International Edition 1981
- D. G. Moursund, C. S. Duris, *Elementary Theory and Applications of Numerical Analysis*, Dover, 1988
- L. Elden, L. Wittmeyer-Koch, H. B. Nielsen, *Introduction to Numerical Computation*, Overseas Press, First Indian Edition 2006
- C. F. Van Loan, *Introduction to Scientific Computing*, Matlab Curriculum Series, 1997

■ Further references for Python

- K. Behrman, *Foundational Python for Data Science*, Pearson Education, 2023
- C. Dierbach, *Introduction to Computer Science using Python*, Wiley, 2013
- J. V. Guttag, *Introduction to Computation and Programming using Python*, MIT Press, 2017
- M. T. Goodrich, R. Tamassia, M. H. Goldwasser, *Data Structures and Algorithms in Python (An Indian Adaptation)*, Wiley, 2021

Working with Jupyter notebook

■ *Launching Jupyter via Anaconda*

- Open **Anaconda Navigator**
- Click **Launch** under **Jupyter Notebook**
- A browser window will open showing your file system.
- Navigate to your working directory. On the right side of the screen, click the **New** button and select **Python 3 (ipykernel)** from the dropdown menu. This opens a new notebook.

■ *Installing and launching Jupyter with pip*

- Open a terminal and run: **pip install notebook**
- Once installed, launch Jupyter with: **jupyter notebook**

■ *Jupyter notebook opens in a web browser*

- When you launch Jupyter Notebook, it opens in your *default web browser* (e.g., Chrome, Firefox, Safari) as a tab or window. This might feel unusual at first, but don't worry, the browser is just the interface. You're not working online unless you're using a cloud service like Google Colab; your files and code stay on your local computer.

■ *Anatomy of a notebook*

- A Jupyter Notebook is organized into **cells**:
- **Code cells**: Where you write and run Python code
- **Markdown cells**: Where you write explanations, equations, and documentation

1. Python

Know your system

■ Python version, OS and architecture

```
import sys

print("Python version:", sys.version)
print("64-bit:", sys.maxsize == 2**63-1) # True if Python is running in 64-bit mode
```

```
Python version: 3.11.6 (main, Oct 2 2023, 20:46:14) [Clang 14.0.3 (clang-1403.0.22.14.1)]
64-bit: True
```

```
import platform

print("OS:", platform.system(), platform.release()) # Operating system
print("Architecture:", platform.machine()) # 'arm64' for ARM-based system, 'x86_64' for Intel/AMD
```

```
OS: Darwin 24.1.0
Architecture: arm64
```

■ Learn to pip install

- For the next exercise, we will need the module **psutil**.
- You can check if your Python installation contains this module as follows

```
%pip show psutil
```

```
Name: psutil
Version: 6.0.0
Summary: Cross-platform lib for process and system monitoring in Python
```

- If **psutil** is not installed in your computer, you will receive the following message

```
WARNING: Package(s) not found: psutil
```

```
Note: you may need to restart the kernel to use updated packages.
```

- You can install **psutil** directly from your Jupyter notebook as follows

```
%pip install psutil
```

■ Disk space and RAM availability

```
import shutil

total, used, free = shutil.disk_usage("/") # disk space of the root filesystem

print(f"Total: {total / 1e9:.2f} GB")
print(f"Used : {used / 1e9:.2f} GB")
print(f"Free : {free / 1e9:.2f} GB")

Total: 245.11 GB
Used : 211.00 GB
Free : 34.11 GB
```

```
import psutil

mem = psutil.virtual_memory() # total and available system RAM

print(f"Total RAM: {mem.total / 1e9:.2f} GB")
print(f"Available: {mem.available / 1e9:.2f} GB")
print(f"Used: {mem.used / 1e9:.2f} GB")
print(f"Percent used: {mem.percent}%")

Total RAM: 8.59 GB
Available: 1.24 GB
Used: 3.13 GB
Percent used: 85.6%
```

■ Use f-strings for printing

```
total, used, free = shutil.disk_usage("/")
print(total)
print(f"Total: {total / 1e9:.2f} GB")
print(f"Total: {total / 1e9:.4f} GB")

245107195904
Total: 245.11 GB
Total: 245.1072 GB
```

■ The dot (.) is used to access data (attributes) or functions (methods) inside modules and objects. You will encounter the following

- module.function()
 - module.submodule.function()
 - module.attribute
 - object.function()
 - object.attribute
- Total RAM does not equal Available + Used because some memory is held in buffers and caches, which are used for fast data transfer and reuse and can be reclaimed by the system when needed.

- NOTE: The expression `{total/1e9:.2f}` shows how f-strings allow us to compute a value and control its numeric formatting at the same time.

Data types

■ Basic datatypes and memory requirement

```
a = 10
b = 123456789012345678901234567890

print(sys.getsizeof(a)) # int, 28 byte
print(sys.getsizeof(b)) # int, 40 byte
```

28
40

```
a = 3.14
b = 3.1415926535897932384626433

print(sys.getsizeof(a)) # float, 24 byte
print(sys.getsizeof(b)) # float, 24 byte
```

```
fmin, fmax = sys.float_info.min, sys.float_info.max # smallest and largest (normalized) finite float
print(fmin, fmax)
```

24
24
2.2250738585072014e-308 1.7976931348623157e+308

```
a = ""
b = "h"
c = "he"

print(sys.getsizeof(a)) # string
print(sys.getsizeof(b))
print(sys.getsizeof(c))
```

49
50
51

- Python's default memory handling for integers and floats is not straightforward. Integers can grow arbitrarily large and consume increasing memory. In numerical computations, we will instead use **numpy** objects, which offer fixed-size numeric types and predictable memory usage.

- This is an example of multiple assignment, where Python unpacks values on the right-hand side into separate variables on the left. Try

```
a, b = 1, 2
print(a+b)
```

- NOTE: this is 10^{308}

■ Numeric / Sequence / Mapping / Boolean

- o Integer, float, complex / string, list, tuple / dictionary / bool (True, False)

```
a = 5
print(a, type(a))

b = 5.0
print(b, type(b))

c = 2 + 4j
print(c, type(c))

d = complex(2,4)
print(d, type(d))

e = 'One is 1'
print(e, type(e))
```

```
5 <class 'int'>
5.0 <class 'float'>
(2+4j) <class 'complex'>
(2+4j) <class 'complex'>
One is 1 <class 'str'>
```

```
x = 8

print(x > 5 and x < 10)    # True
print(x < 5 or x > 10)     # False
print(not (x == 8))        # False
```

```
TruthValue = 8 < 7
print(type(TruthValue))
```

```
True
False
False
<class 'bool'>
```

■ Dictionary

- o To store key/value pairs

```
params={'mu':1.0, 'sigma':0.5}
print(params, type(params))

{'mu': 1.0, 'sigma': 0.5} <class 'dict'>
```

```
print(len(params))
print(params.keys())
print(params.values())
```

```
2
dict_keys(['mu', 'sigma'])
dict_values([1.0, 0.5])
```

```
params.update({'scal':1.0})
print(params)

{'mu': 1.0, 'sigma': 0.5, 'scal': 1.0}
```

```
params['scal'] # query by key returns the value

1.0
```

String manipulation

■ *Splicing, splitting*

```
mystr='A shadow grows on the wall behind you'
a=mystr[0]
b=mystr[0:3] # slice from index 0 up to (but not including) index 3
print(a,",",b)
```

A , A s

```
mystr='1,2,3,4'
a=mystr.split(',') # output is a list
b=mystr.split(',')[0:2] # first two entries of the list
print(a,b)
mystr='ATAAATCT'
sub_seqs=mystr.split('T')
print(sub_seqs)
```

```
['1', '2', '3', '4'] ['1', '2']
['A', 'AAA', 'C', '']
```

```
print(len(sub_seqs), type(sub_seqs))
```

4 <class 'list'>

```
sub_seqs[0]+sub_seqs[1]+sub_seqs[2] # '+' is for concatenation or combining
```

'AAAAC'

```
N=len(sub_seqs)
for i in range(0,N):
    print(i,sub_seqs[i])    # NOTE: 4 empty spaces for indentation
```

```
0 A
1 AAA
2 C
3
```

```
mystr='ATAAATCT'
mystr[2:2]='G' # Strings in Python are immutable
print(mystr)
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[94], line 2
      1 mystr='ATAAATCT'
----> 2 mystr[2:2]='G' # Strings in Python are immutable
      3 print(mystr)
```

TypeError: 'str' object does not support item assignment

List

```
string='Stranger,Things'
sub_str=string.split(',') # This is a list with 2 elements
```

```
print(sub_str, type(sub_str), len(sub_str))
```

```
['Stranger', 'Things'] <class 'list'> 2
```

```
A=[1,2,3] # list of integers
B=[1,'one','1.0'] # list can contain different datatypes
print(A[0:2]) # last index, excluded
```

```
[1, 2]
```

```
# String concatenation
str1='Friends'
str2='don\'t'
str3='lie'
print(str1+str2+str3) # Concatenate with +
print(str1+' '+str2+' '+str3)
```

Escape sequence for special characters

```
Friendsdon'tlie
Friends don't lie
```

```
# Append list
str4=[str1,str2]
str5=[str3]
str4+str5 # + for appending a list to another
```

```
['Friends', 'don't', 'lie']
```

```
t = (1, 2, 3) # a tuple is like a list, but immutable
t[1]=4
```

- Because tuples are immutable, they are often used to store function arguments and return multiple values safely.

■ Always deep copy a list

```
A=[1,2,3,4]
```

```
B=A # is a shallow copy
print("List A is:", A, "List B is:", B)
```

```
B[2]=7 # change a value in B, which also affects A
print("List A is:", A, "List B is:", B)
```

```
List A is: [1, 2, 3, 4] List B is: [1, 2, 3, 4]
List A is: [1, 2, 7, 4] List B is: [1, 2, 7, 4]
```

```
A=[1,2,3,4]
```

```
B=A.copy() # is a deep copy
print("List A is:", A, "List B is:", B)
```

```
B[2]=7 # change a value in B, which does not affect A
print("List A is:", A, "List B is:", B)
```

```
List A is: [1, 2, 3, 4] List B is: [1, 2, 3, 4]
List A is: [1, 2, 3, 4] List B is: [1, 2, 7, 4]
```

```
# assigning a slice works differently
A=[1,2,3,4]; B=A[0:3] # deep copy
B[2]=7; print("List A is:", A, "List B is:", B)
```

```
A=[1,2,3,4]; B=A[0:3].copy()
B[2]=7; print("List A is:", A, "List B is:", B)
```

```
import numpy as np
A=np.array([1,2,3,4]); B=A[0:3] # shallow copy
B[2]=7; print("Array A is:", A, "Array B is:", B)
```

```
A=np.array([1,2,3,4]); B=A[0:3].copy()
B[2]=7; print("Array A is:", A, "Array B is:", B)
```

Numpy

■ Numpy arrays

- Try the following cell-by-cell

```
import numpy as np
```

```
A=[[1,6,3],[4,5,7]]; A=np.array(A); print(A) # multiple statements in one line
```

```
print(A.shape)
```

```
Nrow, Ncol=A.shape[0],A.shape[1]; print(Nrow, Ncol)
```

```
print(A.T, '\n', '-'*10, '\n', np.transpose(A))
```

```
max_inrow=np.max(A,axis=1); print(max_inrow)
```

```
max_incol=np.max(A,axis=0); print(max_incol)
```

```
maxval=np.max(A); print(maxval)
```

```
a=np.array([2,3,1,4,6,1,3,2,0])  
a_sorted = np.sort(a); a_sorted_rev = np.sort(a)[::-1]  
print(a_sorted, a_sorted_rev)
```

```
a_sorted = np.argsort(a)  
a_sorted_rev = np.argsort(a)[::-1]  
print(a_sorted, a_sorted_rev)
```

```
a=np.array([2,3,1,4,6,1,3,2,0]); print(len(a))
```

```
ind_gt3 = np.where(a>3); ind_le2 = np.where(a<=2); nz_ind = np.nonzero(a)  
print(ind_gt3, ind_le2, nz_ind)
```

■ Merge and split arrays

- Try the following cell-by-cell

```
A = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])  
nrow, ncol = 2, 6; B = np.reshape(A, (nrow, ncol)); print(B)
```

```
A = np.array([1, 2, 3, 4]); B = np.array([5, 6, 7, 8])  
C = np.hstack((A,B)); print(C)  
D = np.vstack((A,B)); print(D)
```

```
C = np.concatenate([A, B], axis=0); print(C)
```

```
A_T=A.reshape(-1, 1); B_T=B.reshape(-1, 1)  
print(A_T); print(B_T)  
D = np.concatenate([A_T, B_T], axis=0); print(D)  
E = np.concatenate([A_T, B_T], axis=1); print(E)
```

```
A = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
```

```
sub_arrays = np.split(A, 2); print(sub_arrays) # split the 1D array into 2 equal parts
```

```
sub_arrays = np.split(A, 3); print(sub_arrays) # split the 1D array into 3 equal parts
```

```
sub_arrays = np.split(A, 2, axis=0)  
print(sub_arrays)
```

```
A = np.array([[1, 2, 3, 4, 5, 6,], [ 7, 8, 9, 10, 11, 12]])  
sub_arrays = np.split(A, 2) # split into 2 equal parts along axis 0 (row-wise)  
print(sub_arrays)
```

```
sub_arrays = np.split(A, 2, axis=1) # split into 2 equal parts along axis 1 (column-wise)  
print(sub_arrays)
```

■ Creating (or initializing) numpy arrays

- o Try the following cell-by-cell

```
x = np.zeros(3); print(x)
```

```
x = np.ones(3); print(x)
```

```
x = np.zeros((4,3)); print(x)
```

```
x = np.zeros(3, dtype='int'); print(x)
```

```
x = np.zeros(3, dtype='float'); print(x)
```

```
# create an array of given size with evenly spaced values  
xi, xf, N_x = 0, 1, 10 # initial, final, number  
x = np.linspace(xi, xf, N_x); print(x)
```

```
# creates an array with evenly spaced values of given step size  
xi, xf, dx = 0, 1, 0.1 # initial, final, step  
x = np.arange(xi, xf, dx); print(x)
```

```
# array with uniformly sampled random numbers  
x_rand1 = np.random.rand(2); print(x_rand1, '\n', '-'*20)  
x_rand2 = np.random.rand(2,2); print(x_rand2, '\n', '-'*20)  
x_rand2 = np.random.rand(2,2,2); print(x_rand1, '\n', '-'*20)
```

```
np.random.choice(x_rand1) # choice method gets one random value from a 1D array
```

```
x = np.zeros(3, dtype='int16'); print(x) # 16-bit (2-byte) per element array with zeros
```

```
print(np.iinfo(np.int16).min, np.iinfo(np.int16).max) # -2**15, 2**15-1
```

Exercises

■ *Guess the output of the following code blocks and check your answers by trying them*

```
A = [10, 20, 30, 40]
A[0], A[3] = A[3], A[0]
print(A)
```

```
A = [10, 20, 30, 40, 50]; B = A[len(A)//2] # // means integer (floor) division
print(B)
```

```
A = [10, 20, 30, 40]; A = A[1:] + A[:1]
print(A)
```

```
A = [10, 20, 30, 40]
A[0] = A[0] + 5
A[-1] = A[-1] * 2 # index -1 refers to the last element
print(A)
```

```
A = [1, 2, 3]; B = [1, 2, 3]
print(A == B)
```

```
A = [10, 20, 20, 30]
print(A.count(20))
```

```
print("-" * 30)
```

Conditional expressions

- *A conditional expression is an expression that evaluates to either True or False:* Such expressions are formed using relational operators and may be combined using logical operators.

- *Relational operators for comparison*

```
a=1; b=2; val1, val2, val3, val4, val5, val6 = a==b, a!=b, a<b, a<=b, a>b, a>=b
print(val1, val2, val3, val4, val5, val6)
```

False True True True False False

- *Logical operators for combining scalar boolean values*

```
val1 = True and True; val2 = True and False; val3 = False and False # AND
print(val1, val2, val3)
val1 = True or True; val2 = True or False; val3 = False or False # OR
print(val1, val2, val3)
val1 = not True; # NOT
print(val1)
val1 = True ^ True; val2 = True ^ False; val3 = False ^ False # XOR
print(val1, val2, val3)
```

True False False
True True False
False
False True False

o Note: 12^2 does bitwise XOR and not the same as $12^{**}2$

```
x=np.array([0.3,0.4,0.1])
y=np.array([0.6,0.3,0.7])
(x < 1/2) | (y > 1/2)

array([ True,  True,  True])
```

- *Logical operators for combining boolean arrays*

```
a = np.array([True, False, True]); b = np.array([True, True, True]); val1 = a & b
print(val1)
a = np.array([True, False, True]); b = np.array([True, True, True]); val1 = a | b
print(val1)
```

[True False True]
[True True True]

- *Use and (and never use &) for scalar comparison*

```
val1 = 1 < 2 and 1<3 and 1<4
val2 = (1 < 2) & (1<3 ) & (1<4)
val3 = 1 < 2 & 1 < 3 & 1 < 4 # Unexpected result
print(val1, val2, val3)
```

True True False

if conditional branching

- o The **if** statement executes different blocks of code depending on whether a condition is True or False.

```
a = 5; COND_EXPR = a % 2 == 0
```

```
if COND_EXPR:
    print('a is even')
else:
    print('a is odd')
```

```
a = 5
```

```
if a % 2 == 0:
    print('a is even')
else:
    print('a is odd')
```

```
a = 50
```

```
if a < 0:
    print('a is -ve')
elif a > 0:
    print('a is +ve')
else:
    print('a is 0')
```

```
x = -7
```

```
if x < 0:
    x = -x
print(x)
```

```
a = 10
b = 25
```

```
if a > b:
    max_val = a
else:
    max_val = b
print(max_val)
```

```
year = 2100
```

```
if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
    print("Leap year")
else:
    print("Not a leap year")
```

- *Guess the output of the following code block and check your answers by trying*

```
x = 5
```

```
if x > 0:
    print("A")
elif x > 3:
    print("B")
else:
    print("C")
```

for loop

- o A **for** loop repeats a block of code once for each element in a sequence.

```
list(range(10, -1, -1))
```

```
[10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

```
A=list(range(0, 10, 2)); B=list(range(0, 10)); C=list(range(10)); print(A,B,C)
```

```
[0, 2, 4, 6, 8] [0, 1, 2, 3, 4, 5, 6, 7, 8, 9] [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
for i in range(3): # runs until 2
    print(i)
```

```
0
1
2
```

```
for i in range(3,0,-1): # while i > 0
    print(i)
```

```
3
2
1
```

```
for a in [1,1,2,3,5,8]:
    print(a)
```

```
1
1
2
3
5
8
```

```
for i in range(5):
    if i == 3:
        continue # skip when i == 3
    print(i)
```

```
for i, a in enumerate([1,1,2,3,5,8]):
    print(i, a)
```

```
0 1
1 1
2 2
3 3
4 5
5 8
```

```
mystr="Strangest Things"
mystr=mystr.split(' ')
```

```
for a in mystr:
    print(a)
```

```
Strangest
Things
```

```
for i in range(5, 5):
    print(i)
```

■ break and continue statements

```
for i in range(5):
    if i == 3:
        break # stop when i == 3
    print(i)
```

while loop

- A **while** loop repeatedly executes a block of code as long as a given condition remains true.

```
i = 0          # initialization
while i < 5:    # condition
    print(i)
    i = i + 1   # update, normally written as "i += 1"
```

0
1
2
3
4

```
i = 5
while i > 0:
    print(i)
    i -= 1
```

5
4
3
2
1

```
NumIter = 0
MaxNumIter = 4

while NumIter <= MaxNumIter:
    print('Iteration:', NumIter)

    if NumIter == MaxNumIter:
        print("Maximum number of iterations reached")

    NumIter += 1
```

Iteration: 0
Iteration: 1
Iteration: 2
Iteration: 3
Iteration: 4
Maximum number of iterations reached

- **Babylonian formula for square root:** The square root of a number (N) can be estimated by starting with a guess (x) and updating it as the average of itself and (N/x). Also, try **np.sqrt(225)**

```
N = 225 # for which we need the square root
x = 1.0 # initial guess

MaxIter = 10
NumIter = 0
while NumIter < MaxIter:
    NumIter += 1
    x = 0.5 * (x + N/x)
    print(NumIter, x)
```

1 113.0
2 57.49557522123894
3 30.704459920916527
4 19.01619279483994
5 15.424107094920206
6 15.005830704716155
7 15.000001132796916
8 15.0000000000000043
9 15.0
10 15.0

Implementation of Babylonian formula with stopping criteria

■ Use while loop with stopping criteria

- Convergence tolerance (xtol): stop when change becomes very small, controls numerical precision
- Maximum iterations (MaxIter): stop when iteration count exceeds a limit, prevents infinite loops

```
import numpy as np

N = 225          # number whose square root we want
xold = 1.0       # initial guess

MaxIter = 100    # set large value for maximum number of iterations
xtol = 1e-5      # numerical precision for converging the solution

dx = np.inf      # ensure loop starts, dx is set to a large value
NumIter = 0

while NumIter < MaxIter and dx > xtol:
    NumIter += 1          # NumIter incremented

    xnew = 0.5 * (xold + N / xold)  # 1. xnew = f(xold)
    dx = np.abs(xnew - xold)        # 2. dx = |xnew-xold|, updated
    xold = xnew                    # 3. update xold

    print(f"Iteration: {NumIter:3d}  value: {xnew:12.6f}  dx: {dx:12.6f}")
if dx <= xtol:
    print("Converged successfully.")
else:
    print("Maximum number of iterations reached.")
```

```
Iteration:  1  value:  113.000000  dx:  112.000000
Iteration:  2  value:   57.495575  dx:   55.504425
Iteration:  3  value:   30.704460  dx:   26.791115
Iteration:  4  value:   19.016193  dx:   11.688267
Iteration:  5  value:   15.424107  dx:    3.592086
Iteration:  6  value:   15.005831  dx:    0.418276
Iteration:  7  value:   15.000001  dx:    0.005830
Iteration:  8  value:   15.000000  dx:    0.000001
Converged successfully.
```

How to effectively learn numerical methods

■ *DIY numerical methods:*

- An effective way to learn numerical methods is to work through a simple example for each algorithm without any loop or stopping condition, with Python used only for each elementary step (addition, multiplication, etc.), and then cross-check the results against a full computer implementation.
- This approach builds intuition, shows how each term is generated, and makes debugging numerical code much easier.
- Most importantly, it helps you plan the loop structure and choose appropriate stopping conditions.

```
N = 225

# iteration 0
k=0; xold = 1.0

# header
print(f"{'Iter':>5s} {'x_old':>10s} {'N/x_old':>10s} {'x_new':>10s} {'dx':>10s}")

# iteration 1
k+=1; xnew = 0.5 * (xold + N/xold); dx = np.abs(xnew-xold)
print(f"{k:5d} {xold:10.4f} {N/xold:10.4f} {xnew:10.4f} {dx:10.4f}")
xold = xnew
```

iterations 2-6 not shown

```
# iteration 7
k+=1; xnew = 0.5 * (xold + N/xold); dx = np.abs(xnew-xold)
print(f"{k:5d} {xold:10.4f} {N/xold:10.4f} {xnew:10.4f} {dx:10.4f}")
xold = xnew
```

Iter	x_old	N/x_old	x_new	dx
1	1.0000	225.0000	113.0000	112.0000
2	113.0000	1.9912	57.4956	55.5044
3	57.4956	3.9133	30.7045	26.7911
4	30.7045	7.3279	19.0162	11.6883
5	19.0162	11.8320	15.4241	3.5921
6	15.4241	14.5876	15.0058	0.4183
7	15.0058	14.9942	15.0000	0.0058

Defining a function

■ Code re-usable blocks as a function

```
def babylon_sqrt(N, xold=1.0, xtol=1e-5, MaxIter=100):
    """
    Compute the square root of a positive number using the Babylonian method.
    Parameters (Input)
    -----
    N : float, Number whose square root is to be computed (N > 0).
    xold : float, optional, Initial guess for the square root (default is 1.0).
    xtol : float, optional, Convergence tolerance; iteration stops when successive estimates differ by < xtol.
    MaxIter : int, optional, Maximum number of iterations allowed.
    Returns (Output)
    -----
    NumIter : int, Number of iterations performed.
    xnew : float, Final approximation to sqrt(N).
    dx : float, Difference between the last two iterations.
    """
    # --- input checks ---
    if N < 0:
        raise ValueError("Cannot compute square root of a negative number.")
    if xold == 0:
        raise ValueError("Initial guess xold must be non-zero.")

    dx = np.inf
    NumIter = 0
    xnew = xold  # ensures xnew is always defined
    # --- iteration ---
    while NumIter < MaxIter and dx > xtol:
        NumIter += 1

        xnew = 0.5 * (xold + N / xold)  # Babylonian update
        dx = abs(xnew - xold)           # change in solution
        xold = xnew                     # update guess

    return NumIter, xnew, dx
```

■ *Keep reusable code blocks in a separate Python file, e.g., nm2026_babylon.py*

- This helps maintain the code in one place and reuse (i.e. **import**) it across multiple Jupyter notebooks.

```
import nm2026_babylon
```

```
nm2026_babylon.babylon_sqrt(2)
```

```
(4, 1.4142135623746899, 2.1238998197947723e-06)
```

- NOTE: If you see an error message "... **not defined**", then you have to restart the kernel on the notebook

```
from nm2026_babylon import babylon_sqrt as bbsqrt
```

```
N = 100
```

```
NumIter, root, dx = bbsqrt(N, xold=1.0, xtol=1e-2, MaxIter=10)
```

```
print("Iterations:", NumIter)
```

```
print("Approx sqrt:", root)
```

```
print("Final dx:", dx)
```

```
Iterations: 7
```

```
Approx sqrt: 10.000000000139897
```

```
Final dx: 5.289550279563571e-05
```

- *If a module is located in a different directory, you may need to add that directory to **sys.path** so Python can find it.*

```
import sys
```

```
sys.path.append('/Users/rr/repos/NM2026/ipynb/nm2026modules')
```

```
N = 10000
```

```
NumIter, root, dx = bbsqrt(N, xold=1.0, xtol=1e-2, MaxIter=10)
```

```
print("Iterations:", NumIter)
```

```
print("Approx sqrt:", root)
```

```
print("Final dx:", dx)
```

```
Iterations: 10
```

```
Approx sqrt: 100.00000025490743
```

```
Final dx: 0.00714013221002574
```

- *You can find your current working directory as follows*

```
import os
```

```
os.getcwd()
```

- *A docstring is the text written between triple quotes (''' ... ''') inside a function, and it can be viewed using the built-in **help()** function*

```
help(nm2026_babylon)
```

```
help(nm2026_babylon.babylon_sqrt)
```

Another example function

- Calculate mean and standard deviation of an array and compare with `np.mean()` and `np.std()`

```
import numpy as np

A = np.array([1,2,3,4,5,6])

# --- using numpy ---
mean = np.mean(A); std = np.std(A)

print(f"Mean: {mean : .6f}, Standard deviation: {std : .6f}")

# --- function to calculate mean and standard deviation ---
def statistical_params(x):

    N = len(x)

    mean = 0
    for val in x:
        mean += val # sum the elements
    mean = mean / N # mean = sum / N

    std = 0
    for val in x:
        std += (val-mean)**2 # sum of squares of deviation
    std = std / N # mean of the sum of squares
    std = np.sqrt(std) # root of the mean

    return mean, std

mean, std = statistical_params(A)

print(f"Mean: {mean : .6f}, Standard deviation: {std : .6f}")
```

```
Mean: 3.500000, Standard deviation: 1.707825
Mean: 3.500000, Standard deviation: 1.707825
```

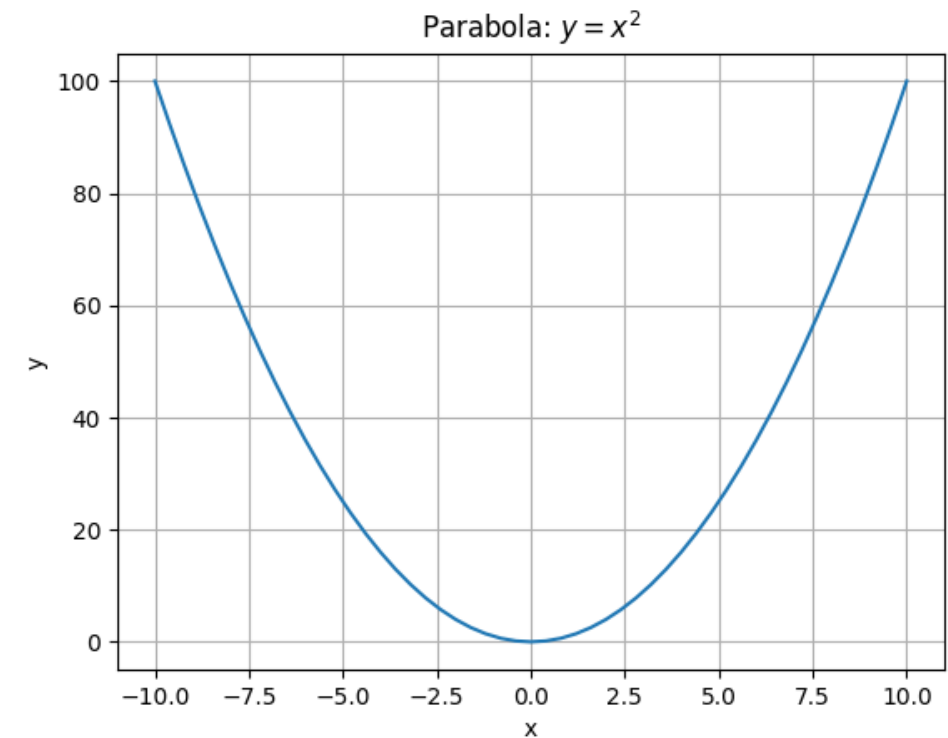
Basic plotting with `matplotlib.pyplot`

```
import numpy as np
import matplotlib.pyplot as plt

xmin = -10; xmax = 10; Ngrids = 51

x = np.linspace(xmin, xmax, Ngrids)
y = x**2

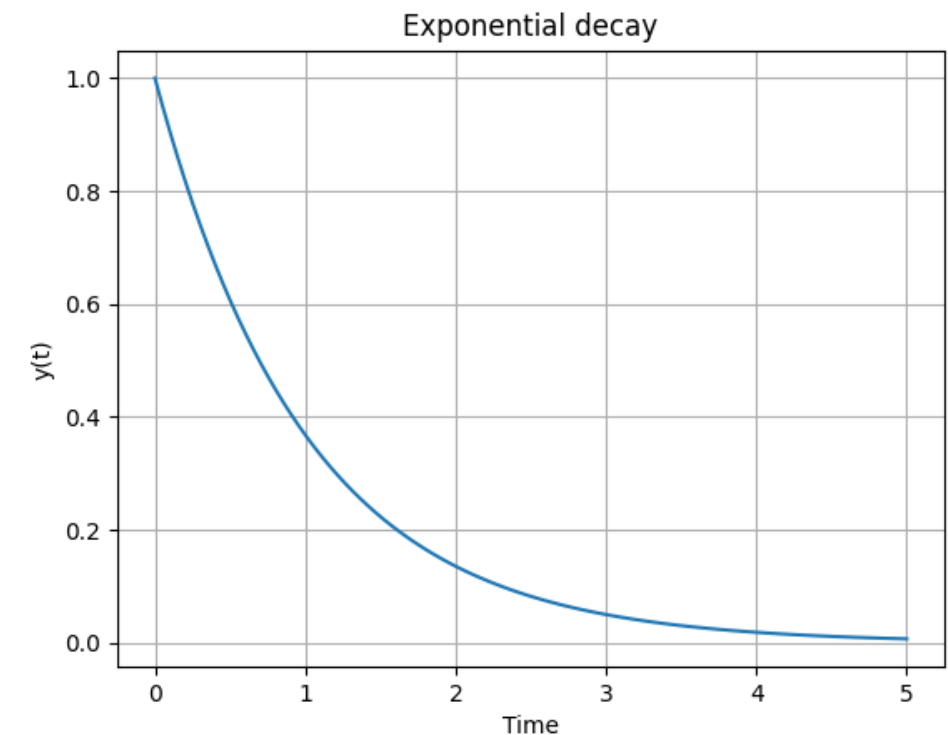
plt.plot(x, y)
plt.title("Parabola:  $y = x^2$ ")
plt.xlabel("x")
plt.ylabel("y")
plt.grid()
plt.savefig('parabola.png') # png, pdf, or svg
plt.show()
```



```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 5, 100)
y = np.exp(-t)

plt.plot(t, y)
plt.title("Exponential decay")
plt.xlabel("Time")
plt.ylabel("y(t)")
plt.grid()
plt.show()
```



```

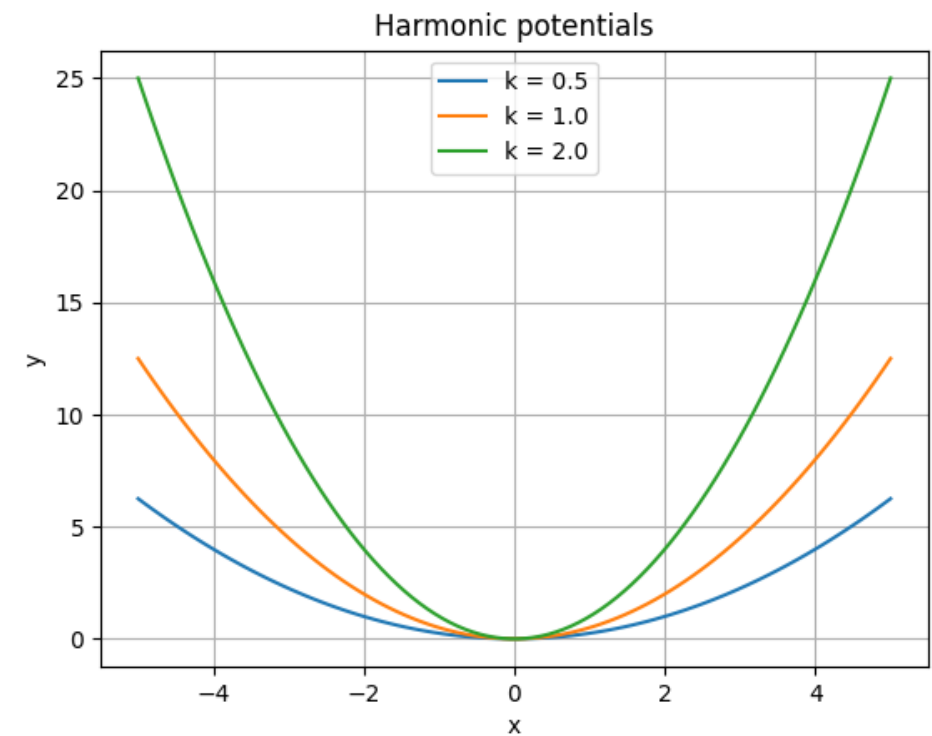
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-5, 5, 100)

k=0.5; plt.plot(x, 0.5*k*x**2, label=f"k = {k}")
k=1.0; plt.plot(x, 0.5*k*x**2, label=f"k = {k}")
k=2.0; plt.plot(x, 0.5*k*x**2, label=f"k = {k}")

plt.title("Harmonic potentials")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid()
plt.show()

```



```

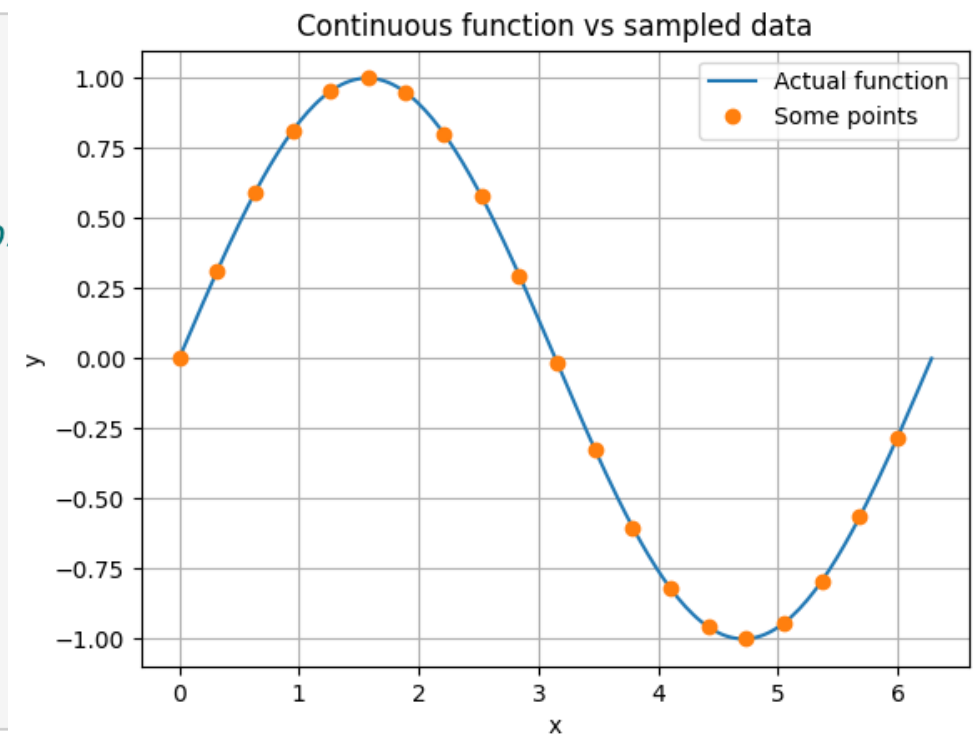
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 2*np.pi, 200); y = np.sin(x)
a=x[::10]; b= y[::10] # a=x[start : stop : step], pick every 10th po

plt.plot(x, y, label="Actual function")
plt.plot(a, b, 'o', label="Some points")

plt.title("Continuous function vs sampled data")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid()
plt.show()

```



```

import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(-np.pi, np.pi, 500)

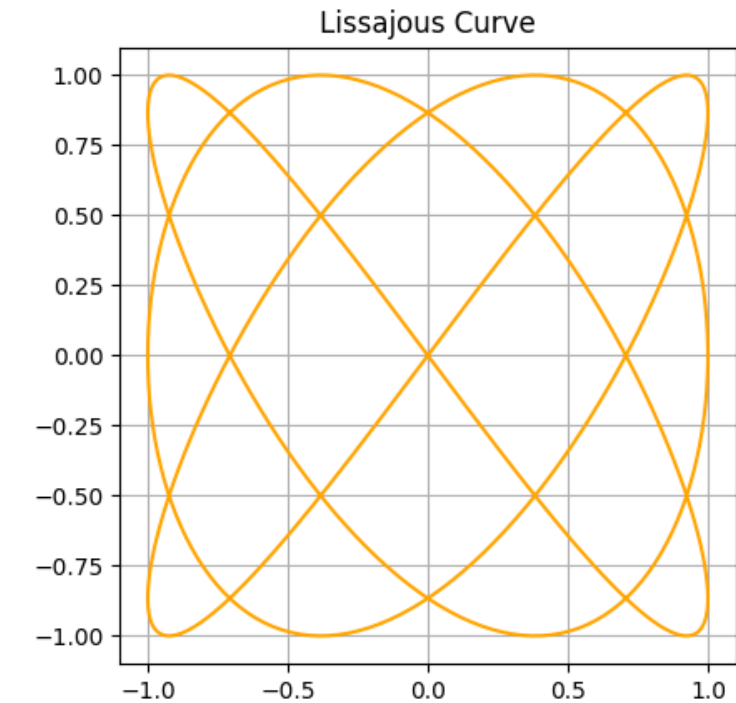
A = 1; B = 1; a = 3; b = 4

x = A*np.sin(a*t + np.pi/2)
y = B*np.sin(b*t)

fig, ax = plt.subplots() # create figure and axes
ax.plot(x, y, color="orange")
ax.set_aspect("equal") # equal scaling: 1 unit in x = 1 unit in y

plt.title("Lissajous Curve")
plt.grid()
plt.show()

```



```

import numpy as np
import matplotlib.pyplot as plt
from scipy.special import factorial as fact

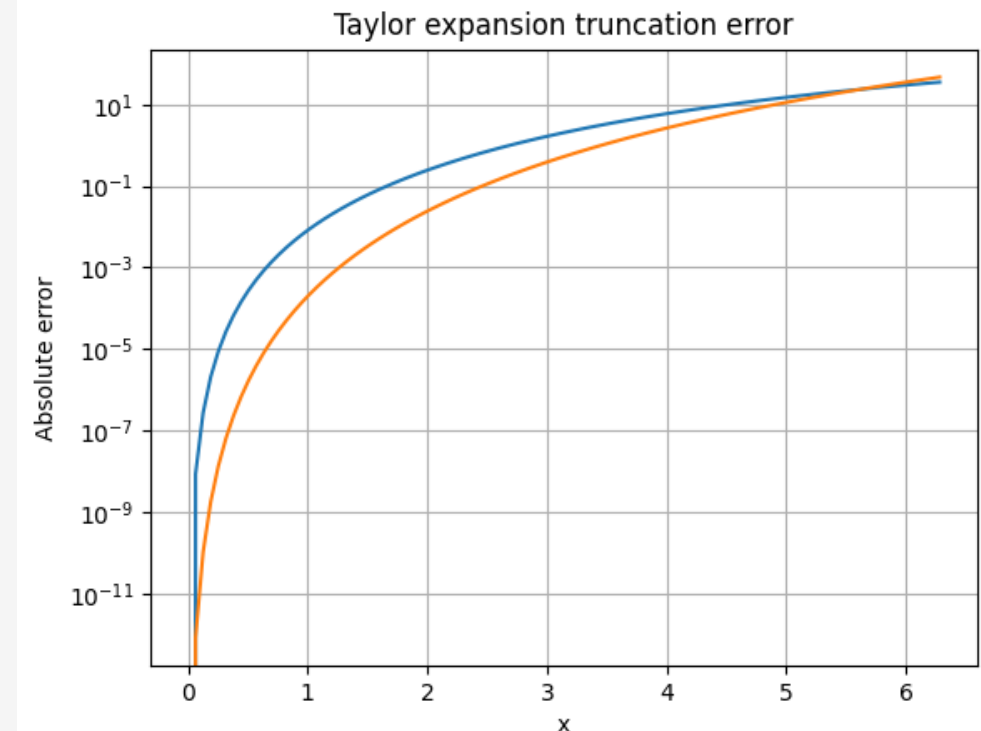
x = np.linspace(0, 2*np.pi, 100)
y_exact = np.sin(x)

# Taylor approximations
y_approx_1 = x - x**3/fact(3)
y_approx_2 = x - x**3/6 + x**5/fact(5)

error1 = np.abs(y_exact - y_approx_1)
error2 = np.abs(y_exact - y_approx_2)

plt.semilogy(x, error1) # log scale on y-axis
plt.semilogy(x, error2)
plt.title("Taylor expansion truncation error")
plt.xlabel("x")
plt.ylabel("Absolute error")
plt.grid()
plt.show()

```



```

import numpy as np
import matplotlib.pyplot as plt
import matplotlib as mpl

def f(x, y):
    return x**2 + y**2

x = np.arange(-2.0, 2.0, 0.1)
y = np.arange(-2.0, 2.0, 0.1)

X, Y = np.meshgrid(x, y)
Z = f(X, Y)

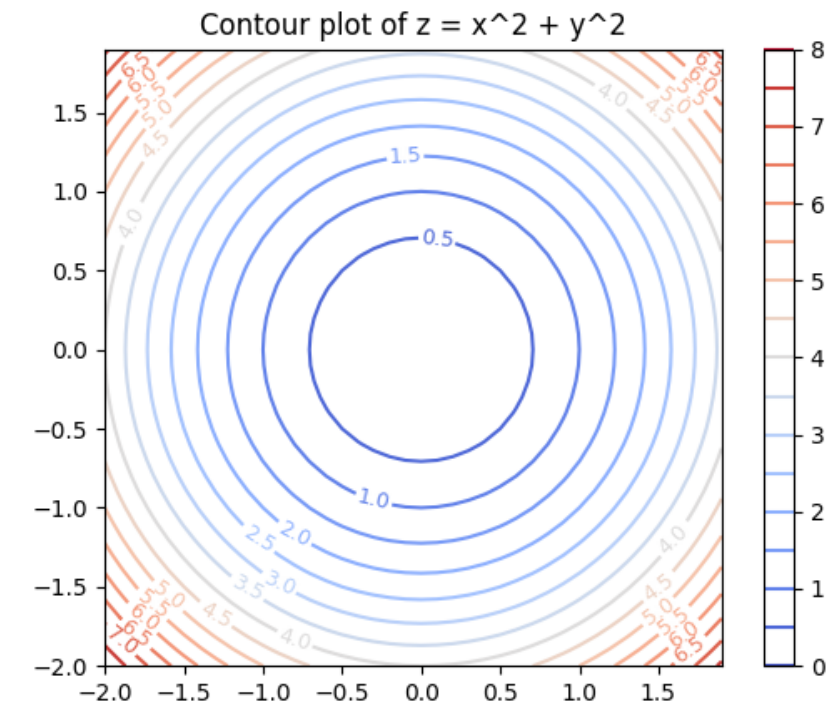
levels = np.arange(0, 8.5, 0.5)

fig = plt.figure()
ax = fig.add_subplot(111)

CS = ax.contour(X, Y, Z, levels, cmap=mpl.cm.coolwarm)
ax.set_aspect("equal", adjustable="box")

plt.clabel(CS, inline=True, fontsize=9)
plt.colorbar(CS)
plt.title("Contour plot of  $z = x^2 + y^2$ ")
plt.show()

```



```

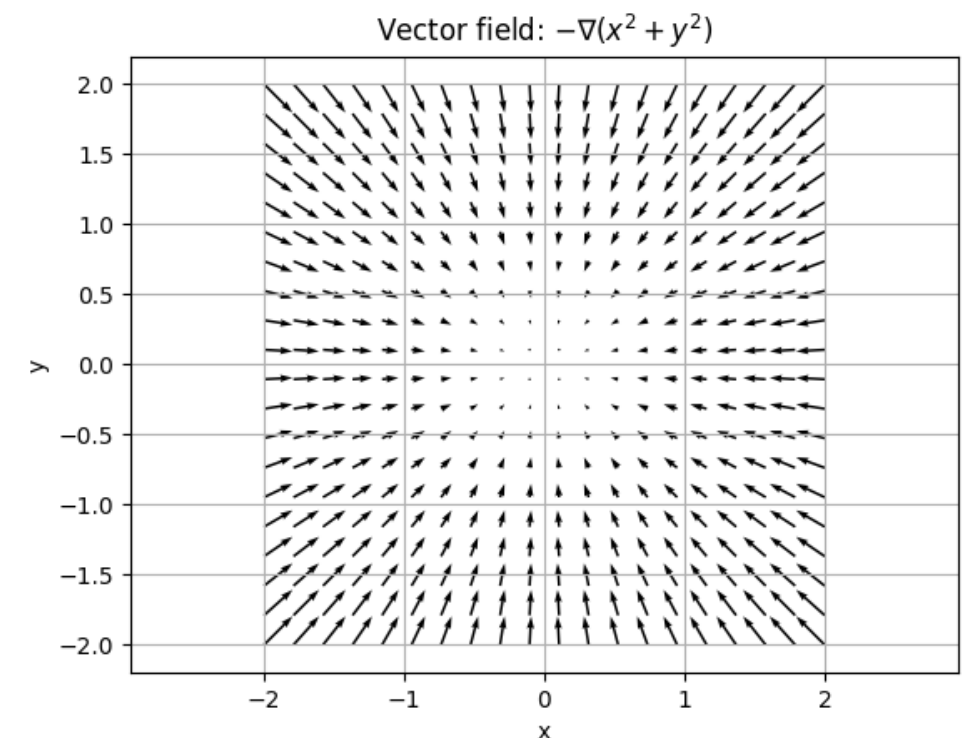
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-2, 2, 20)
y = np.linspace(-2, 2, 20)

X, Y = np.meshgrid(x, y)
U = -2*X
V = -2*Y

plt.quiver(X, Y, U, V)
plt.title("Vector field:  $-\nabla(x^2 + y^2)$ ")
plt.xlabel("x")
plt.ylabel("y")
plt.axis("equal")
plt.grid()
plt.show()

```



Input/Output

```
TC=input('Enter temperature in Celcius: \n') # Read the input and assign it to the variable TC
```

Enter temperature in Celcius:

← Enter the value and hit return

```
TC+273.15 # will return TypeError: can only concatenate str (not "float") to str
```

```
print(type(TC)) # will return <class 'str'>
```

eval() is a Python function that takes a string and executes it as a Python expression, returning the result.

```
TC=eval(TC); print(type(TC)) # This will return <class 'int'>
```

```
TK=TC+273.15
```

```
print(str(TK)+' is the temperature in Kelvin') # TK is converted to a string and concatenated with " is the temp..."
```

■ Write output in a file and read input from a file

```
outfile = open('sqrt_values.csv', 'w')
```

```
for i in range(10):
```

```
    # Scientific format without fixed width (preferred for CSV)
```

```
    # No leading spaces, controlled precision, machine and human friendly
```

```
    outfile.write(f'{i:.6E},{np.sqrt(i):.6E}\n')
```

```
outfile.close()
```

■ Using a fixed-width format like **12.6E** can introduce leading spaces in a CSV file. Using fixed-point formats like **.6f** is not suitable if the integer part becomes very large. Such formats are therefore not recommended for general-purpose numerical CSV output.

```
infile=open('sqrt_values.csv','r')
```

```
for line in infile:
```

```
    line_values=line.split(',') # split the line, line_values is a list
```

```
    N=line_values[0] # N is the first element of line_values
```

```
    sqrtN=line_values[1] # sqrtN is the second element of line_values
```

```
    print(N,sqrtN)
```

```
infile.close()
```

Inspect this file by opening in a file editor

Store vectors in a file and read them back

- **Complete workflow:** Let's calculate particle-in-a-box wavefunctions, store them in a csv file, read them back, convert to numpy arrays, and visualize them.

```
import numpy as np

# Parameters
L = 1.0
n_points = 200
x = np.linspace(0, L, n_points)

# Lowest two wavefunctions
psi1 = np.sqrt(2/L) * np.sin(np.pi * x / L)
psi2 = np.sqrt(2/L) * np.sin(2 * np.pi * x / L)

# Write to file
outfile = open("pib_wavefunctions.csv", "w")
outfile.write("x,psi1,psi2\n") # header

for i in range(n_points):
    outfile.write(f"{x[i]:.6e},{psi1[i]:.6e},{psi2[i]:.6e}\n")

outfile.close()
```

```
x = []
psi1 = []
psi2 = []

infile = open("pib_wavefunctions.csv", "r")
next(infile) # skip header

for line in infile:
    values = line.split(",")
    x.append(float(values[0])) # convert str to float
    psi1.append(float(values[1]))
    psi2.append(float(values[2]))

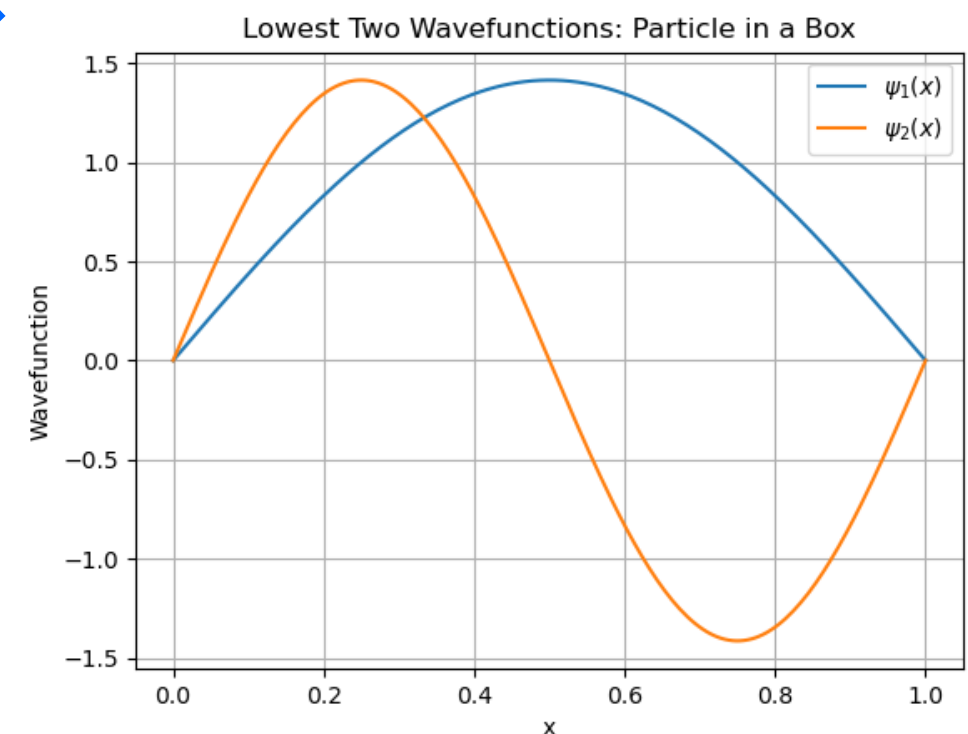
infile.close()

# Convert to NumPy arrays
x = np.array(x)
psi1 = np.array(psi1)
psi2 = np.array(psi2)
```

```
import matplotlib.pyplot as plt

plt.plot(x, psi1, label=r"$\psi_1(x)$")
plt.plot(x, psi2, label=r"$\psi_2(x)$")

plt.xlabel("x")
plt.ylabel("Wavefunction")
plt.title("Lowest Two Wavefunctions: Particle in a Box")
plt.legend()
plt.grid()
plt.show()
```



Recursive function

- *When to use?:* When a problem is naturally defined in terms of smaller versions of itself and has a clear stopping (base) condition. Python limits recursion depth (≈ 3000 calls by default on my laptop).

```
import numpy as np

N=10; a = np.arange(1,N+1)

sum_a=np.sum(a); print(a, sum_a)

[ 1  2  3  4  5  6  7  8  9 10] 55
```

```
sum_a=0

for i in a:
    sum_a += i

print(sum_a)
```

55

1. Exact integer factorial

```
import math
```

```
N = 10; val1 = math.factorial(N)
```

2. Factorial as float

```
from scipy.special import factorial as scfact
```

```
val2 = scfact(N)
```

3. Exact integer factorial using scipy

```
val3 = scfact(N, exact=True)
```

4. Using Gamma function, $n! = \text{gamma}(n+1)$

```
from scipy.special import gamma as scgamma
val4 = scgamma(N+1)
```

```
print(val1, val2, val3, val4)
```

3628800 3628800.0 3628800 3628800.0

```
import sys; print(sys.getrecursionlimit())
```

```
def recursive_sum(arr):
```

```
    # Calculate recursively using the logic,
    # first element + sum of the rest
```

```
    if len(arr) == 0: # no more elements
        return 0 # stop and return 0
```

```
    else:
        val = arr[0] + recursive_sum(arr[1:])
        return val
```

```
print(recursive_sum(a))
```

55

■ Calculate factorial

```
def recursive_fact(n):
```

```
    if n == 0: # 0! is 1
        return 1
```

```
    else:
        return n * recursive_fact(n - 1) # recursive call
```

```
val5 = recursive_fact(N); print(val5)
```

3628800

scipy's factorial and gamma are vector functions

```
a = np.arange(5); print(a, scfact(a), scgamma(a+1))
```

```
[0 1 2 3 4] [ 1.  1.  2.  6. 24.] [ 1.  1.  2.  6. 24.]
```

Scope

- **Scope:** Refers to the region of a program where a variable or parameter can be used.

- Built-in scope, Global scope, Local scope, Enclosing scope

- **Built-in scope**

```
# Names automatically available in Python, which includes functions like len, print, range
```

```
print(len("Python")) # will return 6
```

```
#You can see them using:
```

```
import builtins  
print(dir(builtins))
```

```
# Do not shadow built-ins. Avoid using built-in names like list, sum, max for variables.
```

```
list = [1, 2, 3]    # Bad practice  
print(list)
```

```
list(range(5)) # returns TypeError: 'list' object is not callable
```

```
del list # clear this name and set it to built-in value
```

```
list(range(5)) # returns [0, 1, 2, 3, 4]
```

```
%reset -f # clears all user-defined names; only built-ins remain accessible
```

■ Global scope

Variables defined in a script (.py file) or notebook before the function call is accessible inside the function

```
pi = 3.14159          # global variable
```

```
def area(r):  
    return pi * r * r  
area(2) # will return 12.56636
```

```
def area(r):  
    return pi * r * r  
pi = 3.14159 # this is also a global variable as it is defined before the function call  
area(2)
```

Assigning to a global variable inside a function requires global:

```
count = 0  
def increment():  
    count += 1  
  
increment()  
print(count) # will return UnboundLocalError: cannot access local variable 'count' where it is not associated with a
```

```
count = 0  
def increment():  
    global count  
    count += 1  
  
increment()  
print(count) # will return 1
```

Shows all variable names in the global space

```
for k in globals().keys():  
    print(k)
```

Shows user defined names in the global space

```
for k in globals():  
    if not k.startswith('_'):  
        print(k)
```

■ Local scope

```
# Variables and parameters defined inside a function
# Exist only while the function is executing

def add(a, b):      # a and b are parameters (local scope)
    result = a + b  # local variable
    return result

print(a) # returns NameError: name 'a' is not defined
```

```
def add(a, b):
    c = a + b
    # locals() shows all names in the
    # local scope (function parameters + local variables)
    print(locals())

add(2, 3) # returns {'a': 2, 'b': 3, 'c': 5}
```

■ Enclosed scope

```
# Applies to nested functions.
# Variables defined in an outer function, used in an inner function

def outer():
    x = 10          # enclosing scope
    def inner():
        print(x)    # x is accessible here
    inner()

outer() # returns 10
```

```
# To modify the variable in the enclosing scope, define it as nonlocal

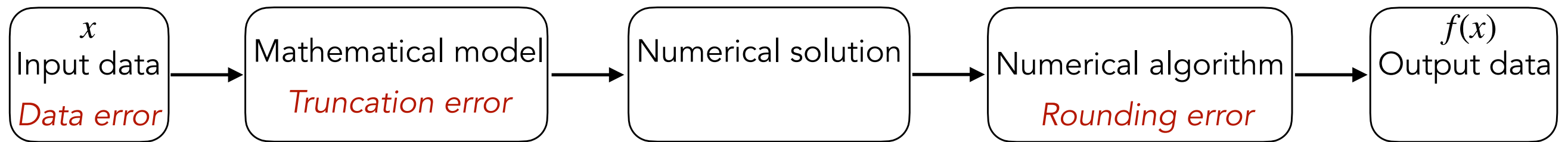
def outer():
    x = 10
    def inner():
        nonlocal x
        x = 5
    inner()
    print(x)

outer() # returns 5
```

2. Errors and Big-O notation

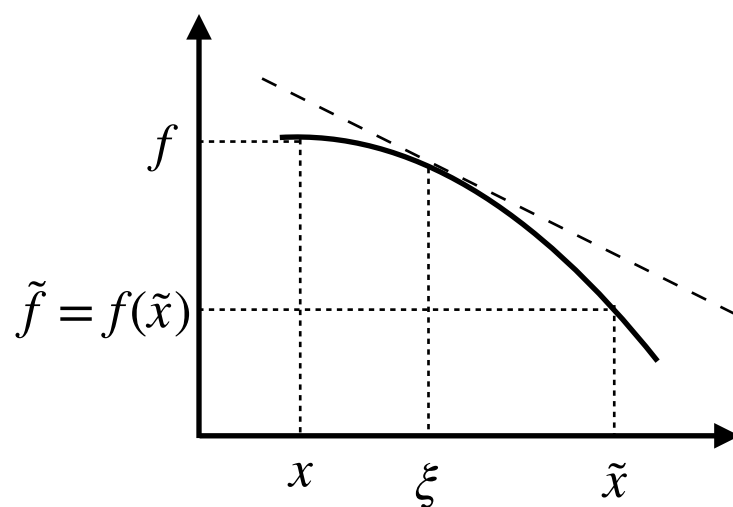
Basic concepts in error analysis

■ Sources of error



■ Forward/backward error analysis

- **Forward error** is the error in approximating $f(x)$ as $\tilde{f}$ if there were no error in the input. Absolute forward error is defined as $|\tilde{f} - f|$, relative forward error is defined as $|\tilde{f} - f|/|f|$ (when $f \neq 0$). Forward error measures how close the computed result is to the true value and is typically the quantity of interest. Since this requires knowing the exact value of $f(x)$ (i.e., the exact solution of the problem we are solving), forward error is difficult to estimate.
- Since the computed value $\tilde{f}$ is not the exact value $f(x)$, we ask if there exists a nearby input $\tilde{x}$ for which $\tilde{f} = f(\tilde{x})$. Then the absolute **backward error** is defined as $|\tilde{x} - x|$. Relative backward error is defined as $|\tilde{x} - x|/|x|$ (when $x \neq 0$).



■ Mean value theorem

- The mean value theorem relates backward error to forward error by showing how a perturbation in the input x produces a corresponding change in the output $f(x)$. If $\tilde{f} = f(\tilde{x})$, then there exists some ξ between x and $\tilde{x}$ such that $|f(\tilde{x}) - f(x)| = |f'(\xi)| |\tilde{x} - x|$.

■ Error bound for forward error

- Suppose we found an approximation for x with an error bound: $x = \tilde{x} \pm \epsilon$. Then, for a monotonic function, $f(x)$, the absolute forward error is $|\Delta f| = |f(x) - f(\tilde{x})| \leq \max_{y \in [\tilde{x}-\epsilon, \tilde{x}+\epsilon]} |f(y) - f(\tilde{x})|$.

■ **Example:** Find the upperbound in finding the square root of a number $f(a) = \sqrt{a}$ for $a = 2.05 \pm 0.01$.

- We can find the bound as
 $\max\{|\sqrt{a-da} - \sqrt{a}|, |\sqrt{a+da} - \sqrt{a}|\} = \max\{0.003496, 0.003488\} = 0.003496$.
- From the mean value theorem, we can also find $|\Delta f| = |f'(\xi)| |\tilde{x} - x|$ as
 $|\Delta f| = |1/(2\sqrt{\xi})| |\Delta a| = 0.01/(2\sqrt{2.05}) = 0.003492$, which satisfies $|\Delta f| < \text{bound}$.
- If we consider the maximum absolute value of the derivative, which occurs for the smallest value of x , we have $|\Delta f| = |1/(2\sqrt{\xi})| |\Delta a| = 0.01/(2\sqrt{2.04}) = 0.003496$, which satisfies $|\Delta f| = \text{bound}$.
- Hence, even though we do not have an exact value of x or the exact value of $f(x)$, we have an upper bound for the error in determining $f(x)$.

```
import numpy as np
a=2.05; da=0.01; fprime_xi=1/(2 * np.sqrt(a))
df = fprime_xi * da; print(f'{df:.6f}')
```

0.003492

```
left_bound=np.abs( np.sqrt(a-da)-np.sqrt(a) ); right_bound=np.abs( np.sqrt(a+da)-np.sqrt(a) )
bound = np.max([left_bound, right_bound])
print(f'{left_bound:.6f}, {right_bound:.6f}, {bound:.6f}')
```

0.003496, 0.003488, 0.003496

```
df <= bound
```

True

- **Problem 1:** Suppose you have measured the radius of a cell or a droplet from the cross-section as $r = 10 \pm 0.1 \mu\text{m}$. Find an error bound for the area of the cross-section.
- **Problem 2:** The rate of a reaction follows $k(T) = \exp(-1/T)$. If the temperature is measured as $T = 300 \pm 1 \text{ K}$, find the maximum error one can expect for the rate constant.

■ *Error propagation for simple arithmetic operations*

- Suppose we want to add, subtract, multiply or divide two numbers x_1 and x_2 . We only have the approximations $\bar{x}_1$ and $\bar{x}_2$. Then, the error bounds follow

- ◆ $f(x_1, x_2) = x_1 + x_2$, $|\Delta f| = |f - \tilde{f}| \leq |\Delta x_1| + |\Delta x_2|$
- ◆ $f(x_1, x_2) = x_1 - x_2$, $|\Delta f| = |f - \tilde{f}| \leq |\Delta x_1| + |\Delta x_2|$
- ◆ $f(x_1, x_2) = x_1 \times x_2$, $|\Delta f|/|f| = |f - \tilde{f}|/|f| \leq |\Delta x_1/x_1| + |\Delta x_2/x_2|$
- ◆ $f(x_1, x_2) = x_1/x_2$, $|\Delta f|/|f| = |f - \tilde{f}|/|f| \leq |\Delta x_1/x_1| + |\Delta x_2/x_2|$

- **Problem 3:** Suppose the efficiency of the solar heater is given as $\eta = KQT_d/I$, where K is a constant that we know very accurately, while the volume flow (Q), net temperature difference between the outlet and inlet (T_d), and irradiance (I). Suppose we have two solar heaters for which we know the errors in the parameters as follows.

- S1: Relative errors of 1.5%, 1% and 3.6% for Q , T_d , and I
- S2: Relative errors of 0.5%, 1% and 2% for Q , T_d , and I

Calculate the error bounds for the efficiency and comment on whether one can conclude whether one solar heater is better than the other.

■ *Solution 1:* From mean value theorem: $|A'(r)| = 2\pi r \implies |\Delta A| \leq 2\pi(10.1)(0.1)$

■ *Solution 2:* $k'(T) = (e^{-1/T})/T^2$, $|\Delta k| \leq \max_{T \in [299, 301]} |k'(T)| \Delta T$

■ *Solution 3:* $\left| \frac{\Delta \eta}{\eta} \right| \leq \left| \frac{\Delta Q}{Q} \right| + \left| \frac{\Delta T_d}{T_d} \right| + \left| \frac{\Delta I}{I} \right|$
 $\left| \frac{\Delta \eta_1}{\eta_1} \right| \leq (1.5 + 1 + 3.6) \cdot 0.01 = 0.061$; $\left| \frac{\Delta \eta_2}{\eta_2} \right| \leq (0.5 + 1 + 2) \cdot 0.01 = 0.035$

Suppose $\eta_1 = x_1$ and $\eta_2 = x_2$, then $x_1 - 0.061 \leq \eta_1 \leq x_1 + 0.061$ and $x_2 - 0.035 \leq \eta_2 \leq x_2 + 0.035$. As long as the intervals do not overlap, one can be sure if one collector is better than the other.

■ *Generalization to a multivariate function*

◦ Suppose $f(\mathbf{x})$ is differentiable in a neighborhood of $\mathbf{x} = (x_1, x_2, \dots, x_n)$, then the maximal error bound

is given by $|\Delta f| \leq \sum_{k=1}^n \left| \frac{\partial f}{\partial x_k} \Delta x_k \right|$.

■ *Problem 4:* Find the error bound for $f(x_1, x_2) = \sin(x_1^2 x_2)$, where $x_1 = 0.75 \pm 0.01$ and $x_2 = 0.413 \pm 0.003$.

Computer representation of numbers

- Let's start with a motivating example. Try the following Python statements and explain the output.

```
print(7./3-4./3-1)
```

```
2.220446049250313e-16
```

```
print(8./3-5./3-1)
```

```
-2.220446049250313e-16
```

We find that one output is positive and another is negative. This means, $7./3-4./3$ should be greater than **1** and $8./3-5./3$ should be less than **1**.

```
print(7./3-4./3)
```

```
1.0000000000000002
```

```
print(8./3-5./3)
```

```
0.9999999999999998
```

To understand why we are getting these answers, neither are precisely 1, we need to understand how real numbers (i.e., floats) are represented in a computer.

- If an N -bit fixed-point representation (where each float is stored in N -bits), suppose M bits are reserved for the integer part, then $N-M$ bits are reserved for the decimal part.
- In an N -bit representation, N significant figures are retained in the binary format.
- Python follows the IEEE-754 standard for floating-point arithmetic.

- In general, a number is stored in the form (known as the normalized form)

$$(-1)^S \times M \times 2^e = (-1)^S \times \left[1 + \sum_i M_i 2^{-i} \right] \times 2^e$$

S is the sign bit, M is the mantissa (bits that encode the fractional significant figures excluding the leading 1, which is left to the decimal point), M_i is the i -th digit in mantissa and e is the exponent to be encoded.

- S is a 1-bit integer with allowed values 0 (denotes positive) or 1 (denotes negative).
- If the exponent E is an n -bit integer, it can store 2^n integers from -2^{n-1} to $2^{n-1} - 1$. Accordingly, -2^{n-1} will be the first integer, and $2^{n-1} - 1$ will be the last integer in the 2^n -th position. So, in general, instead of using E , we will use $e = E + d$, where $d = 2^{n-1} - 1$. This will become clear with an example later.

- **Mantissa:** Let's try to understand the mantissa, M from scratch.

- Decimal numbers are base-10 numbers. For example, 22.5 can be written as $2 \times 10^1 + 2 \times 10^0 + 5 \times 10^{-1}$. We can write 22.5 as $0.225 \times 10^2 = (2 \times 10^{-1} + 2 \times 10^{-2} + 5 \times 10^{-3}) \times 10^2$.

- Let's write 22.5 as a base-2 number.

Let's start with the integer part. $22/2 = 11$ remainder 0 (this is the least significant digit in the binary representation), $11/2 = 5$ remainder 1, $5/2 = 2$ remainder 1, $2/2 = 1$ remainder 0, $1/2 = 0$ remainder 1 (this is the most significant digit in the binary representation). So, we will write the integer part as 10110

Now the decimal part. $0.5 \times 2 = 1.0$ and we can't continue beyond.

Overall, we can write $(22.5)_{10} = (10110.1)_2$.

We can write 10110.1 as $1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 + 1 \times 2^{-1}$ or

1.01101×2^4 which can be written as $1 + (0 \times 2^{-1} + 1 \times 2^{-2} + 1 \times 2^{-3} + 0 \times 2^{-4} + 1 \times 2^{-5}) \times 2^4$.

- Let's try another example. Convert the base-10 number -0.8125 to a base-2 number.

Let's start with the integer part. $-0/2 = 0$ remainder 0

Now the decimal part. $0.8125 \times 2 = 1.6250$, where (1 is the most significant binary digit),

$0.625 \times 2 = 1.25$ (giving another binary digit 1), $0.25 \times 2 = 0.5$ (giving another binary digit 0), and

$0.5 \times 2 = 1$ (giving another binary digit 1), and we can't continue beyond. Overall, we can write

$(-0.8125)_{10} = (-0.1101)_2 = -1.101 \times 2^{-1}$ in the normalized form (which is $\pm 1 \cdot M \times 2^E$). Here we can see that the mantissa M is 101.

As stated before, instead of $\pm 1 \cdot M \times 2^E$, we will use the form $\pm 1 \cdot M \times 2^e$.

Suppose we use 8 bits to store the exponent form, then there are $2^8 = 256$ integer values. As stated before, $d = 2^{8-1} - 1 = 2^7 - 1 = 127$. So, $e = E + d = -1 + 127 = 126$, which means -1 will be the 126th integer among the 256 values.

- According to IEEE-754 standard, to store a float in a 32 bits, we reserve 1 bit for the sign, 8 bits for the exponent (as shown in the example above), and the remaining bits (32-9=23 bits) to store the mantissa. Hence, the mantissa 101 will be stored as 1010 0000 0000 0000 0000 000 (where a space is introduced to aid the eye).

- The following table summarizes the bits allocated for sign, exponent and mantissa according to the IEEE-754 standard.

No. of bits	Precision	Sign	Exponent	Mantissa
32	Single	1	8	23
64	Double	1	11	52
128	Quadruple	1	15	64

- With double precision (64 bit floats), 1 is represented as $+1.000\dots000 \times 2^0$ (with 52 zeros, mantissa) and the next floating point number greater than 1 we can represent in a computer is $+1.000\dots001 \times 2^0$ which, in decimal values equals to $1 + 2^{-52}$, where $2^{-52} = \epsilon$ is the machine epsilon (or machine precision).

```
eps=sys.float_info.epsilon; print(eps, eps==2**(-52))
```

```
2.220446049250313e-16 True
```

- *Rounding off floats*: According to the IEEE-754 standard, rounding off a float is done by following three rules
 - Rule 1: If the 53rd bit is 0, then truncate after the 52nd bit (rounding down)
 - Rule 2: If the 53rd bit is 1, then add 1 to the 52nd bit (rounding up)
 - Rule 3: If the 53rd bit is 1, but if 54th, 55th, .., and all known values beyond are 0, then, add 1 to the 52nd bit, if the 52nd bit is 1.

- Let's write 9.4 as a base-2 number.

Let's start with the integer part. $9/2 = 4$ remainder 1 (this is the least significant digit in the binary representation), $4/2 = 2$ remainder 0, $2/2 = 1$ remainder 0, $1/2 = 0$ remainder 1 (this is the most significant digit in the binary representation which will be the left most). So, we will write the integer part as 1001.

Now the decimal part. $0.4 \times 2 = 0.8$ (giving the digit 0), $0.8 \times 2 = 1.6$ (giving the digit 1), $0.6 \times 2 = 1.2$ (giving the digit 1), $0.2 \times 2 = 0.4$ (giving the digit 0)...overall 0110...which will continue as 0110 0110 ...

- We can write 9.4 as 1001.0110 0110 0110 0110 ... or 1.0010 1100 1100 ... 1100 1100, the 53rd bit should be 1, hence we follow Rule 2 and add 1 to the 52nd bit to get 1.0010 1100 ... 1100 1101.

- *Applying these ideas to $7./3-4./3$ and $8./3-5./3$:*

- $7./3 = 10.0101\ 0101\ \dots\ 0101 \times 2^1 = 1.0010\ 1010\ \dots\ 1010 \times 2^1 = 1.0010\ 1010\ \dots\ 1011 \times 2^1$ (Rule 2)

- $4./3 = 1.0101\ 0101\ \dots\ 0101 \times 2^0 = 0.1010\ 1010\ \dots\ 1010 \times 2^1$ (Rule 1)

- $7./3-4./3 = (1.0010\ 1010\ \dots\ 1011 - 0.1010\ 1010\ \dots\ 1010) \times 2^1 = (0.1000\ 0000\ \dots\ 0001) \times 2^1$
(Rule 3) $= 1 + 2^{-52} = 1 + \epsilon$ (Recall, $1.0 - 0.1 = 0.1$)

- Similarly, repeat $8./3-5./3$ to find that it equals to $1 - 2^{-52} = 1 - \epsilon$.

- *Base-2 representation using the **struct** module:*

```
def get_bit_rep(x):
    import struct

    packed = struct.pack('>d', x) # pack the float into 8 bytes (64 bits)

    bits = ''
    for byte in packed:
        bits += format(byte, '08b')

    return bits
```

```
get_bit_rep(0)
```

[illegible]

```
get_bit_rep(0.0)
```

[illegible]

- Recall that numbers are stored in normalized form (which is $\pm 1 \cdot M \times 2^e$). Hence, 1 should be stored as 1×2^0 . Hence, the mantissa will be 0000 0000 ... 0000 as 1 is represented as 1.000 000 ... 000 $\times 2^0$
- We have $E = 0$ and as stated before in the Table, $n = 11$ bits are allocated to exponents. Hence, $d = 2^{10-1} - 1 = 2^{10} - 1 = 1023$. So, $e = E + d = 0 + 1023 = 1023$, which means 0 will be the 1023rd integer among the 2048 values. We know that the binary representation of 1023 is 1111 1111 11 (10 ones). Since we have 11 bits, 1023 will be represented as 0111 1111 1111 (zero followed by 10 ones).

```
get_bit_rep(1)
```

```
'0011111111110000000000000000000000000000000000000000000000000000'
```

```
get_bit_rep(-1)
```

```
'10111111111100000000000000000000000000000000000000000000000000000000'
```

```
get_bit_rep(7/3)
```

[illegible]

```
get_bit_rep(4/3)
```

```
'00111111111101010101010101010101010101010101010101010101010101'
```

```
get_bit_rep(8/3)
```

```
'010000000000010101010101010101010101010101010101010101010101'
```

```
get_bit_rep(5/3)
```

```
'0011111111111010101010101010101010101010101010101010101010101011'
```

Big-O notation for scaling

■ *Big-O notation is useful to understand*

- How quickly does the value of a function (e.g., error/residual) approach zero or infinity?
- How much CPU time does an algorithm take?
- How much memory (RAM or hard disk) does it require to store an array or matrix?

■ For two functions $f(x)$ and $g(x)$, if $|f(x)| \leq C|g(x)|$ when $\lim_{x \rightarrow 0}$ or $\lim_{x \rightarrow \infty}$, then we say $f(x) = O(g(x))$.

■ *Example:* Let's understand the scaling of the finite-step error in approximating the derivative of a function using the forward-difference formula: $Df(x; h) = \frac{f(x+h) - f(x)}{h}$.

According to Taylor's residual theorem, the error in the forward-difference approximation is $f(x+h) = f(x) + hf'(x) + \frac{h^2}{2}f''(\xi)$ or $\frac{f(x+h) - f(x)}{h} = Df(x; h) = f'(x) + \frac{h}{2}f''(\xi)$, where $\xi \in (x, x+h)$.

The residual in the approximation is $R(h) = Df(x; h) - f'(x) = \frac{h}{2}f''(\xi) = Ch$. Hence, we say that the error in the forward difference formula scales linearly (or decreases linearly) with the step size.

■ *Exercise:* Find the scaling of the finite-step error in approximating the derivative of a function using the central-difference formula: $Df(x; h) = \frac{f(x+h) - f(x-h)}{2h}$ using Taylor's residual formula

According to Taylor's residual theorem, the error in the central-difference approximation is $f(x+h) = f(x) + hf'(x) + \frac{h^2}{2}f''(x) + \frac{h^3}{3!}f'''(\xi)$.

- **Exercise:** For the following values of step size, $h = 0.1, 0.2, 0.3, 0.4, 0.5$ plot the absolute value of the residuals of the forward difference and central difference formulae as a function of h . Comment on the shape of these plots. Take for example, $f(x) = \sin(x)$ and $x = 1$.
- **Exercise:** Fit the residuals of the forward difference formula to a polynomial $R(h) = c_0 + c_1h + c_2h^2$. Use NumPy's polynomial fitting function: `coef_f = np.polyfit(hs, err_f, 2)` where `coef_f` returns $[c_2, c_1, c_0]$.
 - What are the values of the coefficients?
 - Is c_0 approximately zero? Why should it be?
 - What does the fitting tell about the order of the method?

■ *Big-O notation for CPU time*

- Runtime complexity (or time-scaling) of the following loop structures are $O(n)$, $O(n^2)$, and $O(n^3)$.

```
for i in range(n):
    ...
```

```
for i in range(n):
    for j in range(n):
        ...
```

```
for i in range(n):
    for j in range(n):
        for k in range(n):
            ...
```

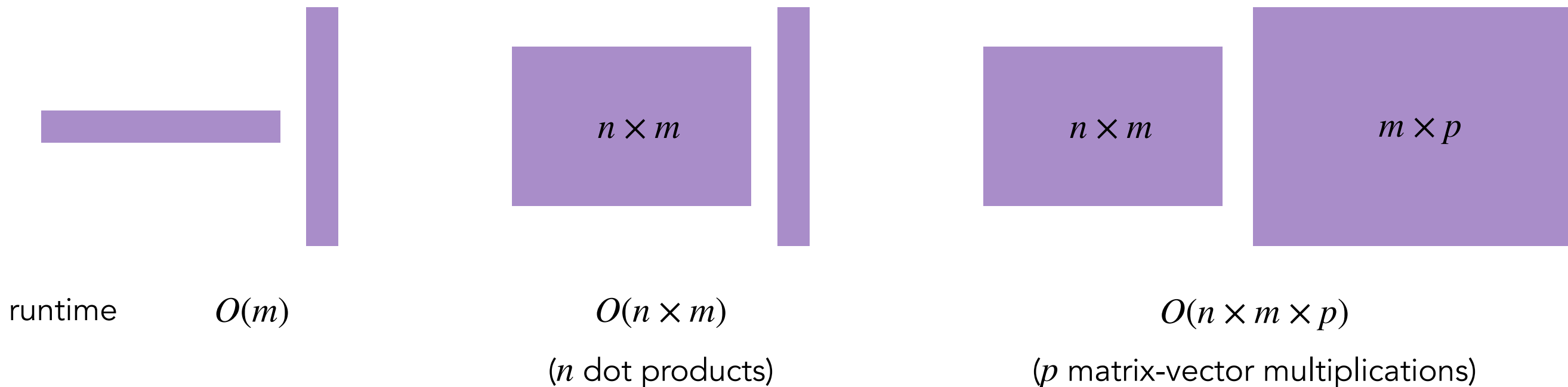
- Suppose the runtime of an algorithm scales as $2n^3 - n^2$, where n is a problem size (such as the length of a vector), then we say that the runtime scales as $O(n^3)$ for large values of n .

- **Logarithmic scaling:** The runtime complexity of the following loop is $O(\log n)$

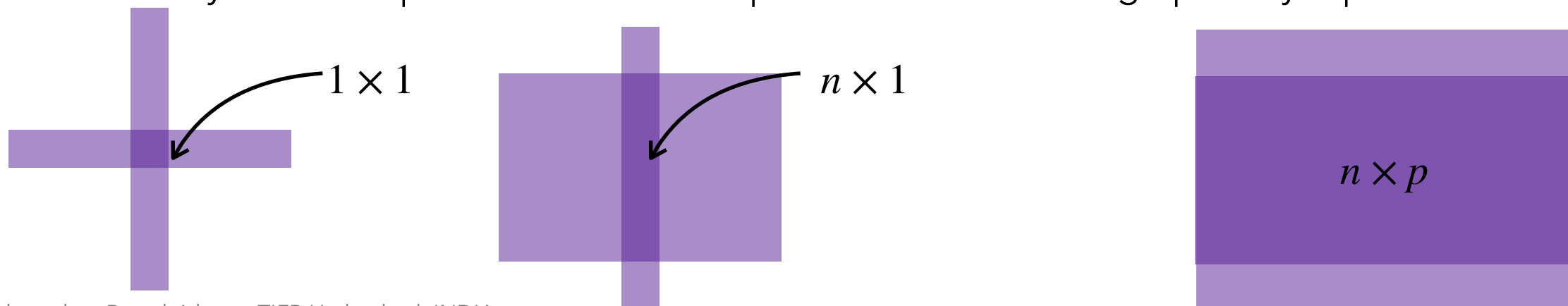
```
i = 1
while i <= n:
    i *= 2
```

Let's assume that the loop has k iterations. Hence, the loop runs until 2^k exceeds n . If we solve for k , we find $k = \log_2 n$. Hence, we say that the number of iterations k is $O(\log n)$.

- **Vector operations:** Graphical representations of the dot product, matrix-vector multiplication, and matrix-matrix multiplication are as follows.



The dimensionality of the output of these vector operations can also be graphically represented as follows



- *Exercise:* Without using Numpy's matrix multiplication or dot product functions, implement the following
 - A function **dot**(**x**, **y**) to compute dot product of two vectors.
 - A function **matvec**(**A**, **x**) to compute matrix-vector multiplication by calling **dot**.
 - A function **matmat**(**X**, **B**) to compute matrix-matrix multiplication by calling **matvec**.
- *Exercise:* Suppose **A**, **B**, and **C** are $n \times n$ matrices. What are the runtime complexities of the operations **A** $\times$ **B** and **A** $\times$ **B** $\times$ **C**. Suppose for a problem in quantum mechanics, you want to numerically perform a similarity transformation $\mathbf{V}^T \times \mathbf{H} \times \mathbf{V}$. How will you implement it?
- *Big-O notation for memory (or storage)*
 - Storing a vector of size n requires $8n$ bytes in RAM. Hence, we say that storing a vector is $O(n)$.
 - Storing a dense matrix, i.e., a matrix with no zeros (or with zeros but without a structure), scales as $O(n^2)$. However, storing a band matrix requires $O(n)$ storage if the bandwidth is small $b \ll n$.

3. Linear Equations (direct methods)

System of linear equations

- **Rank of a matrix:** The rank of a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ (where $\mathbb{R}^{m \times n}$ means the set of all real-valued $m \times n$ matrices) is the number of linearly independent columns of a matrix.
 - The rank of a matrix is easy to demonstrate using the singular value decomposition (SVD) of the matrix $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$, where the diagonal matrix $\mathbf{\Sigma}$ contains the singular values (which are ≥ 0).
 - Singular values are square roots of eigenvalues of $\mathbf{A}^T\mathbf{A}$. So, they are never negative.
 - The rank of $\mathbf{A}$ is the number of non-zero singular values of $\mathbf{\Sigma}$.
 - Important point: The SVD of the transpose of a matrix is $\mathbf{A}^T = (\mathbf{U}\mathbf{\Sigma}\mathbf{V}^T)^T = \mathbf{V}\mathbf{\Sigma}\mathbf{U}^T$ with the same singular values. Hence, the rank is also equal to the number of its linearly independent rows.
- **Example 1:** The rank of the matrix $\begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{bmatrix}$ is 1 because the two columns are proportional to one another. So, there is only one linearly independent column.
- **Example 2:** The rank of the matrix $\begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{bmatrix}$ is 2 because the two columns are linearly independent. The rank of the matrix $\begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}$ is also 2 not only because the sum of first two columns equates third but also that the number of linearly independent rows is 2.

```
A=np.array([[1,2], [2,4], [3,6]])
U,S,Vt=np.linalg.svd(A)
print('Singular values of A are:',S)
```

Singular values of A are: [8.36660027e+00 4.11373409e-16]

```
M=A.shape[0]; N=A.shape[1] # No. of rows, columns
tol=S.max() * max(M, N) * sys.float_info.epsilon # or tol=1e-14, etc.
r=np.sum(S>tol) # S > tol = [True, False] or so
print('Rank of A is:',r)
```

Rank of A is: 1

```
r=np.linalg.matrix_rank(A)
print('Rank of A is:',r)
```

Rank of A is: 1

- *Linear system*: If we have m linear equations involving n variables

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n &= b_1 \\ a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n &= b_2 \\ &\vdots \\ a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n &= b_m \end{aligned}$$

We can write it compactly as $\mathbf{Ax} = \mathbf{b}$, where $\mathbf{A} \in \mathbb{R}^{m \times n}$.

- If $m = n$, and if the rank of the matrix $\mathbf{A}$ is n (full-rank square system), then the system has a unique solution given formally by $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$.
- If $m = n$, and if the rank of the matrix $\mathbf{A}$ is $< n$ (rank-deficient square system), then the system has no or infinitely many solutions.
- If $m > n$ (more equations than unknown), we have an overspecified system (or overdetermined system). In this case, usually there is no exact solution.
- If $m < n$, we have an underspecified system (or underdetermined systems). If the rank of the matrix $\mathbf{A}$ is m (full-rank system), then the system can have infinitely many exact solutions.
- The term full-rank means $\min(m, n)$.

- *Exact and least squares solution*:

- For $m = n$ (and rank of $\mathbf{A} = n$), and $m < n$ (and rank of $\mathbf{A} = m$), the linear system has exact solutions with L.H.S equating R.H.S (unique in the first case, infinitely many in the second).
- If $m > n$, then typically an exact solution does not exist (as we have fewer unknowns to explain many equations). So, the solution in this case is the one that minimizes the norm of $\mathbf{Ax} - \mathbf{b}$.

■ **Consistent system:** A system is consistent if there exists $\mathbf{x}$ such that $\mathbf{Ax} = \mathbf{b}$.

- In other words, a system is consistent only if $\mathbf{b}$ is a linear combination of the columns of $\mathbf{A}$.
- If no $\mathbf{x}$ exists, the system is inconsistent.

■ **Formal solutions:** Let's consider the system $\mathbf{Ax} = \mathbf{b}$, where $\mathbf{A} \in \mathbb{R}^{m \times n}$.

1. For $m = n$ (rank of $\mathbf{A} = n$), $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$
2. For $m > n$ (rank of $\mathbf{A} = n$), the least squares solution is $\mathbf{x} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{b}$. The solution minimizes the residual norm $\|\mathbf{Ax} - \mathbf{b}\|_2$
3. For $m < n$ (rank of $\mathbf{A} = m$), then, there are infinitely many exact solutions. The solution $\mathbf{x} = \mathbf{A}^T (\mathbf{A} \mathbf{A}^T)^{-1} \mathbf{b}$ minimizes the solution norm $\|\mathbf{x}\|_2$
4. As a special case, suppose we have a rank, then if the system is consistent, we can find the minimum-norm exact solution. If the system is inconsistent, we can find the minimum-norm least squares solution using singular value decomposition (SVD).

■ **Pseudo inverse:** For all cases, we can give the solution as $\mathbf{x} = \mathbf{A}^\dagger \mathbf{b}$, where $\mathbf{A}^\dagger$ is called the pseudo-inverse

1. $\mathbf{A}^\dagger = \mathbf{A}^{-1}$
2. $\mathbf{A}^\dagger = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T$
3. $\mathbf{A}^\dagger = \mathbf{A}^T (\mathbf{A} \mathbf{A}^T)^{-1}$
4. $\mathbf{A}^\dagger = \mathbf{V} \mathbf{\Sigma}^\dagger \mathbf{U}^T$ (from SVD). This is the most general formula (of Moore-Penrose pseudoinverse) and other three can be shown as its special case.

Full-rank square system

- *Diagonal system*: Suppose $\mathbf{A} \in \mathbb{R}^{n \times n}$ is a diagonal matrix

$$\begin{array}{rcl} a_{11}x_1 + 0 + \dots + 0 = b_1 & & a_{11}x_1 = b_1 \\ 0 + a_{22}x_2 + \dots + 0 = b_2 & \text{or} & a_{22}x_2 = b_2 \\ \vdots & & \vdots \\ 0 + 0 + \dots + a_{nn}x_n = b_n & & a_{nn}x_n = b_n \end{array}$$

the solution takes the closed form $x_i = b_i/a_{ii}$. The algorithm to solve this system is as follows

```
def solvediag(A,b):  
    n=len(b); x=np.zeros(n)  
    for i in range(n):  
        x[i]=b[i]/A[i,i]  
    return x
```

```
A=np.array([[3,0,0],[0,2,0],[0,0,7]]); b=np.array([4,8,21])  
x=solvediag(A,b)
```

```
print(x)
```

```
[1.33333333 4.          3.          ]
```

```
x=np.linalg.solve(A,b); print(x)
```

```
[1.33333333 4.          3.          ]
```

- For most part of this section, you can verify the solution to linear systems using **linalg.solve** method from **numpy**

- *Recall big-O notation*

- The runtime complexity of the solution of a diagonal system is exactly n , or $O(n)$.

Triangular systems

- **Upper triangular system:** Suppose $\mathbf{A} \in \mathbb{R}^{n \times n}$ is such that $a_{ij} = 0$ when $i > j$.

$$\begin{array}{ccc} a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n = b_1 & & a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n = b_1 \\ 0 + a_{22}x_2 + \dots + a_{2n}x_n = b_2 & \text{or} & a_{22}x_2 + \dots + a_{2n}x_n = b_2 \\ \vdots & & \vdots \\ 0 + 0 + \dots + a_{nn}x_n = b_n & & a_{nn}x_n = b_n \end{array}$$

then, the solution algorithm is as follows.

- First, solve the last equation as $x_n = b_n/a_{nn}$. The penultimate equation is $a_{n-1,n-1}x_{n-1} + a_{n,n}x_n = b_{n-1}$ which can be solved as $x_{n-1} = \frac{1}{a_{n-1,n-1}} (b_{n-1} - a_{n,n}x_n)$. This is called backward substitution.
- Similarly, $x_{n-2} = \frac{1}{a_{n-2,n-2}} (b_{n-2} - a_{n-2,n-1}x_{n-1} - a_{n-2,n}x_n)$, and in general, $x_i = \frac{1}{a_{ii}} \left(b_i - \sum_{j=i+1}^n a_{ij}x_j \right)$ with $i = n, n-1, \dots, 1$.

```
def solveupper(U,b):
    n=len(b); x=np.zeros(n)
    for i in range(n-1,-1,-1):
        x[i]=b[i]
        for j in range(i+1,n):
            x[i]-=U[i,j]*x[j]
        x[i]=x[i]/U[i,i]
    return x

U=np.array([[2,1,-3],[0,-2,1],[0,0,3]]); b=np.array([-10,-2,6])
x=solveupper(U,b); print(x)
```

[-3. 2. 2.]

- The runtime complexity is exactly $n(n-1)/2 \times 2 + n = n(n-1) + n = n^2$, i.e., the runtime scaling is $O(n^2)$.

- **Lower triangular system:** Suppose $\mathbf{A} \in \mathbb{R}^{n \times n}$ is such that $a_{ij} = 0$ when $i < j$.

$$\begin{aligned} a_{11}x_1 &= b_1 \\ a_{21}x_1 + a_{22}x_2 &= b_2 \\ &\vdots \\ a_{n1}x_1 + a_{n2}x_2 + \dots + a_{nn}x_n &= b_n \end{aligned}$$

then, the solution algorithm is as follows.

- First, solve the first equation as $x_1 = b_1/a_{11}$. The second equation is $a_{21}x_1 + a_{22}x_2 = b_2$ which can be solved as $x_2 = \frac{1}{a_{22}} (b_2 - a_{21}x_1)$. This is called forward substitution.
- Similarly, $x_3 = \frac{1}{a_{33}} (b_3 - a_{32}x_2 - a_{31}x_1)$, and in general, $x_i = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j \right)$ with $i = 1, 2, \dots, n$

```
def solvelower(L,b):
    n=len(b); x=np.zeros(n)
    for i in range(n):
        x[i]=b[i]
        for j in range(i):
            x[i]-=L[i,j]*x[j]
        x[i]=x[i]/L[i,i]
    return x

L=np.array([[3,0,0],[1,-2,0],[2,1,-3]]); b=np.array([-10,-2,6])

x=solverlower(L,b); print(x)
```

`[-3.33333333 -0.66666667 -4.44444444]`

Gaussian elimination (without pivoting)

■ *Elementary row operations*: The following operations leave the linear system invariant.

- Multiplying a row (i.e., an equation) by a scalar $R_i \leftarrow sR_i$
- Exchange of two equations $R_i \leftrightarrow R_j$
- Replacing a row with the same row after subtracting a multiple of another row $R_i \leftarrow R_i - sR_j$

```
A0=np.array([[1,2,3],[3,2,4],[4,4,3]],dtype=float); b0=np.array([5,17,26],dtype=float)
x=np.linalg.solve(A0,b0); print(x)
```

```
[ 6.5   0.75 -1. ]
```

```
A=A0.copy(); b=b0.copy()
i=1; s=0.5
A[i,:]=A[i,:]*s; b[i]=b[i]*s
x=np.linalg.solve(A,b); print(x)
```

```
[ 6.5   0.75 -1. ]
```

```
A=A0.copy(); b=b0.copy()
i,j=0,2
A[[i,j],:] = A[[j,i],:]; b[[i,j]] = b[[j,i]]
x=np.linalg.solve(A,b); print(x)
```

```
[ 6.5   0.75 -1. ]
```

```
i,j=0,2; s=0.5
A[i,:]=A0[i,:]-A0[j,:]*s; b[i]=b0[i]-b0[j]*s
x=np.linalg.solve(A,b); print(x)
```

```
[ 6.5   0.75 -1. ]
```

- **Gaussian elimination:** The algorithm (or steps) to convert a linear system into a row-echelon form (or row step form). We will start with the augmented matrix

$$[A \mid b] = \left[\begin{array}{cccc|c} a_{11} & a_{12} & \cdots & a_{1n} & b_1 \\ a_{21} & a_{22} & \cdots & a_{2n} & b_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} & b_n \end{array} \right]$$

and apply elementary row operations to get

$$[\tilde{A} \mid \tilde{b}] = \left[\begin{array}{cccc|c} a_{11} & a_{12} & \cdots & a_{1n} & b_1 \\ 0 & \tilde{a}_{22} & \cdots & \tilde{a}_{2n} & \tilde{b}_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & \tilde{a}_{nn} & \tilde{b}_n \end{array} \right]$$

- If we perform the operation $R_2 \leftarrow R_2 - R_1 \frac{a_{12}}{a_{11}}$, we get

$$[A \mid b] = \left[\begin{array}{cccc|c} a_{11} & a_{12} & \cdots & a_{1n} & b_1 \\ a_{21} - \frac{a_{21}}{a_{11}}a_{11} & a_{22} - \frac{a_{21}}{a_{11}}a_{12} & \cdots & a_{2n} - \frac{a_{21}}{a_{11}}a_{1n} & b_2 - \frac{a_{21}}{a_{11}}b_1 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} & b_n \end{array} \right] = \left[\begin{array}{cccc|c} a_{11} & a_{12} & \cdots & a_{1n} & b_1 \\ 0 & \tilde{a}_{22} & \cdots & \tilde{a}_{2n} & \tilde{b}_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} & b_n \end{array} \right]$$

- If we continue to do $R_3 \leftarrow R_3 - R_1 \frac{a_{13}}{a_{11}}$ until $R_n \leftarrow R_n - R_1 \frac{a_{1n}}{a_{11}}$, we can eliminate the remaining elements in the first column (except the element in the first row).
- Shortly, use the first row to eliminate the rest of the first column.
- The algorithm will fail if $a_{11} = 0$ (or if any of the diagonal element is zero).

■ *Algorithm:* Let's go through the algorithm step-by-step

```
A=np.array([[2,5,5,3],[3,3,1,3],[1,4,1,2],[5,5,1,2]], dtype=float); b=np.array([4,3,3,4], dtype=float)
```

```
Ab0 = np.hstack((A, b[:, None]))
Ab=Ab0.copy()
#print(Ab)
```

```
i=0; j=1
m=Ab[j,i]/Ab[i,i]; Ab[j,:]=Ab[j,:]-m*Ab[i,:]
#print(Ab)
i=0; j=2
m=Ab[j,i]/Ab[i,i]; Ab[j,:]=Ab[j,:]-m*Ab[i,:]
#print(Ab)
i=0; j=3
m=Ab[j,i]/Ab[i,i]; Ab[j,:]=Ab[j,:]-m*Ab[i,:]; print(Ab)
```

```
[[ 2.   5.   5.   3.   4. ]
 [ 0.  -4.5 -6.5 -1.5 -3. ]
 [ 0.   1.5 -1.5  0.5  1. ]
 [ 0.  -7.5 -11.5 -5.5 -6. ]]
```

```
i=1; j=2
m=Ab[j,i]/Ab[i,i]; Ab[j,:]=Ab[j,:]-m*Ab[i,:]
#print(Ab)
i=1; j=3
m=Ab[j,i]/Ab[i,i]; Ab[j,:]=Ab[j,:]-m*Ab[i,:]
print(Ab)
```

```
[[ 2.   5.   5.   3.   4. ]
 [ 0.  -4.5 -6.5 -1.5 -3. ]
 [ 0.   0. -3.66666667  0.   0. ]
 [ 0.   0. -0.66666667 -3.  -1. ]]
```

```
i=2; j=3
m=Ab[j,i]/Ab[i,i]; Ab[j,:]=Ab[j,:]-m*Ab[i,:]
print(Ab)
```

```
[[ 2.   5.   5.   3.   4. ]
 [ 0.  -4.5 -6.5 -1.5 -3. ]
 [ 0.   0. -3.66666667  0.   0. ]
 [ 0.   0.   0.  -3.  -1. ]]
```

■ *Algorithm:* Let's loop over j

```
A=np.array([[2,5,5,3],[3,3,1,3],[1,4,1,2],[5,5,1,2]], dtype=float); b=np.array([4,3,3,4], dtype=float)
```

```
Ab0 = np.hstack((A, b[:, None]))  
Ab=Ab0.copy()  
#print(Ab)
```

```
n=len(b)  
i=0  
for j in range(i+1,n):  
    m=Ab[j,i]/Ab[i,i]; Ab[j,:]=Ab[j,:]-m*Ab[i,:]  
print(Ab)
```

```
[[ 2.   5.   5.   3.   4. ]  
 [ 0.  -4.5 -6.5 -1.5 -3. ]  
 [ 0.   1.5 -1.5  0.5  1. ]  
 [ 0.  -7.5 -11.5 -5.5 -6. ]]
```

```
i=1  
for j in range(i+1,n):  
    m=Ab[j,i]/Ab[i,i]; Ab[j,:]=Ab[j,:]-m*Ab[i,:]  
print(Ab)
```

```
[[ 2.   5.   5.   3.   4. ]  
 [ 0.  -4.5 -6.5 -1.5 -3. ]  
 [ 0.   0.  -3.66666667  0.   0. ]  
 [ 0.   0.  -0.66666667 -3.  -1. ]]
```

```
i=2  
for j in range(i+1,n):  
    m=Ab[j,i]/Ab[i,i]; Ab[j,:]=Ab[j,:]-m*Ab[i,:]  
print(Ab)
```

```
[[ 2.   5.   5.   3.   4. ]  
 [ 0.  -4.5 -6.5 -1.5 -3. ]  
 [ 0.   0.  -3.66666667  0.   0. ]  
 [ 0.   0.   0.  -3.  -1. ]]
```

■ *Algorithm:* Loop over both *i* and *j*

```
A=np.array([[2,5,5,3],[3,3,1,3],[1,4,1,2],[5,5,1,2]], dtype=float); b0=np.array([4,3,3,4], dtype=float)
Ab0 = np.hstack((A, b0[:, None]))
Ab=Ab0.copy()
#print(Ab)
```

```
n=len(b)
for i in range(n-1):
    for j in range(i+1,n):
        m=Ab[j,i]/Ab[i,i]; Ab[j,:]=Ab[j,:]-m*Ab[i,:]
print(Ab)
```

```
[[ 2.         5.         5.         3.         4.        ]
 [ 0.        -4.5        -6.5        -1.5        -3.        ]
 [ 0.         0.        -3.66666667  0.         0.        ]
 [ 0.         0.         0.         -3.         -1.        ]]
```

```
U = Ab[:, :-1]      # all rows, first n columns
b = Ab[:, -1]       # all rows, last column
print(U,b)
```

```
[[ 2.         5.         5.         3.        ]
 [ 0.        -4.5        -6.5        -1.5        ]
 [ 0.         0.        -3.66666667  0.        ]
 [ 0.         0.         0.         -3.        ]] [ 4. -3.  0. -1.]
```

```
x=solveupper(U,b); print(x)
```

```
[ 0.11111111  0.55555556 -0.         0.33333333]
```

```
x=np.linalg.solve(A,b0); print(x)
```

```
[ 1.11111111e-01  5.55555556e-01 -3.60335543e-17  3.33333333e-01]
```

■ Complete code for Gaussian elimination for a full-rank square system

```
def gausselim(A,b0):

    # back substitution
    def solveupper(U,b):
        n=len(b); x=np.zeros(n)
        for i in range(n-1,-1,-1):
            x[i]=b[i]
            for j in range(i+1,n):
                x[i]-=U[i,j]*x[j]
            x[i]=x[i]/U[i,i]
        return x

    # Augmented matrix
    Ab0 = np.hstack((A, b0[:, None]))

    # Gaussian elimination
    Ab=Ab0.copy()
    n=len(b0)
    for i in range(n-1):
        for j in range(i+1,n):
            m=Ab[j,i]/Ab[i,i]
            # Ab[j,:]=Ab[j,:]-m*Ab[i,:]
            for k in range(n+1):
                Ab[j,k]=Ab[j,k]-m*Ab[i,k]

    U = Ab[:, :-1]
    b = Ab[:, -1]

    x=solveupper(U,b)

    return x
```

$O(n)$ iterations
 $O(n)$ iterations
 $O(n)$ iterations
 $O(n^2)$ iterations

○ Total runtime complexity is $O(n^3)$.

```
A=np.array([[2,5,5,3],[3,3,1,3],[1,4,1,2],[5,5,1,2]], dtype=float); b=np.array([4,3,3,4], dtype=float)
x=gausselim(A,b)
print(x)
```

```
[ 0.11111111  0.55555556 -0.          0.33333333]
```

LU decomposition or LU factorization (without pivoting)

- In Gaussian elimination, we solve $\mathbf{Ax} = \mathbf{b}$ by converting the augmented matrix $[\mathbf{A} \mid \mathbf{b}]$ into $[\tilde{\mathbf{A}} \mid \tilde{\mathbf{b}}] = [\mathbf{U} \mid \tilde{\mathbf{b}}]$ and solve $\mathbf{Ux} = \tilde{\mathbf{b}}$ by back substitution. In general, one can solve multiple linear systems of the form $\mathbf{A}[\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n] = [\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_n]$, starting with $[\mathbf{A} \mid \mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_n]$.
- It is more convenient to have a procedure to calculate $\mathbf{U}$ that does not involve the constant vector $\mathbf{b}$.
- In LU decomposition, $\mathbf{A}$ is decomposed as a product of a lower and an upper triangular matrix, $\mathbf{A} = \mathbf{LU}$. The original problem $\mathbf{Ax} = \mathbf{b}$ is solved as $\mathbf{LUx} = \mathbf{b}$.
- The first step is to solve $\mathbf{Ly} = \mathbf{b}$, then one solves $\mathbf{Ux} = \mathbf{y}$.
- *A special case of a nilpotent matrix:* $\mathbf{N}$ is called a nilpotent matrix $\mathbf{N}^k = \mathbf{0}$ for integer value of k
 - A submultiplicative matrix norm follows $\|\mathbf{AB}\| \leq \|\mathbf{A}\| \|\mathbf{B}\|$. Hence, $\|\mathbf{N}^k\| \leq \|\mathbf{N}\|^k$.
 - Further, if $\|\mathbf{N}\| \leq 1$, then $(\mathbf{I} + \mathbf{N})^{-1} = \mathbf{I} - \mathbf{N} + \mathbf{N}^2 - \mathbf{N}^3 + \dots$ converges. This series (which looks like Taylor series) is called as Neuman series.
 - If $\mathbf{N}^2 = \mathbf{0}$, then $(\mathbf{I} + \mathbf{N})^{-1} = \mathbf{I} - \mathbf{N}$, so $(\mathbf{I} + \mathbf{N})^{-1}(\mathbf{I} + \mathbf{N}) = (\mathbf{I} - \mathbf{N})(\mathbf{I} + \mathbf{N}) = \mathbf{I} + \mathbf{N} - \mathbf{N} - \mathbf{N}^2 = \mathbf{I}$.
 - A special case of $\mathbf{N}$ is $\mathbf{M} = m\mathbf{E}_{ij}$, where $\mathbf{E}_{ij}$ contains 1 at row- i and column- j , and 0 elsewhere. Hence $\mathbf{M}$ contains m at row- i and column- j , and 0 elsewhere.
 - $\mathbf{M}^2 = \mathbf{0}$

$$\mathbf{M} = \begin{bmatrix} 0 & 0 & \dots & 0 \\ 0 & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ m & 0 & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 0 \end{bmatrix}, (\mathbf{I} + \mathbf{M})^{-1} = \begin{bmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ -m & 0 & 1 & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{bmatrix}$$

■ Row reduction as matrix multiplication:

```
A0=np.array([[1,2,1,-1],[3,2,4,4],[4,4,3,4],[2,0,1,5]]); A0
```

```
array([[ 1,  2,  1, -1],
       [ 3,  2,  4,  4],
       [ 4,  4,  3,  4],
       [ 2,  0,  1,  5]])
```

```
A=A0.copy()
```

```
i=0; j=1; m=A[j,i]/A[i,i]; M01=np.identity(4); M01[j,i]=-m; A= M01 @ A; print(M, '\n', A)
```

```
[[ 1.  0.  0. ]
 [-0.1 1.  0. ]
 [ 0.  0.  1. ]]
[[ 1.  2.  1. -1.]
 [ 0. -4.  1.  7.]
 [ 4.  4.  3.  4.]
 [ 2.  0.  1.  5.]]
```

→ remove second-row, first column

```
A=A0.copy()
```

```
i=0; j=1; m=A[j,i]/A[i,i]; M01=np.identity(4); M01[j,i]=-m; A= M01 @ A
i=0; j=2; m=A[j,i]/A[i,i]; M02=np.identity(4); M02[j,i]=-m; A= M02 @ A
i=0; j=3; m=A[j,i]/A[i,i]; M03=np.identity(4); M03[j,i]=-m; A= M03 @ A; print(A)
```

```
[[ 1.  2.  1. -1.]
 [ 0. -4.  1.  7.]
 [ 0. -4. -1.  8.]
 [ 0. -4. -1.  7.]]
```

→ remove the rest of first column

```
A=A0.copy()
```

```
i=0; j=1; m=A[j,i]/A[i,i]; M01=np.identity(4); M01[j,i]=-m; A= M01 @ A
i=0; j=2; m=A[j,i]/A[i,i]; M02=np.identity(4); M02[j,i]=-m; A= M02 @ A
i=0; j=3; m=A[j,i]/A[i,i]; M03=np.identity(4); M03[j,i]=-m; A= M03 @ A
i=1; j=2; m=A[j,i]/A[i,i]; M12=np.identity(4); M12[j,i]=-m; A= M12 @ A
i=1; j=3; m=A[j,i]/A[i,i]; M13=np.identity(4); M13[j,i]=-m; A= M13 @ A
i=2; j=3; m=A[j,i]/A[i,i]; M23=np.identity(4); M23[j,i]=-m; U = M23 @ A
```

U

```
array([[ 1.,  2.,  1., -1.],
       [ 0., -4.,  1.,  7.],
       [ 0.,  0., -2.,  1.],
       [ 0.,  0.,  0., -1.]])
```

Upper triangular matrix, U

■ **M** as a nilpotent matrix:

- From the above exercise, we can write $\mathbf{U} = \mathbf{M}\mathbf{A}$, where $\mathbf{U} = \prod_{ij} \mathbf{M}_{ij} \mathbf{A}$
- Note that $\mathbf{M}_{ij} = \mathbf{I} - m\mathbf{E}_{ij}$, and we know that $\mathbf{M}_{ij}^{-1} = (\mathbf{I} - m\mathbf{E}_{ij})^{-1} = \mathbf{I} + m\mathbf{E}_{ij}$.
- Since, $\mathbf{M} = \mathbf{M}_{34}\mathbf{M}_{24}\mathbf{M}_{14}\mathbf{M}_{23}\mathbf{M}_{13}\mathbf{M}_{12}$, we have $\mathbf{M}^{-1} = \mathbf{M}_{12}^{-1}\mathbf{M}_{13}^{-1}\mathbf{M}_{23}^{-1}\mathbf{M}_{14}^{-1}\mathbf{M}_{24}^{-1}\mathbf{M}_{34}^{-1}$
- So, we have

$$\mathbf{M}_{ij} = \begin{bmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & -m_{ij} & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{bmatrix}, \mathbf{M}_{ij}^{-1} = \begin{bmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & m_{ij} & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{bmatrix}$$

- Overall, $\mathbf{U} = \mathbf{M}\mathbf{A} \implies \mathbf{M}^{-1}\mathbf{U} = \mathbf{M}^{-1}\mathbf{M}\mathbf{A} = \mathbf{A}$. Since, $\mathbf{M}^{-1}$ is a lower-triangular matrix, we denote it as $\mathbf{L}$. So, we have $\mathbf{L}\mathbf{U} = \mathbf{A}$, where $\mathbf{M}^{-1} = \mathbf{L}$ (which collects all the m values).

■ LU-decomposition (expanded):

- Store m at each step to form the $\mathbf{L}$ matrix

```
A0=np.array([[1,2,1,-1],[3,2,4,4],[4,4,3,4],[2,0,1,5]])
```

```
A=A0.copy()
```

```
L=np.eye(4)
```

```
i=0; j=1; m=A[j,i]/A[i,i]; M01=np.identity(4); M01[j,i]=-m; A= M01 @ A; L[j,i]=m;
i=0; j=2; m=A[j,i]/A[i,i]; M02=np.identity(4); M02[j,i]=-m; A= M02 @ A; L[j,i]=m;
i=0; j=3; m=A[j,i]/A[i,i]; M03=np.identity(4); M03[j,i]=-m; A= M03 @ A; L[j,i]=m;
i=1; j=2; m=A[j,i]/A[i,i]; M12=np.identity(4); M12[j,i]=-m; A= M12 @ A; L[j,i]=m;
i=1; j=3; m=A[j,i]/A[i,i]; M13=np.identity(4); M13[j,i]=-m; A= M13 @ A; L[j,i]=m;
i=2; j=3; m=A[j,i]/A[i,i]; M23=np.identity(4); M23[j,i]=-m; U = M23 @ A; L[j,i]=m;
```

Collect elements for the lower triangular matrix, $\mathbf{L}$



```
L,U,L@U,L@U==A0
```

```
(array([[1., 0., 0., 0.],
       [3., 1., 0., 0.],
       [4., 1., 1., 0.],
       [2., 1., 1., 1.]]),
 array([[ 1.,  2.,  1., -1.],
       [ 0., -4.,  1.,  7.],
       [ 0.,  0., -2.,  1.],
       [ 0.,  0.,  0., -1.]]),
 array([[ 1.,  2.,  1., -1.],
       [ 3.,  2.,  4.,  4.],
       [ 4.,  4.,  3.,  4.],
       [ 2.,  0.,  1.,  5.]]),
 array([[ True,  True,  True,  True],
       [ True,  True,  True,  True],
       [ True,  True,  True,  True],
       [ True,  True,  True,  True]]))
```

■ *LU-decomposition (compact):*

- Store m at each step to form the $\mathbf{L}$ matrix
- Find $\mathbf{U}$ as done in **gausselim** (on page 63)

```
A0=np.array([[1,2,1,-1],[3,2,4,4],[4,4,3,4],[2,0,1,5]], dtype=float)
A=A0.copy()

n = A.shape[0]

L=np.eye(n)

for i in range(n-1):          # i goes from 0 to 2
    for j in range(i+1,n):    # j goes from i+1 to 3
        m=A[j,i]/A[i,i]
        L[j,i]=m;
        A[j, :] -= m * A[i,:]
U=A

print(U,L,L@U,L@U==A0)
print(np.allclose(A0, L @ U))
```

```
[[ 1.  2.  1. -1.]
 [ 0. -4.  1.  7.]
 [ 0.  0. -2.  1.]
 [ 0.  0.  0. -1.]] [[1.  0.  0.  0.]
 [3.  1.  0.  0.]
 [4.  1.  1.  0.]
 [2.  1.  1.  1.]] [[ 1.  2.  1. -1.]
 [ 3.  2.  4.  4.]
 [ 4.  4.  3.  4.]
 [ 2.  0.  1.  5.]] [[ True  True  True  True]
 [ True  True  True  True]
 [ True  True  True  True]
 [ True  True  True  True]]
True
```

- Complete code for solving a full-rank square system using LU decomposition
- The original problem $\mathbf{Ax} = \mathbf{b}$ will be solved as $\mathbf{Ly} = \mathbf{b}$, and $\mathbf{Ux} = \mathbf{y}$.

```
def LU(A,b0):
```

```
    # back substitution
```

```
    def solveupper(U,b):
```

```
        n=len(b); x=np.zeros(n)
```

```
        for i in range(n-1,-1,-1):
```

```
            x[i]=b[i]
```

```
            for j in range(i+1,n):
```

```
                x[i]-=U[i,j]*x[j]
```

```
            x[i]=x[i]/U[i,i]
```

```
        return x
```

```
    # forward substitution
```

```
    def solvelower(L, b):
```

```
        n = len(b); x = np.zeros(n)
```

```
        for i in range(n):
```

```
            x[i] = b[i]
```

```
            for j in range(i):
```

```
                x[i] -= L[i, j] * x[j]
```

```
            x[i] = x[i]/L[i, i]
```

```
        return x
```

```
    # LU decomposition
```

```
    U=A.copy(); n=A.shape[0]; L=np.eye(n)
```

```
    for i in range(n-1):
```

```
        for j in range(i+1,n):
```

```
            m=U[j,i]/U[i,i]
```

```
            L[j,i]=m;
```

```
            U[j, :] -= m * U[i,:]
```

```
    y=solverlower(L,b); x=solveupper(U,y)
```

```
    return L, U, x
```

◦ Total runtime complexity is $O(n^3)$.

$O(n)$ iterations

$O(n)$ iterations

$O(n)$ iterations

$O(n^2)$ iterations

$O(n^2)$ iterations

```
A=np.array([[2,5,5,3],[3,3,1,3],[1,4,1,2],[5,5,1,2]], dtype=float); b=np.array([4,3,3,4], dtype=float)
L,U,x=LU(A,b)
print(x)
```

```
[ 0.11111111  0.55555556 -0.          0.33333333]
```

■ Compact storage:

- The diagonal elements of $\mathbf{L}$ are 1. So, the other non-zero elements can be stored along with $\mathbf{U}$

```
def LU_compact_storage(A,b0):  
  
    # back substitution  
    def solveupper(U,b):  
        n=len(b); x=np.zeros(n)  
        for i in range(n-1,-1,-1):  
            x[i]=b[i]  
            for j in range(i+1,n):  
                x[i]-=U[i,j]*x[j]  
            x[i]=x[i]/U[i,i]  
        return x  
  
    # forward substitution  
    def solvelower(L, b):  
        n = len(b); x = np.zeros(n)  
        for i in range(n):  
            x[i] = b[i]  
            for j in range(i):  
                x[i] -= L[i, j] * x[j]  
            # x[i] = x[i]/L[i, i] #<--- skipped as implicitly the diagonal elements are 1  
        return x  
  
    # LU decomposition  
    UL=A.copy(); n=A.shape[0]  
    for i in range(n-1):  
        for j in range(i+1,n):  
            m=UL[j,i]/UL[i,i]  
            UL[j,i]=m;  
            UL[j, i+1:] -= m * UL[i,i+1:] #<--- only the upper triangular is updated  
  
    y=solverlower(UL,b); x=solveupper(UL,y)  
  
    return UL, x
```

```
A=np.array([[2,5,5,3],[3,3,1,3],[1,4,1,2],[5,5,1,2]], dtype=float); b=np.array([4,3,3,4], dtype=float)  
UL,x=LU_compact_storage(A,b)  
print(x)
```

```
[ 0.11111111  0.55555556 -0.          0.33333333]
```

Pivoting

- Gaussian elimination and LU decomposition fail if during row-reduction, the diagonal elements entering as denominator becomes zero. This is shown in the following example.

```
A0=np.array([[-1,2,5],[2,-4,5],[-1,0,5]], dtype=float)

A = A0.copy()
n = A.shape[0]
L = np.eye(n)

for i in range(n-1):
    for j in range(i+1, n):
        print(i,j,A)
        m = A[j, i] / A[i, i]
        L[j, i] = m          # store the multiplier in L
        A[j, :] -= m * A[i,:] # elimination step

U = A
```

```
0 1 [[-1.  2.  5.]
      [ 2. -4.  5.]
      [-1.  0.  5.]]
0 2 [[-1.  2.  5.]
      [ 0.  0. 15.]
      [-1.  0.  5.]]
1 2 [[-1.  2.  5.]
      [ 0.  0. 15.]
      [ 0. -2.  0.]]
```

```
/var/folders/ph/rspflgdn68n9z7kh87zq8n9m0000gn/T/ipykernel_58351/3930046121.py:10: RuntimeWarning: divide by zero encountered in scalar divide
  m = A[j, i] / A[i, i]
/var/folders/ph/rspflgdn68n9z7kh87zq8n9m0000gn/T/ipykernel_58351/3930046121.py:12: RuntimeWarning: invalid value encountered in multiply
  A[j, :] -= m * A[i,:] # elimination step
```

■ Partial pivoting:

- For each column i , among rows $i, i + 1, \dots, n - 1$, swap rows such that A_{ii} has maximum absolute value
- With rows permuted, the equations become $\mathbf{PA} = \mathbf{LU}$.

```
A0=np.array([[ -1,2,5],[2,-4,5],[ -1,0,5]], dtype=float)
```

```
A = A0.copy(); n = A.shape[0]; L = np.eye(n); P = np.eye(n)
```

```
for i in range(n-1):
```

```
    # look at rows i..n-1 in column i with max value
    pivot_row = i + np.argmax(np.abs(A[i:, i]))
```

```
    # if pivot is (numerically) zero stop
```

```
    if np.abs(A[pivot_row, i]) < 1e-12:
        raise ValueError("Matrix is singular (pivot ~ 0)")
```

```
    # swap rows i and pivot_row in A, P, and L
```

```
    if pivot_row != i:
        A[[i, pivot_row], :] = A[[pivot_row, i], :] # swap rows in A
        P[[i, pivot_row], :] = P[[pivot_row, i], :] # swap rows in P
```

```
        if i > 0:
```

```
            L[[i, pivot_row], :i] = L[[pivot_row, i], :i] # swap rows in L, but only columns 0..i-1
```

```
    for j in range(i+1, n):
```

```
        m = A[j, i] / A[i, i]
        L[j, i] = m
        A[j, :] -= m * A[i, :]
```

```
U = A
P
```

```
array([[0., 1., 0.],
       [0., 0., 1.],
       [1., 0., 0.]])
```

A0

```
array([[ -1.,  2.,  5.],
       [  2., -4.,  5.],
       [ -1.,  0.,  5.]])
```

P @ A0

```
array([[ 2., -4.,  5.],
       [-1.,  0.,  5.],
       [-1.,  2.,  5.]])
```

L @ U

```
array([[ 2., -4.,  5.],
       [-1.,  0.,  5.],
       [-1.,  2.,  5.]])
```

■ Using `scipy.linalg.lu`

```
import numpy as np
from scipy.linalg import lu

A=np.array([[ -1,2,5],[2,-4,5],[ -1,0,5]], dtype=float)

p, l, u = lu(A)
```

```
print("P =\n", P)
print("L =\n", L)
print("U =\n", U)
print("Is PA = LU? ", np.allclose(P @ A, L @ U))
```

```
P =
[[0.  1.  0.]
 [0.  0.  1.]
 [1.  0.  0.]]
L =
[[ 1.  0.  0. ]
 [-0.5  1.  0. ]
 [-0.5 -0.  1. ]]
U =
[[ 2. -4.  5. ]
 [ 0. -2.  7.5]
 [ 0.  0.  7.5]]
Check PA = LU?  True
```

Cholesky decomposition for positive definite $\mathbf{A}$

■ Positive definite matrix:

- A positive definite matrix has to be symmetric
- $\mathbf{x}^T \mathbf{A} \mathbf{x} > 0$ for any nonzero vector, $\mathbf{x}$, which means all the eigenvalues of $\mathbf{A}$ are positive
- $\mathbf{A}$ is nonsingular; $|\mathbf{A}| > 0$
- All diagonal elements are positive
- The largest element of $\mathbf{A}$ lies on the diagonal

■ Cholesky decomposition:

- For a positive definite, $\mathbf{A}$, Cholesky decomposition is an efficient method
- The goal is to factorize $\mathbf{A}$ as $\mathbf{L}\mathbf{L}^T$.

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{12} & a_{22} & a_{23} \\ a_{13} & a_{23} & a_{33} \end{bmatrix} = \begin{bmatrix} L_{11} & 0 & 0 \\ L_{21} & L_{22} & 0 \\ L_{31} & L_{32} & L_{33} \end{bmatrix} \begin{bmatrix} L_{11} & L_{21} & L_{31} \\ 0 & L_{22} & L_{32} \\ 0 & 0 & L_{33} \end{bmatrix}$$

- First column

$$\blacklozenge a_{11} = L_{11}L_{11} \implies L_{11} = \sqrt{a_{11}}$$

$$\blacklozenge a_{12} = L_{11}L_{21} \implies L_{21} = a_{12}/L_{11}, a_{13} = L_{11}L_{31} \implies L_{31} = a_{13}/L_{11}$$

- Second column

$$\blacklozenge a_{22} = L_{21}^2 + L_{22}^2 \implies L_{22}^2 = a_{22} - L_{21}^2 \implies L_{22} = \sqrt{a_{22} - L_{21}^2}$$

$$\blacklozenge a_{23} = L_{31}L_{21} + L_{32}L_{22} \implies L_{32}L_{22} = a_{23} - L_{31}L_{21} \implies L_{32} = (a_{23} - L_{31}L_{21})/L_{22}$$

- Third column

$$\blacklozenge a_{33} = L_{31}^2 + L_{32}^2 + L_{33}^2 \implies L_{33}^2 = a_{33} - L_{31}^2 - L_{32}^2 \implies L_{33} = \sqrt{a_{33} - L_{31}^2 - L_{32}^2}$$

```

A=np.array([[9,-3,3,9],[-3,17,-1,-7],[3,-1,17,15],[9,-7,15,44]], dtype=float)

n=A.shape[0]
L=np.zeros((n,n))

L[0,0]=np.sqrt(A[0,0])
L[1,0]=A[0,1]/L[0,0]
L[2,0]=A[0,2]/L[0,0]
L[3,0]=A[0,3]/L[0,0]

L[1,1]=np.sqrt(A[1,1]-L[1,0]**2)
L[2,1]=(A[1,2]-L[2,0]*L[1,0])/L[1,1]
L[3,1]=(A[1,3]-L[3,0]*L[1,0])/L[1,1]

L[2,2]=np.sqrt(A[2,2]-(L[2,0]**2+L[2,1]**2))
L[3,2]=(A[2,3]-(L[3,0]*L[2,0]+L[3,1]*L[2,1]))/L[2,2]

L[3,3]=np.sqrt(A[3,3]-(L[3,0]**2+L[3,1]**2+L[3,2]**2))

```

L

```

array([[ 3.,  0.,  0.,  0.],
       [-1.,  4.,  0.,  0.],
       [ 1.,  0.,  4.,  0.],
       [ 3., -1.,  3.,  5.]])

```

L@L.T

```

array([[ 9., -3.,  3.,  9.],
       [-3., 17., -1., -7.],
       [ 3., -1., 17., 15.],
       [ 9., -7., 15., 44.]])

```

```

A=np.array([[9,-3,3,9],[-3,17,-1,-7],[3,-1,17,15],[9,-7,15,44]], dtype=float)

n=A.shape[0]
L=np.zeros((n,n))

L[0,0]=np.sqrt(A[0,0]-np.sum(L[0,:0]**2))
L[1,0]=(A[0,1]-np.sum(L[1,:0]*L[0,:0]))/L[0,0]
L[2,0]=(A[0,2]-np.sum(L[2,:0]*L[0,:0]))/L[0,0]
L[3,0]=(A[0,3]-np.sum(L[3,:0]*L[0,:0]))/L[0,0]

L[1,1]=np.sqrt(A[1,1]-np.sum(L[1,:1]**2))
L[2,1]=(A[1,2]-np.sum(L[2,:1]*L[1,:1]))/L[1,1]
L[3,1]=(A[1,3]-np.sum(L[3,:1]*L[1,:1]))/L[1,1]

L[2,2]=np.sqrt(A[2,2]-np.sum(L[2,:2]**2))
L[3,2]=(A[2,3]-np.sum(L[3,:2]*L[2,:2]))/L[2,2]

L[3,3]=np.sqrt(A[3,3]-np.sum(L[3,:3]**2))

L@L.T

```

```

array([[ 9., -3.,  3.,  9.],
       [-3., 17., -1., -7.],
       [ 3., -1., 17., 15.],
       [ 9., -7., 15., 44.]])

```

```

A=np.array([[9,-3,3,9],[-3,17,-1,-7],[3,-1,17,15],[9,-7,15,44]], dtype=float)

n=A.shape[0]
L=np.zeros((n,n))

for i in range(n):
    L[i,i]=np.sqrt(A[i,i]-np.sum(L[i,:i]**2))
    for j in range(i,n):
        L[j,i]=(A[i,j]-np.sum(L[j,:i]*L[i,:i]))/L[i,i]

L@L.T

```

```

array([[ 9., -3.,  3.,  9.],
       [-3., 17., -1., -7.],
       [ 3., -1., 17., 15.],
       [ 9., -7., 15., 44.]])

```

```

def Cholesky(A,b0):

    # back substitution
    def solveupper(U,b):
        n=len(b); x=np.zeros(n)
        for i in range(n-1,-1,-1):
            x[i]=b[i]
            for j in range(i+1,n):
                x[i]-=U[j,i]*x[j]
            x[i]=x[i]/U[i,i]
        return x

    # forward substitution
    def solvelower(L, b):
        n = len(b); x = np.zeros(n)
        for i in range(n):
            x[i] = b[i]
            for j in range(i):
                x[i] -= L[i, j] * x[j]
            x[i] = x[i]/L[i, i]
        return x

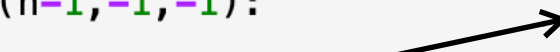
    # Cholesky
    n=A.shape[0]; L=np.zeros((n,n))
    for i in range(n):
        L[i,i]=np.sqrt(A[i,i]-np.sum(L[i,:i]**2))
        for j in range(i,n):
            L[j,i]=(A[i,j]-np.sum(L[j,:i]*L[i,:i]))/L[i,i]

    y=solverlower(L,b); x=solveupper(L,y)

    return L, U, x

```

Compare with page 70, $U[i,j]$



```

A=np.array([[9,-3,3,9],[-3,17,-1,-7],[3,-1,17,15],[9,-7,15,44]], dtype=float); b=np.array([4,3,3,4], dtype=float)
L,U,x=LU(A,b)
print(x)

```

```
[ 0.50444444  0.26916667  0.10916667 -0.00666667]
```

■ *Using `scipy.linalg.cho_factor` and `scipy.linalg.cho_solve`*

```
x=np.linalg.solve(A,b); print(x)
```

```
[ 0.50444444  0.26916667  0.10916667 -0.00666667]
```

```
import numpy as np
from scipy.linalg import cho_factor, cho_solve

A=np.array([[9,-3,3,9],[-3,17,-1,-7],[3,-1,17,15],[9,-7,15,44]], dtype=float); b=np.array([4,3,3,4], dtype=float)

c, low = cho_factor(A)
x = cho_solve((c, low), b)

print(x)
```

```
[ 0.50444444  0.26916667  0.10916667 -0.00666667]
```

```
np.allclose(A @ x - b, np.zeros(4))
```

True

- *Exercise:* Test the code **Cholesky** for $n = 100$ and $b_i = i$, and $A_{ii} = i$, $A_{ij} = 1/(i + j)$ and compare with the solution from `np.linalg.solve` and the `scipy.linalg.cho_factor` and `scipy.linalg.cho_solve` combo. You can calculate the CPU time as follows.

```
import time

start = time.time()

...

end = time.time()

time_elapsed = end-start
```

Example: Similar matrices of different behaviour

- Solve $\mathbf{A}_1 \mathbf{x}_1 = \mathbf{b}_1$ for $\mathbf{A}_1 = \begin{bmatrix} 0.45368292 & 0.19382865 \\ 0.70364726 & 0.52104011 \end{bmatrix}$, $\mathbf{b}_1 = \begin{bmatrix} 0.48554638 \\ 0.87421091 \end{bmatrix}$, and
 $\mathbf{A}_2 \mathbf{x}_2 = \mathbf{b}_2$ for $\mathbf{A}_2 = \begin{bmatrix} 0.40563526 & 0.26686200 \\ 0.73033346 & 0.48047658 \end{bmatrix}$, $\mathbf{b}_2 = \begin{bmatrix} 0.39812529 \\ 0.92276554 \end{bmatrix}$
- The system where the solution varies strongly with small changes in $\mathbf{b}$ is an ill-conditioned system
- Find $\kappa_2(\mathbf{A})$ for both systems.

```
A=np.array([
    [0.45368292, 0.19382865],
    [0.70364726, 0.52104011]
])
b=np.array([0.48554638, 0.87421091])

x=np.linalg.solve(A,b)

print(x)

U,S,VT=np.linalg.svd(A)
s1,s2=np.max(S),np.min(S)
kappa=s1/s2

print(kappa)
```

```
[0.83542019 0.54961178]
9.99999999585511
```

```
A=np.array([
    [0.40563526, 0.26686200],
    [0.73033346, 0.48047658]
])
b=np.array([0.39812529, 0.92276554])

x=np.linalg.solve(A,b)

print(x)

U,S,VT=np.linalg.svd(A)
s1,s2=np.max(S),np.min(S)
kappa=s1/s2

print(kappa)
```

```
[ 10272523.98653411 -15614428.20637149]
186905082.46031442
```

Exercise: Condition number

- Show that these vectors $\mathbf{v}_1 = [1, 2, 2]^T$, $\mathbf{v}_2 = [-2, -1, 2]^T$, and $\mathbf{v}_3 = [-2, 2, -1]^T$ are orthogonal to one another. Once normalized, these vectors form an orthonormal basis of $\mathbb{R}^3$. Normalize $\{\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3\}$ and save as $\{\mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3\}$.
- For $\mathbf{A} = \begin{bmatrix} 4 & 11 & 14 \\ 8 & 7 & -2 \end{bmatrix}$, verify that $\|\mathbf{A}\mathbf{e}_1\|^2 = \mathbf{e}_1^T \mathbf{A}^T \mathbf{A} \mathbf{e}_1 = 360$ or $\|\mathbf{A}\mathbf{e}_1\| = 6\sqrt{10} = 18.9737 = \sigma_1$ and $\|\mathbf{A}\mathbf{e}_2\|^2 = 90$ or $\|\mathbf{A}\mathbf{e}_2\| = 3\sqrt{10} = 9.4868 = \sigma_2$.
- Verify that for an arbitrary vector in $\mathbb{R}^3$, $\mathbf{x} = x_1\mathbf{e}_1 + x_2\mathbf{e}_2 + x_3\mathbf{e}_3$, $\mathbf{A}\mathbf{x} = \begin{bmatrix} 18x_1 + 3x_2 \\ 6x_1 - 9x_2 \end{bmatrix}$ and $\|\mathbf{A}\mathbf{x}\|^2 = (18x_1 + 3x_2)^2 + (6x_1 - 9x_2)^2 = 360x_1^2 + 90x_2^2$
- Verify that σ_1, σ_2 are the singular values of $\mathbf{A}$ and $\mathbf{V} = [\mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3]$ is the right singular vector: $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$
- Find the condition number, $\kappa_2(\mathbf{A})$, of $\mathbf{A}$ as the ratio between its maximum and minimum singular values.

EXAMPLE 1 If $A = \begin{bmatrix} 4 & 11 & 14 \\ 8 & 7 & -2 \end{bmatrix}$, then the linear transformation $\mathbf{x} \mapsto A\mathbf{x}$ maps the unit sphere $\{\mathbf{x} : \|\mathbf{x}\| = 1\}$ in $\mathbb{R}^3$ onto an ellipse in $\mathbb{R}^2$, shown in Fig. 1. Find a unit vector $\mathbf{x}$ at which the length $\|A\mathbf{x}\|$ is maximized, and compute this maximum length.

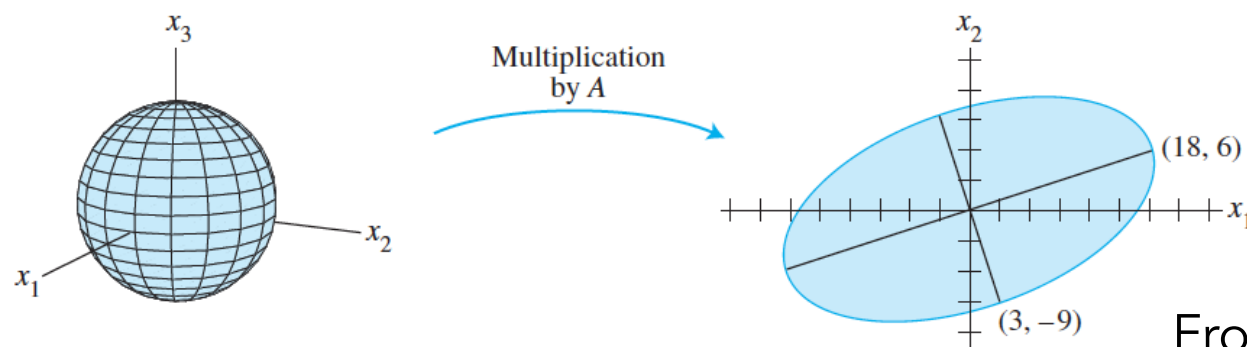


FIGURE 1 A transformation from $\mathbb{R}^3$ to $\mathbb{R}^2$.

From "Linear algebra and its applications" by David C. Lay

Exercise: Correlation matrix

- An equicorrelation matrix is defined as $\mathbf{A} = \begin{bmatrix} 1 & \rho & \rho & & \rho \\ \rho & 1 & \rho & & \rho \\ \rho & \rho & 1 & & \rho \\ & & & \ddots & \\ \rho & \rho & \rho & & 1 \end{bmatrix}$, where $\rho \in [0,1]$. For various values $\rho = 0, 0.1, 0.2, \dots, 1$ and $n = 10$ calculate $\kappa_2(\mathbf{A})$.

- *Solution:*

- This matrix has one non-repeating (i.e. multiplicity 1) eigenvalue $\lambda_1 = 1 + (n - 1)\rho$ and one repeating eigenvalue of multiplicity $n - 1$, $\lambda_2 = 1 - \rho$.
- For $\rho = 0$, $\kappa_2(\mathbf{A}) = \frac{\sigma_{\max}}{\sigma_{\min}} = \frac{|\lambda_{\max}|}{|\lambda_{\min}|} = \frac{1}{1} = 1$, and for $\rho = 1$, $\kappa_2(\mathbf{A}) = \frac{\sigma_{\max}}{\sigma_{\min}} = \frac{|\lambda_{\max}|}{|\lambda_{\min}|} = \frac{n}{0} = \infty$.

Exercise: Laplacian matrix

- The 1-D Laplacian matrix is defined as $\mathbf{A} = \begin{bmatrix} -2 & 1 & 0 & & 0 \\ 1 & -2 & 1 & & 0 \\ 0 & 1 & -2 & & 0 \\ & & & \ddots & \\ 0 & 0 & 0 & & -2 \end{bmatrix}$.. For various sizes of $\mathbf{A}$ ($n = 2^1, 2^2, \dots, 2^{10}$), plot $\kappa_2(\mathbf{A})$ as a function of n .

■ *Solution:*

- The eigenvalues of $\mathbf{A}$ are $\lambda_k = -2 + 2 \cos \left(\frac{k\pi}{n+1} \right)$, $k = 1, \dots, n$. The singular values are eigenvalues of $\sigma_k = \sqrt{\text{eig} . (\mathbf{A}^T \mathbf{A})} = |\lambda_k|$.
Hence, $\sigma_k \in [0, 4]$ and $\sigma_{\max} = 4$.
- Using $\cos x \approx 1 - \frac{x^2}{2}$, $\lambda_k = -2 + 2 \left(1 - \frac{k\pi}{2(n+1)} \right) = 2 + 2 - 2 \frac{(k\pi)^2}{2(n+1)^2} = - \frac{(k\pi)^2}{(n+1)^2}$.
Hence, $\sigma_{\min} = \sigma_1 = \frac{(\pi)^2}{(n+1)^2}$.
- $\kappa_2(\mathbf{A}) = \frac{\sigma_{\max}}{\sigma_{\min}} \approx \frac{4}{\pi^2/(n+1)^2} \approx Cn^2$

4. Linear Equations (iterative methods)

Stationary iterative method: Jacobi method

■ Fixed point iteration

- To solve an equation $f(x) = 0$, we can rewrite the equation as $x = g(x)$, and apply the working formula $x_{k+1} = g(x_k)$. If the iteration converges, $x_k \rightarrow x^*$ as $k \rightarrow \infty$ (where x^* is the solution).
- For an n -dimensional problem, $\mathbf{x}_{k+1} = \mathbf{g}(\mathbf{x}_k)$

■ Stationary iterative methods

- To solve $\mathbf{Ax} = \mathbf{b}$, we split the matrix as $\mathbf{A} = \mathbf{M} - \mathbf{N}$ and write
$$(\mathbf{M} - \mathbf{N})\mathbf{x} = \mathbf{b} \implies \mathbf{Mx} = \mathbf{Nx} + \mathbf{b} \implies \mathbf{x} = \mathbf{M}^{-1}(\mathbf{Nx} + \mathbf{b}) \implies \mathbf{x}_{k+1} = \mathbf{M}^{-1}(\mathbf{Nx}_k + \mathbf{b})$$
- For an n -dimensional problem, $\mathbf{x}_{k+1} = \mathbf{g}(\mathbf{x}_k)$

■ Jacobi method

- $\mathbf{A}$ can be written as $\mathbf{L} + \mathbf{U} + \mathbf{D}$ where $\mathbf{L}$, $\mathbf{U}$, and $\mathbf{D}$, are the lower triangular, upper triangular, and diagonal parts of $\mathbf{A}$.
- In Jacobi method $\mathbf{M} = \mathbf{D}$, hence $\mathbf{A} = \mathbf{L} + \mathbf{U} + \mathbf{D} = \mathbf{L} + \mathbf{U} + \mathbf{M} = \mathbf{M} - \mathbf{N}$, and $\mathbf{N} = -(\mathbf{L} + \mathbf{U})$.
- The working formula is $\mathbf{x}_{k+1} = \mathbf{M}^{-1}(\mathbf{Nx}_k + \mathbf{b}) \implies \mathbf{x}_{k+1} = \mathbf{D}^{-1}(-(\mathbf{L} + \mathbf{U})\mathbf{x}_k + \mathbf{b})$.

■ Let's try the same example as before

```
A=np.array([[0.45368292, 0.19382865],[0.70364726, 0.52104011]])
b=np.array([0.48554638, 0.87421091])

x=np.linalg.solve(A,b)

print(x)
```

```
[0.83542019 0.54961178]
```

■ Simple implementation:

- Let's run the iteration a few times using matrix operations

```
A=np.array([[0.45368292, 0.19382865],[0.70364726, 0.52104011]])
b=np.array([0.48554638, 0.87421091])
```

```
D=np.array([[A[0,0],0],[0,A[1,1]]])
```

```
L=np.array([[0,0],[A[1,0],0]])
```

```
U=np.array([[0,A[0,1]],[0,0]])
```

```
#x0=[np.random.rand(2)]
```

```
x0=[1,1]
```

```
x1=np.zeros(2)
```

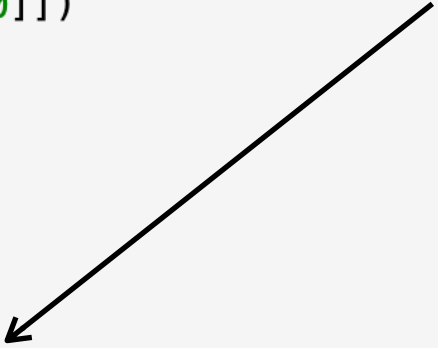
```
for k in range(100):
```

```
    print(x0)
```

```
    x1 = np.linalg.inv(D) @ (-(L+U)@x0 + b)
```

```
    x0 = x1.copy()
```

matrix-matrix or matrix-vector multiplication



```
[1, 1]
[0.64299915 0.32735225]
[0.93037696 0.80946997]
[0.7244      0.42137583]
[0.89020691 0.69954083]
[0.77136544 0.47562414]
[0.8670302   0.63611558]
[0.79846285 0.50692351]
[0.85365806 0.59952144]
[0.8140971   0.52498215]
[0.8459428   0.5784079]
[0.82311751 0.53540135]
[0.84149136 0.56622613]
_
```

■ *Convergence termination with a **while** loop:*

```
A=np.array([[0.45368292, 0.19382865],[0.70364726, 0.52104011]])
b=np.array([0.48554638, 0.87421091])

D=np.array([[A[0,0],0],[0,A[1,1]]])
L=np.array([[0,0],[A[1,0],0]])
U=np.array([[0,A[0,1]],[0,0]])

x0=[1,1]

NumIter=0
MaxIter=1000

xtol=1e-5
dx=np.inf

x1=np.zeros(2)
while NumIter < MaxIter and dx > xtol:
    NumIter += 1
    x1 = np.linalg.inv(D) @ (-(L+U)@x0 + b)
    dx = np.max(np.abs(x1-x0))
    x0 = x1.copy()

if dx <= xtol:
    print("Jacobi iteration converged")
    print(f"Iteration {NumIter:3d}: solution: {x1} with dx: {dx}")
```

Jacobi iteration converged

Iteration 42: solution: [0.83542177 0.54961613] with dx: 8.05639606238362e-06

■ *Elementwise operation to avoid zeros:*

- Let's write the definitions of elements in $\mathbf{x}_k = \mathbf{D}^{-1}(-(\mathbf{L} + \mathbf{U})\mathbf{x}_k + \mathbf{b})$.
- Let's start with $\mathbf{x}_k = \mathbf{M}^{-1}(\mathbf{N}\mathbf{x}_k + \mathbf{b})$. At the k -iteration, $\mathbf{x} = (\mathbf{T}\mathbf{N}\mathbf{x} + \mathbf{T}\mathbf{b})$, where $\mathbf{T} = \mathbf{M}^{-1}$.
- Element wise formula for matrix vector multiplication : $x_j = \sum T_{jm} N_{m,n} x_n + \sum T_{jm} b_m$
- Since $\mathbf{T} = \mathbf{M}^{-1}$ is a diagonal matrix, $x_j = \sum_{m,n} T_{jm} \delta_{jm} N_{m,n} x_n + \sum_{m,n} T_{jm} \delta_{jm} b_m = \sum_n T_{jj} N_{j,n} x_n + T_{jj} b_j$
- $x_j = \sum_n T_{jj} N_{j,n} x_n + T_{jj} b_j = T_{jj} (\sum_n N_{j,n} x_n + b_j) = (1/D_{jj}) (\sum_n - (L_{j,n} + U_{j,n}) x_n + b_j)$
- $x_j = (1/D_{jj}) (\sum_{n \neq j} - (L_{j,n} + U_{j,n}) x_n + b_j) = (1/A_{jj}) (b_j - \sum_{n \neq j} A_{j,n} x_n)$

```
x1 = np.linalg.inv(D) @ (-(L+U)@x0 + b)
```

- Elementwise replacement of the above line is as follows

```
x1[0] = b[0]
for n in range(2):
    if n != 0:
        x1[0] = x1[0] - A[0,n] * x0[n]
x1[0] = x1[0]/A[0,0]

x1[1] = b[1]
for n in range(2):
    if n != 1:
        x1[1] = x1[1] - A[1,n] * x0[n]
x1[1] = x1[1]/A[1,1]
```

```

A=np.array([[0.45368292, 0.19382865],[0.70364726, 0.52104011]])
b=np.array([0.48554638, 0.87421091])

D=np.array([[A[0,0],0],[0,A[1,1]])
L=np.array([[0,0],[A[1,0],0]])
U=np.array([[0,A[0,1]],[0,0]])

x0=[1,1]

NumIter=0
MaxIter=1000

xtol=1e-5
dx=np.inf

x1=np.zeros(2)
while NumIter < MaxIter and dx > xtol:
    NumIter += 1

    for j in range(2):
        x1[j] = b[j]
        for n in range(2):
            if n != j:
                x1[j] = x1[j] - A[j,n] * x0[n]
        x1[j] = x1[j]/A[j,j]

    dx = np.max(np.abs(x1-x0))
    x0 = x1.copy()

if dx <= xtol:
    print("Jacobi iteration converged")
    print(f"Iteration {NumIter:3d}: solution: {x1} with dx: {dx}")

```

```

Jacobi iteration converged
Iteration 42: solution: [0.83542177 0.54961613] with dx: 8.05639606238362e-06

```

■ **Exercise:** Write a function to implement an iterative linear solver using the Jacobi iteration.

Non-stationary iterative method: Steepest descent method

■ *Steepest descent method*

- To solve, $\mathbf{Ax} = \mathbf{b}$, using fixed-point iteration $\mathbf{x}_{k+1} = \mathbf{g}(\mathbf{x}_k)$.
- Works for symmetric positive definite $\mathbf{A}$ (the corresponding equation has a unique solution).
- The idea is to apply $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k$, where α_k is a scalar and $\mathbf{p}_k$ is an approximate search direction which is parallel to the error vector $\mathbf{e}_k = \mathbf{x} - \mathbf{x}_k$.
- The method minimizes the function $\phi(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T \mathbf{Ax} - \mathbf{b}^T \mathbf{x}$. We will start with a guess vector $\mathbf{x}_0$ and we need to find a direction, $\mathbf{p}_0$, of steepest descent (towards minimum) of $\phi(\mathbf{x})$. For this we have to find $-\nabla \phi(\mathbf{x}_0)$.

■ *Direction of steepest descent*

- Start with $\phi(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T \mathbf{Ax} - \mathbf{b}^T \mathbf{x}$ and apply a displacement
$$\phi(\mathbf{x} + \mathbf{h}) = \frac{1}{2} (\mathbf{x} + \mathbf{h})^T \mathbf{A} (\mathbf{x} + \mathbf{h}) - \mathbf{b}^T (\mathbf{x} + \mathbf{h}) = \frac{1}{2} (\mathbf{x}^T \mathbf{Ax} + \mathbf{x}^T \mathbf{Ah} + \mathbf{h}^T \mathbf{Ax} + \mathbf{h}^T \mathbf{Ah}) - \mathbf{b}^T \mathbf{x} - \mathbf{b}^T \mathbf{h}$$
- $$\phi(\mathbf{x} + \mathbf{h}) - \phi(\mathbf{x}) = \frac{1}{2} (\mathbf{x}^T \mathbf{Ax} + \mathbf{x}^T \mathbf{Ah} + \mathbf{h}^T \mathbf{Ax} + \mathbf{h}^T \mathbf{Ah}) - \mathbf{b}^T \mathbf{x} - \mathbf{b}^T \mathbf{h} - \left(\frac{1}{2} \mathbf{x}^T \mathbf{Ax} - \mathbf{b}^T \mathbf{x} \right)$$
$$\phi(\mathbf{x} + \mathbf{h}) - \phi(\mathbf{x}) \approx \frac{1}{2} (\mathbf{x}^T \mathbf{Ah} + \mathbf{h}^T \mathbf{Ax}) - \mathbf{b}^T \mathbf{h}; \quad \mathcal{O}(\mathbf{h}^2) = 0$$
- $\mathbf{x}^T \mathbf{Ah}$ is a scalar, so $\mathbf{x}^T \mathbf{Ah} = (\mathbf{x}^T \mathbf{Ah})^T = \mathbf{h}^T \mathbf{A}^T \mathbf{x} = \mathbf{h}^T \mathbf{Ax}$. Similarly $\mathbf{b}^T \mathbf{h} = \mathbf{h}^T \mathbf{b}$
- $$\phi(\mathbf{x} + \mathbf{h}) - \phi(\mathbf{x}) = \frac{1}{2} (\mathbf{h}^T \mathbf{Ax} + \mathbf{h}^T \mathbf{Ax}) - \mathbf{h}^T \mathbf{b} = \mathbf{h}^T (\mathbf{Ax} - \mathbf{b}) \approx \mathbf{h}^T \nabla \phi(\mathbf{x})$$
- $\nabla \phi(\mathbf{x}) = \mathbf{Ax} - \mathbf{b} \implies -\nabla \phi(\mathbf{x}) = \mathbf{b} - \mathbf{Ax}$.
- The iteration formula becomes $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{r}_k$ where $\mathbf{r}_k = \mathbf{b} - \mathbf{Ax}_k$.

- *Exercise:* Start with $\phi(\alpha) = \frac{1}{2}(\mathbf{x}_k + \alpha \mathbf{r}_k)^T \mathbf{A}(\mathbf{x}_k + \alpha \mathbf{r}_k) - \mathbf{b}^T(\mathbf{x}_k + \alpha \mathbf{r}_k)$ and solve $d\phi(\alpha)/d\alpha = 0$ to find

$$\alpha_{\min} = \alpha_k = \frac{\mathbf{r}_k^T \mathbf{r}_k}{\mathbf{r}_k^T \mathbf{A} \mathbf{r}_k}.$$

- *Steepest descent iteration*

- Start with a guess vector $\mathbf{x}_0$

- Find $\mathbf{x}_1 = \mathbf{x}_0 + \alpha_0 \mathbf{r}_0$ where $\mathbf{r}_0 = \mathbf{b} - \mathbf{A}\mathbf{x}_0$ and $\alpha_0 = \frac{\mathbf{r}_0^T \mathbf{r}_0}{\mathbf{r}_0^T \mathbf{A} \mathbf{r}_0}$, and so on

- *Simple implementation:*

```
A = np.array([[2,1],[1,2]], dtype=float)
b = np.array([1,2], dtype=float)

r=np.linalg.matrix_rank(A); U,S,Vt=np.linalg.svd(A)
print('Rank of A:',r,'singular values:',S)

x=np.linalg.solve(A,b)
print('Exact solution:', x)
```

```
Rank of A: 2 singular values: [3. 1.]
Exact solution: [0. 1.]
```

```
xold=np.array([1,1])
for i in range(10):
    print(xold)
    r=b - A@xold
    alpha= (r@r) / (r@A@r)
    xnew=xold+alpha*r
    xold=xnew
```

```
[1 1]
[0.28571429 0.64285714]
[0.10714286 1.         ]
[0.03061224 0.96173469]
[0.01147959 1.         ]
[0.00327988 0.99590015]
[0.00122996 1.         ]
[3.51416077e-04 9.99560730e-01]
[1.31781029e-04 1.00000000e+00]
[3.76517225e-05 9.99952935e-01]
```

Exercise: Laplacian matrix

- Solve $\mathbf{Ax} = \mathbf{b}$, where $\mathbf{A}$ is the Laplacian matrix defined as $\mathbf{A} = \begin{bmatrix} -2 & 1 & 0 & & 0 \\ 1 & -2 & 1 & & 0 \\ 0 & 1 & -2 & & 0 \\ & & & \ddots & \\ 0 & 0 & 0 & & -2 \end{bmatrix}$ and $\mathbf{b}$ is defined as $b_k = k$.

- Select the size of the matrix n , and check if $\mathbf{A}$ is positive definite.
- Check if $-\mathbf{A}$ is positive definite.
- Verify if $\mathbf{x}$ that solves $\mathbf{Ax} = \mathbf{b}$ also solves $-\mathbf{Ax} = -\mathbf{b}$.
- For $n = 10, 20, 30, 40, 50$, find $\mathbf{x}$ using Jacobi and steepest descent methods and verify if they give correct solution (compare with exact numerical solution)
- Is there any difference between the convergence rate (iteration count) of both methods.
- Suggest a strategy to code both methods such that one can solve the problem on your laptop for very large values of n such as $n = 10^3$.
- Submit your solution as a Jupyter notebook.

■ Geometrical meaning of Steepest Descent method

- Recall that the iteration formula in steepest descent is $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{r}_k$ where $\mathbf{r}_k = \mathbf{b} - \mathbf{A}\mathbf{x}_k$ is the search direction.
- Successive search directions are orthogonal to each other, i.e., $\mathbf{r}_k \perp \mathbf{r}_{k-1}$.
- Since $\alpha_k \mathbf{r}_k = \mathbf{x}_{k+1} - \mathbf{x}_k$, we can also think that the successive steps are in orthogonal directions.

```
A = np.array([[2,1],[1,2]], dtype=float)
b = np.array([1,2], dtype=float)

r=np.linalg.matrix_rank(A)
U,S,Vt=np.linalg.svd(A)
print('Rank of A:',r,'singular values:',S)
print('-' * 50)

x=np.linalg.solve(A,b)
print('Exact solution:', x)
print('-' * 50)

xold=np.array([1,1])
rold=np.zeros(2)

for i in range(10):
    r=b - A@xold
    alpha= (r@r) / (r@A@r)
    xnew=xold+alpha*r
    print(xold,r,r@rold)
    xold=xnew
    rold=r
```

Rank of A: 2 singular values: [3. 1.]

Exact solution: [0. 1.]

[1 1]	[-2. -1.]	0.0		
[0.28571429 0.64285714]	[-0.21428571 0.42857143]		-4.440892098500626e-16	
[0.10714286 1.]	[-0.21428571 -0.10714286]		-4.503200533556121e-17	
[0.03061224 0.96173469]	[-0.02295918 0.04591837]		4.776558562350717e-17	
[0.01147959 1.]	[-0.02295918 -0.01147959]		5.554221874886e-21	

■ *Exercise:* Solve $\mathbf{Ax} = \mathbf{b}$, where $\mathbf{b} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$ and

◦ Case 1: $\mathbf{A}$ is a non-singular matrix, $\mathbf{A} = \begin{bmatrix} 4 & 2 \\ 2 & 2 \end{bmatrix}$

◦ Case 2: $\mathbf{A}$ is an approximately singular matrix, $\mathbf{A} = \begin{bmatrix} 4 & 2 \\ 2 & 1.5 \end{bmatrix}$

◦ Case 3: $\mathbf{A}$ is an approximately singular matrix, $\mathbf{A} = \begin{bmatrix} 4 & 2 \\ 2 & 1.1 \end{bmatrix}$

◦ Case 4: $\mathbf{A}$ is an approximately singular matrix, $\mathbf{A} = \begin{bmatrix} 4 & 2 \\ 2 & 1.01 \end{bmatrix}$

■ Find the singular values of the matrix $\mathbf{A}$ in each case, and find the condition number $\kappa_2(\mathbf{A})$.

■ For all cases, find the iterations required to converge the maximum element in the solution vector to 10^{-5}

■ What can one say about the successive directions in well-conditioned and ill-conditioned problems?

Lanczos iteration

- For illustrating Lanczos, let's solve $\mathbf{Ax} = \mathbf{b}$, where $\mathbf{A} = \begin{bmatrix} 4 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 2 \end{bmatrix}$ and $\mathbf{b} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$. First, let's solve this system with steepest descent and find that the solution converges to **0.22222, 0.11111, 1.44444**

```
A = np.array([[4.0, 1.0, 0.0],[1.0, 3.0, 1.0],[0.0, 1.0, 2.0]])
b = np.array([1.0, 2.0, 3.0])

xold=np.array([1,1,1])

for i in range(20):
    r=b - A@xold
    alpha= (r@r) / (r@A@r)
    xnew=xold+alpha*r
    print(xold)
    xold=xnew
```

```
[1 1 1]
[0.13043478 0.34782609 1.         ]
[0.21404682 0.23634337 1.4180602 ]
[0.19026075 0.14865014 1.39943255]
[0.22489179 0.13498142 1.4195595 ]
[0.21448551 0.12010312 1.42736064]
[0.22266787 0.11929978 1.43674335]
[0.21966149 0.11408056 1.43891826]
[0.22237213 0.11377444 1.44193056]
[0.22138833 0.11207817 1.44264345]
[0.22227101 0.11197893 1.44362542]
[0.22195052 0.1114262  1.44385765]
[0.22223812 0.11139386 1.44417759]
[0.2221337  0.11121377 1.44425325]
[0.2222274  0.11120324 1.4443575 ]
[0.22219338 0.11114456 1.44438215]
[0.22222391 0.11114113 1.44441612]
[0.22221282 0.11112201 1.44442415]
[0.22222277 0.11112089 1.44443521]
[0.22221916 0.11111466 1.44443783]
```

- In steepest descent, the iteration formula is $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{r}_k$, where $\mathbf{r}_k = \mathbf{b} - \mathbf{A}\mathbf{x}_k$.
- Only successive search directions are orthogonal to each other, i.e., $\mathbf{r}_k \perp \mathbf{r}_{k-1}$, and as we have seen in the previous examples, orthogonality is not guaranteed between other iterations.
- In Lanczos and conjugate-gradient methods, the residual is orthogonal to all previous residuals with the iteration formula $\mathbf{x}_k = \mathbf{x}_0 + \mathbf{Q}_k \mathbf{y}_k$, where $\mathbf{Q}_k = [\mathbf{q}_1, \mathbf{q}_2, \dots, \mathbf{q}_k]$ is an $n \times k$ matrix with orthonormal columns and $\mathbf{y}_k$ is a k -vector of coefficients.
- *Lanczos iteration*
 - Initial step, $k = 0$. Start with a guess vector $\mathbf{x}_0$, the q-vector is the zero-vector $\mathbf{q}_0 = [0, 0, 0]^T$, calculate the residual $\mathbf{r}_0 = \mathbf{b} - \mathbf{A}\mathbf{x}_0$, and $\beta_0 = \|\mathbf{r}_0\|_2 = \sqrt{\mathbf{r}_0^T \cdot \mathbf{r}_0}$
 - Iteration 1:
 - (1) $\mathbf{q}_1 = \mathbf{r}_0 / \beta_0$
 - (2) $\alpha_1 = \mathbf{q}_1^T \cdot \mathbf{A} \cdot \mathbf{q}_1$
 - (3) $\mathbf{T}_1 = [\alpha_1]$ is a 1×1 matrix.
 - (4) Find $\mathbf{y}_1$ by solving $\mathbf{T}_1 \mathbf{y}_1 = [\beta_0]^T$
 - (5) $\mathbf{Q}_1 = [\mathbf{q}_1]$
 - (6) $\mathbf{x}_1 = \mathbf{x}_0 + \mathbf{Q}_1 \mathbf{y}_1$
 - (7) $\mathbf{v}_1 = \mathbf{A}\mathbf{q}_1 - \alpha_1 \mathbf{q}_1 - \beta_0 \mathbf{q}_0$, which is not a residual but a Lanczos recurrence vector
 - (8) $\beta_1 = \|\mathbf{v}_1\|_2$

○ Iteration 2:

(1) $\mathbf{q}_2 = \mathbf{v}_1 / \beta_1$

(2) $\alpha_2 = \mathbf{q}_2^T \cdot \mathbf{A} \cdot \mathbf{q}_2$

(3) $\mathbf{T}_2 = \begin{bmatrix} \alpha_1 & \beta_1 \\ \beta_1 & \alpha_2 \end{bmatrix}$ is a 2×2 matrix. Include second row and second column.

(4) Find $\mathbf{y}_2$ by solving $\mathbf{T}_2 \mathbf{y}_2 = [\beta_0, 0]^T$

(5) $\mathbf{Q}_2 = [\mathbf{q}_1, \mathbf{q}_2]$

(6) $\mathbf{x}_2 = \mathbf{x}_0 + \mathbf{Q}_2 \mathbf{y}_2$

(7) $\mathbf{v}_2 = \mathbf{A} \mathbf{q}_2 - \alpha_2 \mathbf{q}_2 - \beta_1 \mathbf{q}_1$

(8) $\beta_2 = \|\mathbf{v}_2\|_2$

○ Iteration 3:

(1) $\mathbf{q}_3 = \mathbf{v}_2 / \beta_2$

(2) $\alpha_3 = \mathbf{q}_3^T \cdot \mathbf{A} \cdot \mathbf{q}_3$

(3) $\mathbf{T}_3 = \begin{bmatrix} \alpha_1 & \beta_1 & 0 \\ \beta_1 & \alpha_2 & \beta_2 \\ 0 & \beta_2 & \alpha_3 \end{bmatrix}$ is a 3×3 matrix. Include third row and third column.

(4) Find $\mathbf{y}_3$ by solving $\mathbf{T}_3 \mathbf{y}_3 = [\beta_0, 0, 0]^T$

(5) $\mathbf{Q}_3 = [\mathbf{q}_1, \mathbf{q}_2, \mathbf{q}_3]$

(6) $\mathbf{x}_3 = \mathbf{x}_0 + \mathbf{Q}_3 \mathbf{y}_3$

The next two steps are needed if we want to proceed to Iteration 4

(7) $\mathbf{v}_3 = \mathbf{A} \mathbf{q}_3 - \alpha_3 \mathbf{q}_3 - \beta_2 \mathbf{q}_2$

(8) $\beta_3 = \|\mathbf{v}_3\|_2$

```
import numpy as np

A = np.array([
    [4.0, 1.0, 0.0],
    [1.0, 3.0, 1.0],
    [0.0, 1.0, 2.0]
])

b = np.array([1.0, 2.0, 3.0])
x0 = np.array([0.0, 0.0, 0.0])
```

```
# k = 0
q0 = np.zeros(3)
v0 = b - A @ x0
beta0 = np.linalg.norm(v0)
```

```
# k = 1
q1 = v0 / beta0
alpha1 = q1 @ (A @ q1)
T1 = np.array([[alpha1]])
y1 = np.linalg.solve(T1, np.array([beta0]))
Q1 = np.column_stack([q1])
x1 = x0 + Q1 @ y1
v1 = A @ q1 - alpha1 * q1 - beta0 * q0
beta1 = np.linalg.norm(v1)

print("Iteration 1")
print("x1 =", x1)
```

```
Iteration 1
x1 = [0.28 0.56 0.84]
```

```

# k = 2
q2 = v1 / beta1
alpha2 = q2 @ (A @ q2)
T2 = np.array([
    [alpha1, beta1],
    [beta1, alpha2]
])
y2 = np.linalg.solve(T2, np.array([beta0, 0.0]))
Q2 = np.column_stack([q1, q2])
x2 = x0 + Q2 @ y2
v2 = A @ q2 - alpha2 * q2 - beta1 * q1
beta2 = np.linalg.norm(v2)

print("Iteration 2")
print("x2 =", x2)

```

Iteration 2
x2 = [0.04923077 0.32923077 1.30153846]

```

# k = 3
q3 = v2 / beta2
alpha3 = q3 @ (A @ q3)
T3 = np.array([
    [alpha1, beta1, 0.0],
    [beta1, alpha2, beta2],
    [0.0, beta2, alpha3]
])
y3 = np.linalg.solve(T3, np.array([beta0, 0.0, 0.0]))
Q3 = np.column_stack([q1, q2, q3])
x3 = x0 + Q3 @ y3

print("Iteration 3")
print("x3 =", x3)

```

Iteration 3
x3 = [0.22222222 0.11111111 1.44444444]

converged in 3 iterations

■ Krylov subspace:

- The Lanczos recurrence vectors are of the form, $\mathbf{v}_1 \approx \mathbf{A}\mathbf{q}_1 \approx \mathbf{A}\mathbf{r}_0$, $\mathbf{v}_2 \approx \mathbf{A}\mathbf{q}_2 \approx \mathbf{A}^2\mathbf{q}_1 \approx \mathbf{A}^2\mathbf{r}_0$, and so on.
- In a simpler language, $\mathbf{q}_1 \propto \mathbf{r}_0$, $\mathbf{q}_2 \in \text{span}\{\mathbf{r}_0, \mathbf{A}\mathbf{r}_0\}$, $\mathbf{q}_3 \in \text{span}\{\mathbf{r}_0, \mathbf{A}\mathbf{r}_0, \mathbf{A}^2\mathbf{r}_0\}$, and so on.
- k -dimensional Krylov subspace is spanned by $\{\mathbf{r}_0, \mathbf{A}\mathbf{r}_0, \mathbf{A}^2\mathbf{r}_0, \dots, \mathbf{A}^{k-1}\mathbf{r}_0\}$
- The Lanczos iteration constructs an orthonormal basis $\mathbf{Q}_k = [\mathbf{q}_1, \mathbf{q}_2, \mathbf{q}_3]$ for this Krylov subspace.
- The matrix $\mathbf{T}_k$ is the representation of $\mathbf{A}$ in the Krylov subspace: $\mathbf{T}_k = \mathbf{Q}_k^T \mathbf{A} \mathbf{Q}_k$
- The matrix $\mathbf{T}_k$ is tridiagonal due to the recurrence relation $\mathbf{v}_k = \mathbf{A}\mathbf{q}_k - \alpha_k \mathbf{q}_k - \beta_{k-1} \mathbf{q}_{k-1}$.
- As $\mathbf{q}_{k+1} = \mathbf{v}_k / \beta_k$, we have $\beta_k \mathbf{q}_{k+1} = \mathbf{A}\mathbf{q}_k - \alpha_k \mathbf{q}_k - \beta_{k-1} \mathbf{q}_{k-1}$ or $\mathbf{A}\mathbf{q}_k = \beta_k \mathbf{q}_{k+1} + \alpha_k \mathbf{q}_k + \beta_{k-1} \mathbf{q}_{k-1}$. The components of $\mathbf{A}\mathbf{q}_k$ lie along $\mathbf{q}_{k-1}$, $\mathbf{q}_k$, and $\mathbf{q}_{k+1}$ but orthogonal to other $\mathbf{q}$ -vectors.

■ Simple implementation of Lanczos iteration

```
x0 = np.array([0.0, 0.0, 0.0])
v = b - A @ x0
beta0 = np.linalg.norm(v)

q_prev = np.zeros(3)
beta_prev = beta0

Q_list = []

while beta_prev > 1e-14:
    q = v / beta_prev
    Q_list.append(q)
    Q = np.column_stack(Q_list)

    alpha = q @ (A @ q)

    T = Q.T @ A @ Q

    rhs = np.zeros(len(Q_list))
    rhs[0] = beta0
    y = np.linalg.solve(T, rhs)

    x = x0 + Q @ y
    print(f"Iteration {len(Q_list)}: x = {x}")

    v_new = A @ q - alpha * q - beta_prev * q_prev
    beta_new = np.linalg.norm(v_new)

    q_prev = q
    v = v_new
    beta_prev = beta_new
```

5. Eigenvalue problems

Eigenvalue and eigenvector

■ Eigenvalue equation

- $\mathbf{A}\mathbf{v} = a\mathbf{v}$ is an eigenvalue equation where $\mathbf{A} \in \mathbb{C}^{n \times n}$ and $\mathbf{v} \in \mathbb{C}^{n \times 1}$ (and $\mathbf{v} \neq 0$) is the right-eigenvector of $\mathbf{A}$ associated with the eigenvalue a .
- Left multiplying the equation by $\mathbf{v}^H \in \mathbb{C}^{1 \times n}$ and $(\mathbf{v}^H)_i = \overline{(\mathbf{v})_i}$, we get $\mathbf{v}^H \mathbf{A} \mathbf{v} = \mathbf{v}^H a \mathbf{v} = a \mathbf{v}^H \mathbf{v}$. This equation can be written in a more familiar form as $a = \frac{\mathbf{v}^H \mathbf{A} \mathbf{v}}{\mathbf{v}^H \mathbf{v}}$.
- The symbol $()^H$ denotes Hermitian transpose (or transpose conjugate), $()^H = \overline{()}^T$. For a scalar, $a^H = \bar{a}$.
- Note that for an arbitrary vector, $\mathbf{x}$, which is not an eigenvector of $\mathbf{A}$, $\frac{\mathbf{x}^H \mathbf{A} \mathbf{x}}{\mathbf{x}^H \mathbf{x}}$ (which is called the Rayleigh quotient) will not correspond to an eigenvalue.
- For $\mathbf{A} \in \mathbb{C}^{n \times n}$, $\mathbf{w} \in \mathbb{C}^{1 \times n}$ (and $\mathbf{w} \neq 0$) is the left-eigenvector (with the eigenvalue b), if $\mathbf{w} \mathbf{A} = b \mathbf{w}$.

- **Example:** $\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 0 & 2 \end{bmatrix}$ and $\mathbf{v}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$, $\mathbf{A} \mathbf{v}_1 = 1 \mathbf{v}_1$, where the eigenvalue $a_1 = 1$. Similarly, for $\mathbf{v}_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$, $\mathbf{A} \mathbf{v}_2 = 2 \mathbf{v}_2$, where the eigenvalue $a_2 = 2$. For the transpose, $\mathbf{A}^T = \begin{bmatrix} 1 & 0 \\ 1 & 2 \end{bmatrix}$, $\mathbf{A}^T \mathbf{v}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$, is not an eigenvalue equation. However, we have $\mathbf{v}_1^T \mathbf{A}^T = [1 \ 0] = 1 \mathbf{v}_1^T$. So, $\mathbf{v}_1^T$ is the left eigenvector of $\mathbf{A}^T$ with the eigenvalue $a_1 = 1$. Similarly, $\mathbf{v}_2^T$ is the left eigenvector of $\mathbf{A}^T$ with the eigenvalue $a_2 = 2$.

- **Example:** Consider the complex matrix $\mathbf{A} = \begin{bmatrix} i & 1 \\ 0 & 2 \end{bmatrix}$. Verify that the right eigenvectors are $\mathbf{v}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$, $\mathbf{v}_2 = \begin{bmatrix} 1 \\ 2-i \end{bmatrix}$ associated with the eigenvalues $a_1 = i$ and $a_2 = 2$. Consider the Hermitian transpose $\mathbf{A}^H = \begin{bmatrix} -i & 1 \\ 0 & 2 \end{bmatrix}$ and verify that the left eigenvectors are $\mathbf{v}_1^H$ and $\mathbf{v}_2^H$ associated with the eigenvalues $\bar{a}_1 = -i$ and $\bar{a}_2 = 2$. From this example, we see $\mathbf{A} \mathbf{v} = a \mathbf{v} \Leftrightarrow (\mathbf{A} \mathbf{v})^H = (a \mathbf{v})^H \Leftrightarrow \mathbf{v}^H \mathbf{A}^H = \bar{a} \mathbf{v}^H$.

Similarity transformation

■ *Invariant subspace and similar matrices*

- Suppose W is a k -dimensional subspace of $\mathbb{C}^n$, and $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_k]$ is an $n \times k$ matrix with columns forming the basis of the subspace W . If $\mathbf{A}\mathbf{X} \in W$, then W is an invariant subspace of $\mathbf{A}$.
- If $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_n]$ is an $n \times n$ matrix. We know that in $\mathbf{A}\mathbf{X}$, each column is a linear combination of columns of $\mathbf{X}$, i.e., $\mathbf{A}\mathbf{X} = \mathbf{X}\mathbf{B}$. Then, $\mathbf{A}$ and $\mathbf{B}$ are similar matrices, related by the similarity transformation $\mathbf{X}^{-1}\mathbf{A}\mathbf{X} = \mathbf{B}$.
- If $\mathbf{v}$ is an eigenvector of $\mathbf{A}$ associated with the eigenvalue λ , i.e., $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$, then $\mathbf{B} = \mathbf{X}^{-1}\mathbf{A}\mathbf{X} \implies \mathbf{B}\mathbf{X}^{-1}\mathbf{v} = \mathbf{X}^{-1}\mathbf{A}\mathbf{X}\mathbf{X}^{-1}\mathbf{v} = \mathbf{X}^{-1}\mathbf{A}\mathbf{v} = \mathbf{X}^{-1}\lambda\mathbf{v} = \lambda\mathbf{X}^{-1}\mathbf{v}$. This means, λ is also an eigenvalue of $\mathbf{B}$ with the eigenvector $\mathbf{X}^{-1}\mathbf{v}$.
- If $\mathbf{X}^{-1}\mathbf{A}\mathbf{X} = \mathbf{B}$, then all eigenvalues of $\mathbf{A}$ are also eigenvalues of $\mathbf{B}$ and vice versa.
- Suppose $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_k]$ is an $n \times k$ matrix ($k < n$), then the eigenvalues of $\mathbf{B}$ is a subset of eigenvalues of $\mathbf{A}$. The equation $\mathbf{X}^{-1}\mathbf{A}\mathbf{X} = \mathbf{B}$ is not applicable as $\mathbf{X}$ is not invertible. Still, one can apply the equation $\mathbf{A}\mathbf{X} = \mathbf{X}\mathbf{B}$ and derive the following
$$\mathbf{A}\mathbf{X} = \mathbf{X}\mathbf{B} \implies \mathbf{X}^T\mathbf{A}\mathbf{X} = \mathbf{X}^T\mathbf{X}\mathbf{B} \implies (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{A}\mathbf{X} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{X}\mathbf{B} \implies \mathbf{B} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{A}\mathbf{X},$$
where we use the property that $\mathbf{X}^T\mathbf{X}$ is invertible even if $\mathbf{X}$ (which is rectangular) is not invertible.

■ *Exercise:* For $\mathbf{A} \in \mathbb{C}^{n \times n}$, $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_k] \in \mathbb{C}^{n \times k}$, and $\mathbf{A}\mathbf{X} = \mathbf{X}\mathbf{B}$, show that each column of $\mathbf{A}\mathbf{X}$ is a linear combination of columns of $\mathbf{X}$.

■ *Exercise:* Find the eigenvalues of $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix}$ using the Numpy function `a, v = np.linalg.eig(A)`, where `a` is an 1D array of eigenvalues and `v` is matrix whose columns are the eigenvectors. For $\mathbf{X} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$, find $\mathbf{B} = \mathbf{X}^{-1}\mathbf{A}\mathbf{X}$ (using `np.linalg.inv(X)` for matrix inverse) and its eigenvalues.

- *Exercise:* Find the eigenvalues of $\mathbf{A} = \begin{bmatrix} 2 & 1 & 1 \\ 0 & 3 & 4 \\ 0 & 0 & 1 \end{bmatrix}$ and for $\mathbf{X} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \\ 0 & 0 \end{bmatrix}$ find $\mathbf{B} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{A} \mathbf{X}$ and show that the eigenvalues of $\mathbf{B}$ are a subset of eigenvalues of $\mathbf{A}$. Hint: find the transpose of $\mathbf{X}$ as $\mathbf{x}.\mathbf{T}$

```
A=np.array([[2,1,1],[0,3,4],[0,0,1]])  
a,V=np.linalg.eig(A)  
  
print(a)  
print(V)
```

```
[2. 3. 1.]  
[[ 1.          0.70710678  0.40824829]  
 [ 0.          0.70710678 -0.81649658]  
 [ 0.          0.          0.40824829]]
```

```
X=np.array([[1,1],[1,-1],[0,0]])  
B=np.linalg.inv(X.T@X)@X.T@A@X  
  
print(B)
```

```
[[ 3. -1.]  
 [ 0.  2.]]
```

```
b,W=np.linalg.eig(B)  
  
print(b)
```

```
[3. 2.]
```

Power Method

■ Dominant eigenpair

- Given an $n \times n$ matrix $\mathbf{A}$, with $|\lambda_1| > |\lambda_2| \geq |\lambda_3| \geq \dots$, the power method computes the eigenvector corresponding to λ_1 by repeatedly multiplying $\mathbf{A}$ with a random unit vector $\mathbf{q}_0$
- Choose a unit vector $\mathbf{q}_0$ such that $\|\mathbf{q}_0\|_2 = 1$ and for various iterations $k = 1, 2, \dots$
 - (1) $\mathbf{z}_k = \mathbf{A}\mathbf{q}_{k-1}$
 - (2) $\mathbf{q}_k = \mathbf{z}_k / \|\mathbf{z}_k\|_2$
- The method fails to converge when $|\lambda_1| = |\lambda_2|$. In such case, one can find the corresponding eigenvector by fixing the phase of the first component.

■ Proof

- Let the initial guess vector be a non-zero vector $\mathbf{x}^{(0)}$. The power method generates $\mathbf{x}^{(k)} = \mathbf{A}\mathbf{x}^{(k-1)} = \mathbf{A}^k\mathbf{x}^{(0)}$, where $k = 1, 2, \dots$
- The eigenvectors of $\mathbf{A}$ form the basis of an n -dimensional space, $\mathbb{R}^n$. Hence, $\mathbf{x}^{(0)}$ can be expressed as a linear combination of eigenvectors

$$\mathbf{x}^{(0)} = \sum_{i=1}^n c_i \mathbf{x}_i \implies \mathbf{A}^k \mathbf{x}^{(0)} = \sum_{i=1}^n \mathbf{A}^k c_i \mathbf{x}_i = \sum_{i=1}^n c_i \mathbf{A}^k \mathbf{x}_i = \sum_{i=1}^n c_i \lambda_i^k \mathbf{x}_i = \lambda_1^k \sum_{i=1}^n c_i \left(\frac{\lambda_i}{\lambda_1} \right)^k \mathbf{x}_i = \lambda_1^k \left[c_1 \mathbf{x}_1 + \sum_{i=2}^n c_i \left(\frac{\lambda_i}{\lambda_1} \right)^k \mathbf{x}_i \right]$$

- As $k \rightarrow \infty$, $|\lambda_1|^k \rightarrow \infty$, and $\left(\frac{\lambda_i}{\lambda_1} \right)^k \rightarrow 0$. Hence, $\lim_{k \rightarrow \infty} \mathbf{A}^k \mathbf{x}^{(0)} = \lambda_1^k [c_1 \mathbf{x}_1]$. Upon normalization, we will get the eigenvector, $\mathbf{x}_1$.

■ Simple implementation of Power iteration

```
import numpy as np

A=np.array([[2,1,1],[0,3,4],[0,0,1]])

n=np.shape(A[0])[0]

#x0=np.random.rand(n)
x0=np.ones(n)

for k in range(0,20):
    xk = A @ x0
    xk = xk / np.linalg.norm(xk)
    x0 = xk.copy()
    if np.mod(k,5)==0:
        print('Iter:', k, xk)
```

```
Iter: 0 [0.49236596 0.86164044 0.12309149]
Iter: 5 [6.74453583e-01 7.38317174e-01 3.37902597e-04]
Iter: 10 [7.02985712e-01 7.11203971e-01 1.33826078e-06]
Iter: 15 [7.06567911e-01 7.07645241e-01 5.47966826e-09]
```

■ Eigenvalue is the Rayleigh quotient

- Once the eigenvector, $\mathbf{x}_1$, is determined, the eigenvalue can be determined as the Rayleigh quotient

$$r(\mathbf{x}_1, \mathbf{A}) = \frac{\mathbf{x}_1^T \mathbf{A} \mathbf{x}_1}{\mathbf{x}_1^T \mathbf{x}_1}$$

```
a1=(xk.T@A@xk) / (xk.T@xk)
print(a1)
```

```
3.000150364418795
```

■ Simple implementation of Power iteration as a function

```
def power(A, x0=None, MaxIter=100, xtol=1e-8):
```

```
    n = A.shape[0]
```

```
    if x0 is None:
```

```
        x0 = np.ones(n, dtype=float)
```

```
    else:
```

```
        x0 = np.array(x0, dtype=float)
```

```
    x0 = x0 / np.linalg.norm(x0)
```

```
    NumIter = 0
```

```
    dx = np.inf
```

```
    while NumIter < MaxIter and dx > xtol:
```

```
        NumIter += 1
```

```
        x1 = A @ x0
```

```
        x1 = x1 / np.linalg.norm(x1)
```

```
        dx = np.linalg.norm(x0 - x1)
```

```
        x0 = x1.copy()
```

```
    if dx <= xtol:
```

```
        print("Power method converged")
```

```
    else:
```

```
        print("Power method did not converge")
```

```
    return NumIter, x0
```

```
A=np.array([[2,1,1],[0,3,4],[0,0,1]], dtype=float)
```

```
NumIter,x0=power(A)
```

```
eigenvalue = (x0.T @ A @ x0) / (x0.T @ x0)
```

```
print(f"Iteration: {NumIter:3d}, Eigenvector: {x0}, Eigenvalue: {eigenvalue:12.8f}")
```

Power method converged

Iteration: 43, Eigenvector: [7.07106772e-01 7.07106791e-01 7.18041921e-22], Eigenvalue: 3.00000001

■ Inverse power method for eigenvalue of smallest magnitude

- If one repeats the power method using $\mathbf{A}^{-1}$ instead of $\mathbf{A}$, the procedure converges to the eigenvector corresponding to the eigenvalue of smallest magnitude. Instead of $\mathbf{z}_k = \mathbf{A}^{-1}\mathbf{q}_{k-1}$, we will do $\mathbf{z}_k = \text{linalg.solve}(\mathbf{A}, \mathbf{q}_{k-1})$

```
def inversepower(A, x0=None, MaxIter=100, xtol=1e-8):

    n = A.shape[0]
    if x0 is None:
        x0 = np.ones(n, dtype=float)
    else:
        x0 = np.array(x0, dtype=float)
    x0 = x0 / np.linalg.norm(x0)

    NumIter = 0
    dx = np.inf

    while NumIter < MaxIter and dx > xtol:
        NumIter += 1
        x1 = np.linalg.solve(A, x0)
        x1 = x1 / np.linalg.norm(x1)
        dx = np.linalg.norm(x0 - x1)
        x0 = x1.copy()

    if dx <= xtol:
        print("Inverse power method converged")
    else:
        print("Inverse power method did not converge")

    return NumIter, x0

A=np.array([[2,1,1],[0,3,4],[0,0,1]], dtype=float)

NumIter,x0=inversepower(A)
eigenvalue = (x0.T @ A @ x0) / (x0.T @ x0)
print(f"Iteration: {NumIter:3d}, Eigenvector: {x0}, Eigenvalue: {eigenvalue:12.8f}")
```

Inverse power method converged

Iteration: 27, Eigenvector: [0.40824828 -0.81649658 0.40824829], Eigenvalue: 1.00000000

■ *Shifted inverse power method for eigenvalue close to a target*

- Suppose we are interested in the eigenvalue close to a real number μ , one can modify the inverse-power method using $\mathbf{z}_k = \text{linalg.solve}(\mathbf{A} - \mu\mathbf{I}, \mathbf{q}_{k-1})$.

```
def shiftedinversepower(A, mu, x0=None, MaxIter=100, xtol=1e-8):

    n = A.shape[0]
    if x0 is None:
        x0 = np.ones(n, dtype=float)
    else:
        x0 = np.array(x0, dtype=float)
    x0 = x0 / np.linalg.norm(x0)

    NumIter = 0
    dx = np.inf

    while NumIter < MaxIter and dx > xtol:
        NumIter += 1
        x1 = np.linalg.solve(A-mu*np.eye(n), x0) ←
        x1 = x1 / np.linalg.norm(x1)
        dx = np.linalg.norm(x0 - x1)
        x0 = x1.copy()

    if dx <= xtol:
        print("Shifted inverse power method converged")
    else:
        print("Shifted inverse power method did not converge")

    return NumIter, x0

A=np.array([[2,1,1],[0,3,4],[0,0,1]], dtype=float)

NumIter,x0=shiftedinversepower(A,mu=1.8) ←
eigenvalue = (x0.T @ A @ x0) / (x0.T @ x0)
print(f"Iteration: {NumIter:3d}, Eigenvector: {x0}, Eigenvalue: {eigenvalue:12.8f}")
```

Shifted inverse power method converged

Iteration: 15, Eigenvector: [-1.00000000e+00 6.23008539e-10 -3.10440858e-10], Eigenvalue: 2.00000000

- *Prevent oscillatory non-convergence by fixing the phase of the eigenvector*

```
def power(A, x0=None, MaxIter=100, xtol=1e-8):  
    n = A.shape[0]  
    if x0 is None:  
        x0 = np.ones(n, dtype=float)  
    else:  
        x0 = np.array(x0, dtype=float)  
    x0 = x0 / np.linalg.norm(x0)  
  
    NumIter = 0  
    dx = np.inf  
  
    while NumIter < MaxIter and dx > xtol:  
        NumIter += 1  
        x1 = A @ x0  
        x1 = x1 / np.linalg.norm(x1)  
        if x1[0] < 0:  
            x1 = -x1  
        dx = np.linalg.norm(x0 - x1)  
        x0 = x1.copy()  
  
    if dx <= xtol:  
        print("Power method converged")  
    else:  
        print("Power method did not converge")  
  
    return NumIter, x0
```

First element of the eigenvector is always positive

- **Exercise:** Apply the power, inverse power, and shifted inverse power methods to $A = \begin{bmatrix} -7 & 13 & -16 \\ 13 & -10 & 13 \\ -16 & 13 & -7 \end{bmatrix}$. Use $\mu = 10$ for the shifted inverse power method. Does the procedure converge? If not, diagnose the problem.

```
A=np.array([[-7,13,-16],[13,-10,13],[-16,13,-7]], dtype=float)
print(A)
```

```
E,V=np.linalg.eig(A)
print('Exact eigenvalues:',E)
```

```
NumIter,x0=power(A)
eigenvalue = (x0.T @ A @ x0) / (x0.T @ x0)
print(f"Iteration: {NumIter:3d}, Eigenvector: {x0}, Eigenvalue: {eigenvalue:12.8f}")
```

```
NumIter,x0=inversepower(A)
eigenvalue = (x0.T @ A @ x0) / (x0.T @ x0)
print(f"Iteration: {NumIter:3d}, Eigenvector: {x0}, Eigenvalue: {eigenvalue:12.8f}")
```

```
NumIter,x0=shiftedinversepower(A,mu=10)
eigenvalue = (x0.T @ A @ x0) / (x0.T @ x0)
print(f"Iteration: {NumIter:3d}, Eigenvector: {x0}, Eigenvalue: {eigenvalue:12.8f}")
```

```
[[ -7.  13. -16.]
 [ 13. -10.  13.]
 [-16.  13. -7.]]
```

```
Exact eigenvalues: [-36.   9.   3.]
```

```
Power method converged
```

```
Iteration:   9, Eigenvector: [ 0.57735027 -0.57735027  0.57735027], Eigenvalue: -36.00000000
```

```
Inverse power method converged
```

```
Iteration:   9, Eigenvector: [0.40824829 0.81649658 0.40824829], Eigenvalue:   3.00000000
```

```
Shifted inverse power method converged
```

```
Iteration:  31, Eigenvector: [ 7.07106781e-01 -2.75454669e-10 -7.07106781e-01], Eigenvalue:   9.00000000
```

Hotelling deflation

■ Spectral representation of a matrix

- Let us consider a real, symmetric matrix as in the previous problem.
- If a matrix can be diagonalized, we can write $\mathbf{X}^T \mathbf{A} \mathbf{X} = \mathbf{\Lambda}$, where $\mathbf{\Lambda}$ is a diagonal matrix.
- The orthogonal matrix $\mathbf{X}$ is a row of eigenvector columns $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n]$. The eigenvectors are normalized, i.e, $\mathbf{X}^T \mathbf{X} = \mathbf{I} \implies \mathbf{x}_k^T \mathbf{x}_k = 1$.
- We can rewrite the original equation as $\mathbf{X}^T \mathbf{A} \mathbf{X} = \mathbf{\Lambda} \implies \mathbf{A} = \mathbf{X} \mathbf{\Lambda} \mathbf{X}^T$. Since $\mathbf{\Lambda}$ is a diagonal matrix, we can write it as

$$\mathbf{A} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n] \mathbf{\Lambda} [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n]^T = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n] \mathbf{\Lambda} [\mathbf{x}_1^T, \mathbf{x}_2^T, \dots, \mathbf{x}_n^T] = \sum_{i=1}^n \mathbf{x}_i \lambda_i \mathbf{x}_i^T = \sum_{i=1}^n \lambda_i \mathbf{x}_i \mathbf{x}_i^T$$

- The last equation gives the spectral representation of a symmetric matrix.

■ Deflation

- If we subtract from $\mathbf{A}$ one of the components, the resulting matrix, $\mathbf{A} - \lambda_1 \mathbf{x}_1 \mathbf{x}_1^T = \sum_{i=2}^n \lambda_i \mathbf{x}_i \mathbf{x}_i^T$, will have $n - 1$ eigenvalues, $\lambda_2, \lambda_3, \dots, \lambda_n$.

■ Power method with deflation

- After finding the eigenvector $\mathbf{x}_1$ and the corresponding eigenvalue λ_1 with the power method, one can set $\mathbf{A} = \mathbf{A} - \lambda_1 \mathbf{x}_1 \mathbf{x}_1^T$ and continue to find the next dominant eigenpair and so on.

```
A=np.array([[-7,13,-16],[13,-10,13],[-16,13,-7]], dtype=float)

E,V=np.linalg.eig(A)
print('Exact eigenvalues:',E)
```

```
Exact eigenvalues: [-36.   9.   3.]
```

```
# first eigenvalue and eigenvector
a1=E[0]; v1=V[:,0]

print(a1,v1)

# Deflate
A=A-a1*np.outer(v1,v1)
A
```

```
-36.0 [ 0.57735027 -0.57735027  0.57735027]
```

```
array([[ 5.,  1., -4.],
       [ 1.,  2.,  1.],
       [-4.,  1.,  5.]])
```

```
E,V=np.linalg.eig(A)
print('Exact eigenvalues:',E)
```

```
Exact eigenvalues: [9.00000000e+00 6.22435621e-15 3.00000000e+00]
```

```
a2=E[0]; v2=V[:,0]; A=A-a2*np.outer(v2,v2); print(A)
E,V=np.linalg.eig(A)
print('Exact eigenvalues:',E)
```

```
[[0.5 1.  0.5]
 [1.  2.  1. ]
 [0.5 1.  0.5]]
```

```
Exact eigenvalues: [3.00000000e+00 7.86778624e-15 4.70678088e-15]
```

■ Repeating one more deflation will give a zero matrix

■ Power method with deflation

```
A=np.array([[-7,13,-16],[13,-10,13],[-16,13,-7]], dtype=float)

print(A, '\n')

for i in range(n):
    NumIter,x0=power(A)
    eigenvalue = (x0.T @ A @ x0) / (x0.T @ x0)
    print(f"Iteration: {NumIter:3d}, Eigenvector: {x0}, Eigenvalue: {eigenvalue:12.8f}")
    if i < n-1:
        A=A-eigenvalue*np.outer(x0,x0)
        print(A, '\n')
```

```
[[ -7.  13. -16.]
 [ 13. -10.  13.]
 [-16.  13. -7.]]
```

Power method converged

Iteration: 9, Eigenvector: [0.57735027 -0.57735027 0.57735027], Eigenvalue: -36.00000000

```
[[ 4.99999999  1.          -4.00000001]
 [ 1.          2.00000002  1.          ]
 [-4.00000001  1.          4.99999999]]
```

Power method converged

Iteration: 2, Eigenvector: [0.40824829 0.81649658 0.40824829], Eigenvalue: 3.00000000

```
[[ 4.50000000e+00  3.44169138e-15 -4.50000000e+00]
 [ 3.44169138e-15 -2.66453526e-15  2.66453526e-15]
 [-4.50000000e+00  2.66453526e-15  4.50000000e+00]]
```

Power method converged

Iteration: 3, Eigenvector: [7.07106781e-01 6.10591512e-17 -7.07106781e-01], Eigenvalue: 9.00000000

- Note that removing one component does not make the resulting matrix block-diagonal as we are removing an eigenvector and not a coordinate in which the matrix is represented.

Jacobi rotation

■ 2x2 problem with analytic eigenvalues and eigenvectors

◦ Suppose we have $\mathbf{X}^T \mathbf{A} \mathbf{X} = \mathbf{\Lambda}$.

◦ Since $\mathbf{X}$ is an orthogonal matrix, we can express it as $\mathbf{X} = \begin{bmatrix} \cos \theta & \sin \theta \\ \sin \theta & -\cos \theta \end{bmatrix}$. It is easy to verify that $\mathbf{X}^T \mathbf{X} = \mathbf{I}$

◦ We can express $\mathbf{X}^T \mathbf{A} \mathbf{X}$ as

$$\begin{aligned} \mathbf{X}^T \mathbf{A} \mathbf{X} &= \begin{bmatrix} \cos \theta & \sin \theta \\ \sin \theta & -\cos \theta \end{bmatrix} \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} \cos \theta & \sin \theta \\ \sin \theta & -\cos \theta \end{bmatrix} \\ &= \begin{bmatrix} A_{11} \cos^2 \theta + A_{22} \sin^2 \theta + A_{12} \sin 2\theta & \frac{1}{2}(A_{11} - A_{22})\sin 2\theta - A_{12} \cos 2\theta \\ \frac{1}{2}(A_{11} - A_{22})\sin 2\theta - A_{12} \cos 2\theta & A_{11} \sin^2 \theta + A_{22} \cos^2 \theta - A_{12} \sin 2\theta \end{bmatrix} \end{aligned}$$

◦ This matrix will be diagonal if the off-diagonal element vanishes, $\frac{1}{2}(A_{11} - A_{22})\sin 2\theta - A_{12} \cos 2\theta = 0$

$$\implies \frac{1}{2}(A_{11} - A_{22})\sin 2\theta = A_{12} \cos 2\theta \implies \frac{\sin 2\theta}{\cos 2\theta} = \tan 2\theta = \frac{2A_{12}}{A_{11} - A_{22}} \implies \theta_0 = \frac{1}{2} \tan^{-1} \frac{2A_{12}}{A_{11} - A_{22}}$$

◦ The two eigenvalues are

$$\lambda_1 = A_{11} \cos^2 \theta_0 + A_{22} \sin^2 \theta_0 + A_{12} \sin 2\theta_0 \text{ and } \lambda_2 = A_{11} \sin^2 \theta_0 + A_{22} \cos^2 \theta_0 - A_{12} \sin 2\theta_0$$

◦ The corresponding eigenvectors are the columns of $\mathbf{X} = \begin{bmatrix} \cos \theta_0 & \sin \theta_0 \\ \sin \theta_0 & -\cos \theta_0 \end{bmatrix}$

■ *Implementation for 2x2 case*

```
def jacobi_2x2(A):  
    t0=(1/2) * np.arctan(2*A[0,1]/(A[0,0]-A[1,1]))  
    X=np.array(  
        [np.cos(t0),np.sin(t0)],  
        [np.sin(t0),-np.cos(t0)]]  
    )  
    return X
```

```
A=np.array([[ -7,13],[13,-10]], dtype=float)
```

```
E,V=np.linalg.eig(A)  
print('Eigenvalues:',E)  
print('Eigenvectors:'+'\n',V)
```

```
Eigenvalues: [  4.58625233 -21.58625233]  
Eigenvectors:  
[[ 0.74653336 -0.66534799]  
 [ 0.66534799  0.74653336]]
```

```
X=jacobi_2x2(A)  
L=X.T@A@X  
print('Eigenvalues:'+'\n',L)  
print('Eigenvectors:'+'\n',X)
```

```
Eigenvalues:  
[[ 4.58625233e+00  4.62634113e-16]  
 [ 2.32626074e-15 -2.15862523e+01]]  
Eigenvectors:  
[[ 0.74653336  0.66534799]  
 [ 0.66534799 -0.74653336]]
```

■ *n × n problem*

- For a general symmetric eigenvalue problem, we will apply the Jacobi rotation through a sequence of planar rotations to eliminate the off-diagonal elements.
- Suppose the largest off-diagonal element is $A_{r,s'}$, then we will define a planar rotation matrix as

$$\mathbf{Q}^{(1)}(i_1, j_1, \theta_1) = \begin{bmatrix} 1 & & & & & \\ & \ddots & & & & \\ & & \cos \theta & \cdots & \sin \theta & \\ & & \vdots & \ddots & \vdots & \\ & & \sin \theta & \cdots & -\cos \theta & \\ & & & & & \ddots & \\ & & & & & & 1 \end{bmatrix}$$

- Find the next largest off-diagonal element of the rotated matrix $\mathbf{A}^{(1)} = \mathbf{Q}^{(1)}\mathbf{A}\mathbf{Q}^{(1)}$ and build the corresponding planar rotation matrix $\mathbf{Q}^{(2)}(i_2, j_2, \theta_2)$.
- Upon several such rotations, the matrix $\mathbf{A}^{(m)}$ becomes a diagonal matrix, while the eigenvector matrix (or the similarity transformation matrix) is defined as

$$\mathbf{X} = \mathbf{Q}^{(m)}(i, j_m, \theta_m) \cdot \dots \cdot \mathbf{Q}^{(2)}(i_2, j_2, \theta_2) \cdot \mathbf{Q}^{(1)}(i_1, j_1, \theta_1), \text{ where } m \text{ is a large number}$$

- To generalize, $\mathbf{X} = \lim_{m \rightarrow \infty} \prod_{k=1}^m \mathbf{Q}(i_k, j_k, \theta_k)$.

```
def jacobi_rotation(A):
    n = A.shape[0]

    # largest off-diagonal entry, upper triangle only
    B = np.triu(np.abs(A), k=1)
    i, j = np.unravel_index(np.argmax(B), A.shape)
    maxoff = np.abs(A[i,j])

    if A[i, i] == A[j, j]:
        t0 = np.pi / 4
    else:
        t0 = 0.5 * np.arctan2(2 * A[i, j], A[i, i] - A[j, j])

    c = np.cos(t0); s = np.sin(t0)

    Q = np.eye(n); Q[i, i], Q[i, j], Q[j, i], Q[j, j] = c, s, s, -c

    return Q, (i, j), maxoff, t0
```

```
A = np.array([[-7, 13, -16], [13, -10, 13], [-16, 13, -7]], dtype=float)
```

```
Q1, idx1, maxoff, t1 = jacobi_rotation(A)
A1 = Q1.T @ A @ Q1
print("first pivot:", idx1, maxoff)
print(A1)

Q2, idx2, maxoff, t2 = jacobi_rotation(A1)
A2 = Q2.T @ A1 @ Q2
print("second pivot:", idx2, maxoff)
print(A2)

Q3, idx3, maxoff, t3 = jacobi_rotation(A2)
A3 = Q3.T @ A2 @ Q3
print("third pivot:", idx3, maxoff)
print(A3)
```

```
first pivot: (0, 2) 16.0
[[-2.30000000e+01  1.83847763e+01  2.48504198e-15]
 [ 1.83847763e+01 -1.00000000e+01 -9.99200722e-16]
 [ 2.88823576e-15 -9.99200722e-16  9.00000000e+00]]
second pivot: (0, 1) 18.384776310850235
[[ 3.00000000e+00  1.00429842e-15  6.18895682e-16]
 [ 2.00752447e-15 -3.60000000e+01  2.60591708e-15]
 [ 8.51679723e-16  2.93512343e-15  9.00000000e+00]]
third pivot: (1, 2) 2.6059170847294504e-15
[[ 3.00000000e+00  6.18895682e-16  1.00429842e-15]
 [ 8.51679723e-16  9.00000000e+00  1.79668135e-16]
 [ 2.00752447e-15 -1.49538213e-16 -3.60000000e+01]]
```

```

def jacobi_rotation(A):
    n = A.shape[0]

    # largest off-diagonal entry, upper triangle only
    B = np.triu(np.abs(A), k=1) # k = 1 means exclude the main diagonal
    i, j = np.unravel_index(np.argmax(B), A.shape)
    maxoff = np.abs(A[i,j])

    if A[i, i] == A[j, j]:
        t0 = np.pi / 4
    else:
        t0 = 0.5 * np.arctan2(2 * A[i, j], A[i, i] - A[j, j])

    c = np.cos(t0); s = np.sin(t0)

    Q = np.eye(n); Q[i, i], Q[i, j], Q[j, i], Q[j, j] = c, s, s, -c

    return Q, (i, j), maxoff, t0

def jacobi_seq(A, MaxIter=1000, MaxTol=1e-8):
    n = A.shape[0]; MaxOff=np.inf; NumIter=0; X=np.eye(n)

    while NumIter < MaxIter and MaxOff > MaxTol:
        NumIter+=1
        Q, idx1, MaxOff, t1 = jacobi_rotation(A)
        A = Q.T @ A @ Q
        X = X @ Q

    return A, X, NumIter

```

```

A = np.array([[ -7, 13, -16], [13, -10, 13], [-16, 13, -7]], dtype=float)
L,X,NumIter=jacobi_seq(A)
print('Eigenvalues: '+'\n',np.diag(L))
print('Eigenvectors: '+'\n',X)
print('Iterations:',NumIter)

```

```

Eigenvalues:
[ 3.  9. -36.]
Eigenvectors:
[[ 4.08248290e-01  7.07106781e-01  5.77350269e-01]
 [ 8.16496581e-01 -3.53525080e-17 -5.77350269e-01]
 [ 4.08248290e-01 -7.07106781e-01  5.77350269e-01]]
Iterations: 3

```

■ Simplifying the updates

◦ $\mathbf{X}_{\text{new}} = \mathbf{X}\mathbf{Q}$ (which scales as $\mathcal{O}(N^3)$) can be simplified as

$$\begin{aligned} X_{\text{new}}[:, i] &= cX[:, i] + sX[:, j] \\ X_{\text{new}}[:, j] &= sX[:, i] - cX[:, j] \end{aligned}$$

◦ $\mathbf{A}_{\text{new}} = \mathbf{Q}\mathbf{A}\mathbf{Q}$ (which scales as $\mathcal{O}(N^3)$) can be simplified as follows

◦ $a_{ii} = A[i, i]$, $a_{ij} = A[i, j]$, $a_{jj} = A[j, j]$

◦ $A[i, i] = c^2 a_{ii} + 2csa_{ij} + s^2 a_{jj}$

◦ $A[i, j] = csa_{ii} + (s^2 - c^2)a_{ij} - csa_{jj}$

◦ $A[j, i] = A[i, j]$

◦ $A[j, j] = s^2 a_{ii} - 2csa_{ij} + c^2 a_{jj}$

◦ For $k \neq i$; $k \neq j$

$$A[k, i] = ca_{ki} + sa_{kj}, \quad A[i, k] = A[k, i]$$

$$A[k, j] = sa_{ki} - ca_{kj}, \quad A[j, k] = A[k, j]$$

```

def jacobi_seq(A, MaxIter=1000, MaxTol=1e-8):
    n = A.shape[0]; MaxOff=np.inf; NumIter=0; X=np.eye(n)

    while NumIter < MaxIter and MaxOff > MaxTol:
        NumIter+=1

        Q, idx1, MaxOff, t1 = jacobi_rotation(A)
        i,j=idx1

        #X = X @ Q
        X=X.copy()
        c=np.cos(t1)
        s=np.sin(t1)

        for k in range(n):
            X[k, i] = c*X[k, i] + s*X[k, j]
            X[k, j] = s*X[k, i] - c*X[k, j]

        #A = Q.T @ A @ Q
        aii=A[i,i]; aij=A[i,j]; ajj=A[j,j]
        A[i,i]=c**2 * aii + s**2 * ajj + 2*c*s*aij
        A[i,j]=c*s*(aii-ajj) + (s**2-c**2)*aij
        A[j,i]=A[i,j]
        A[j,j]=s**2 * aii + c**2 * ajj - 2*c*s*aij

        for k in range(n):
            if k != i and k != j:
                aki = A[k, i]; akj = A[k, j]
                A[k,i]=c*aki+s*akj
                A[i,k]=A[k,i]
                A[k,j]=s*aki-c*akj
                A[j,k]=A[k,j]

    return A, X, NumIter

```

```

Eigenvalues:
[ 3.  9. -36.]
Eigenvectors:
[[ 4.08248290e-01  7.07106781e-01  5.77350269e-01]
 [ 8.16496581e-01 -3.53525080e-17 -5.77350269e-01]
 [ 4.08248290e-01 -7.07106781e-01  5.77350269e-01]]
Iterations: 3

```

```

A = np.array([[-7, 13, -16], [13, -10, 13], [-16, 13, -7]], dtype=float)
L,X,NumIter=jacobi_seq(A)
print('Eigenvalues: '+'\n',np.diag(L))
print('Eigenvectors: '+'\n',X)
print('Iterations:',NumIter)

```